



國家實驗研究院
國家高速網路與計算中心
National Center for High-performance Computing

NVIDIA Sharing

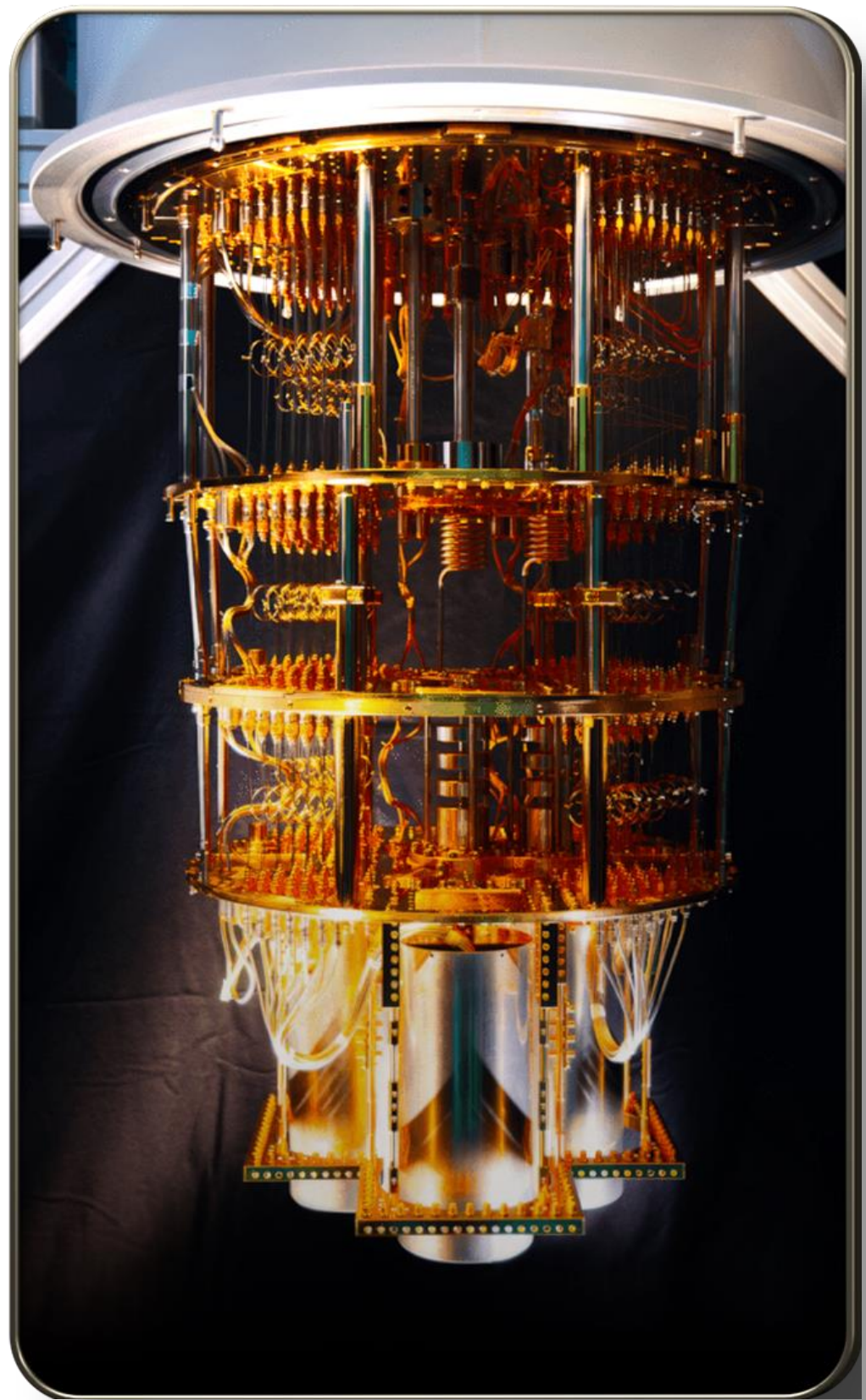
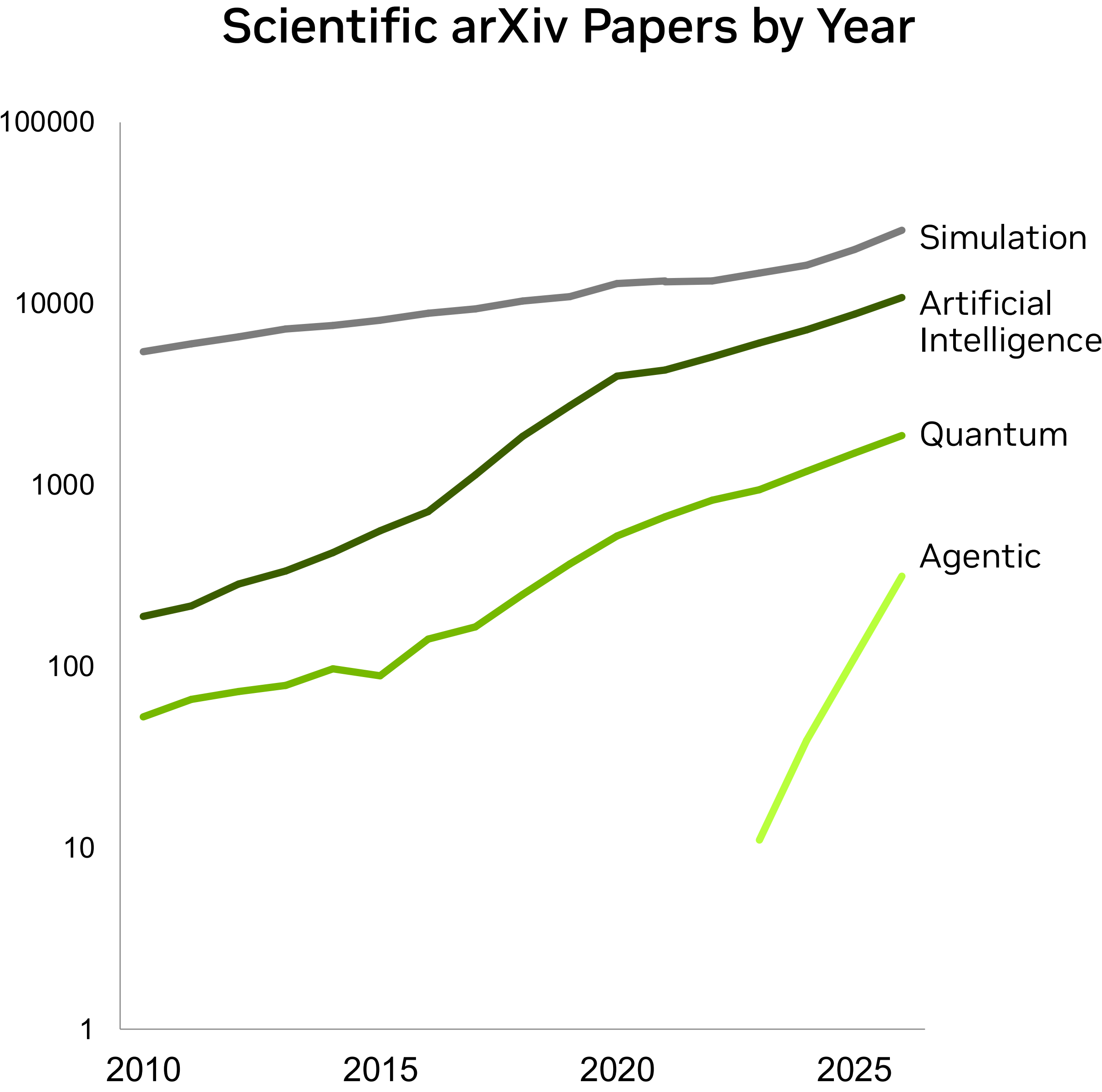
- Quantum - Pika Wang
- LLM – Cliff Chiu
- HPC & AI Physics – Jay Chen

July 10 | HiPAC 2026



Scientific Computing at an Inflection Point

From One to Four Pillars of Scientific Computing



Quantum



Agentic & LLM

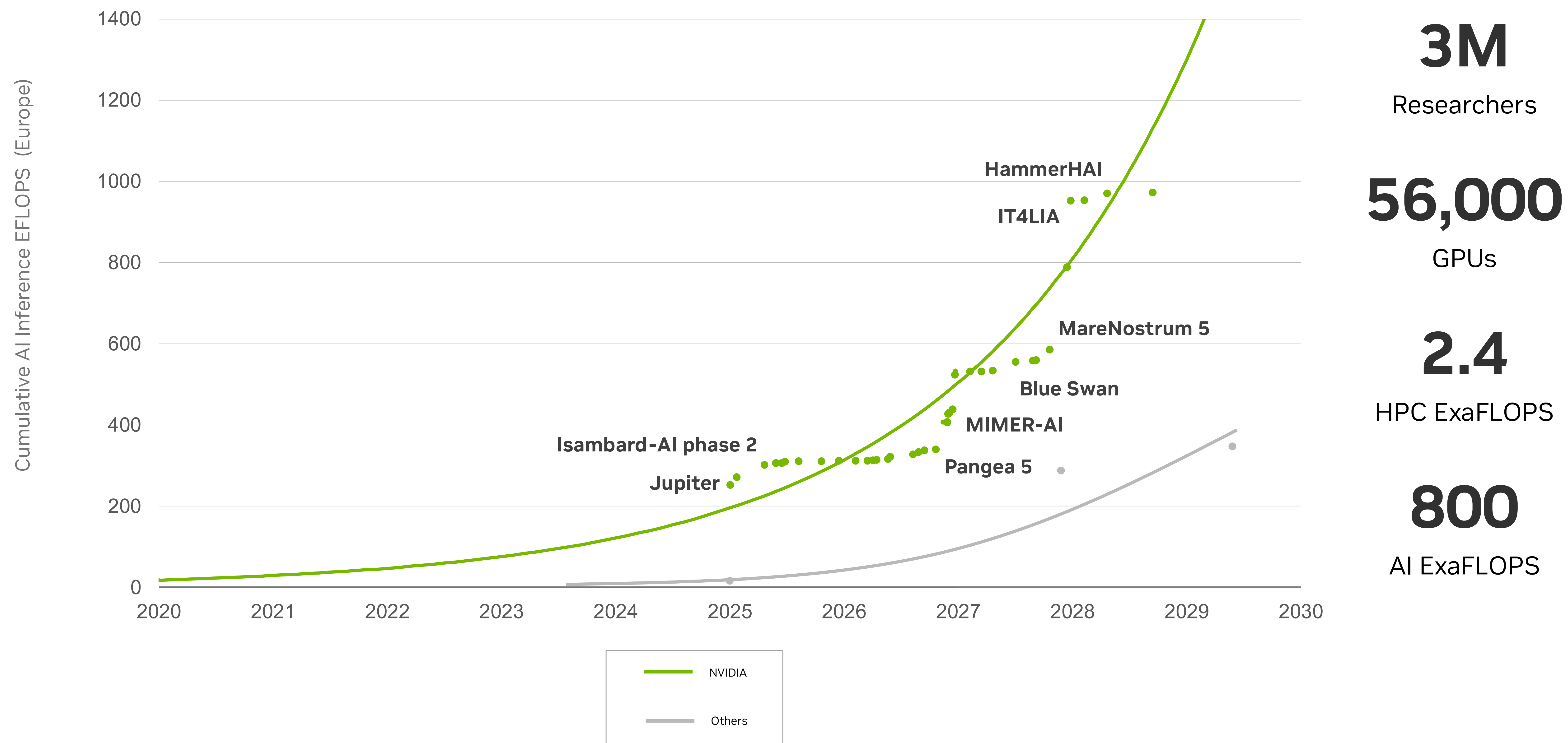


Simulation



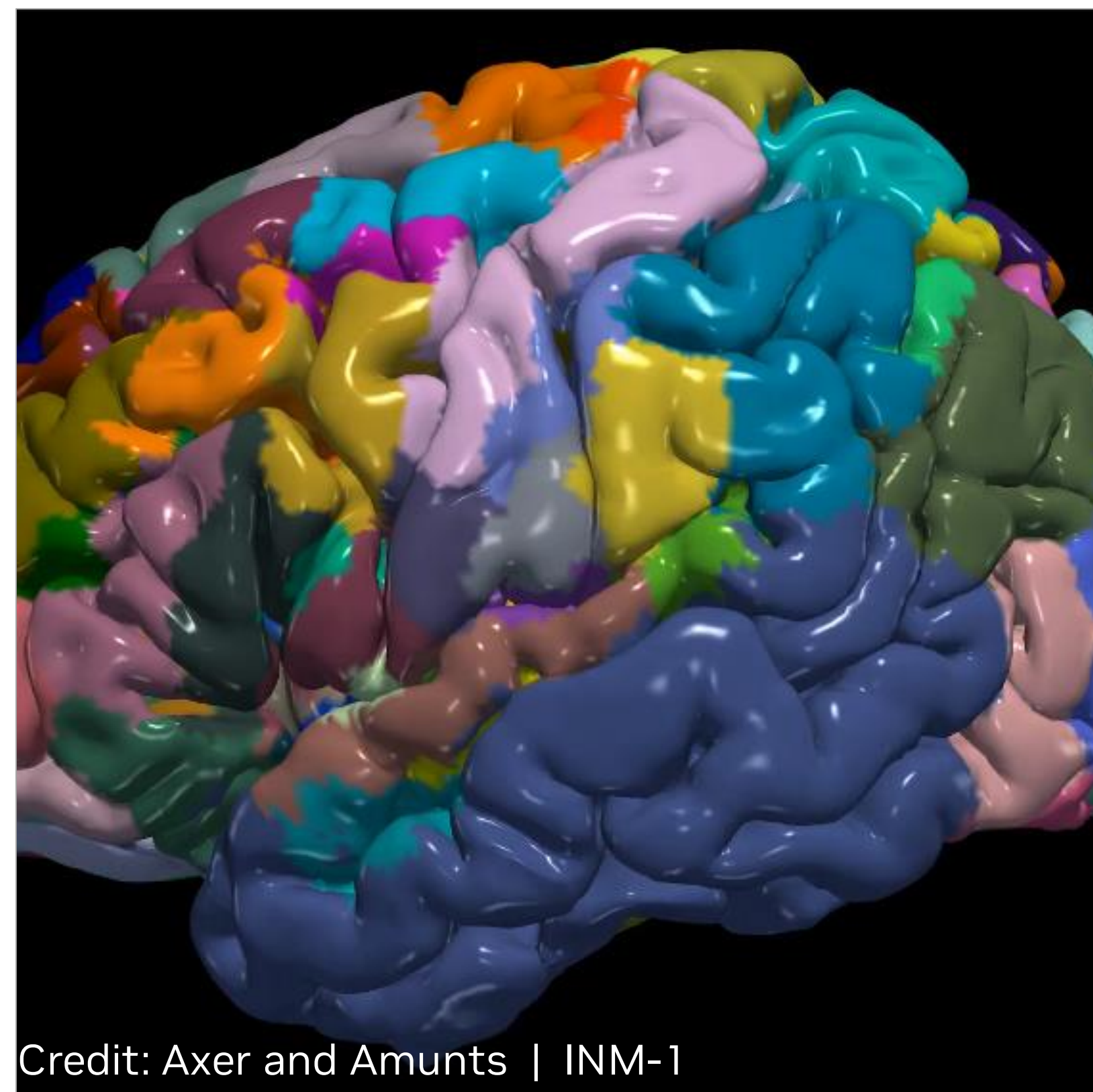
AI for Science

Europe Unveils a Record 35 New NVIDIA AI Supercomputers in One Year



Jülich JUPITER Supercharging Sciences

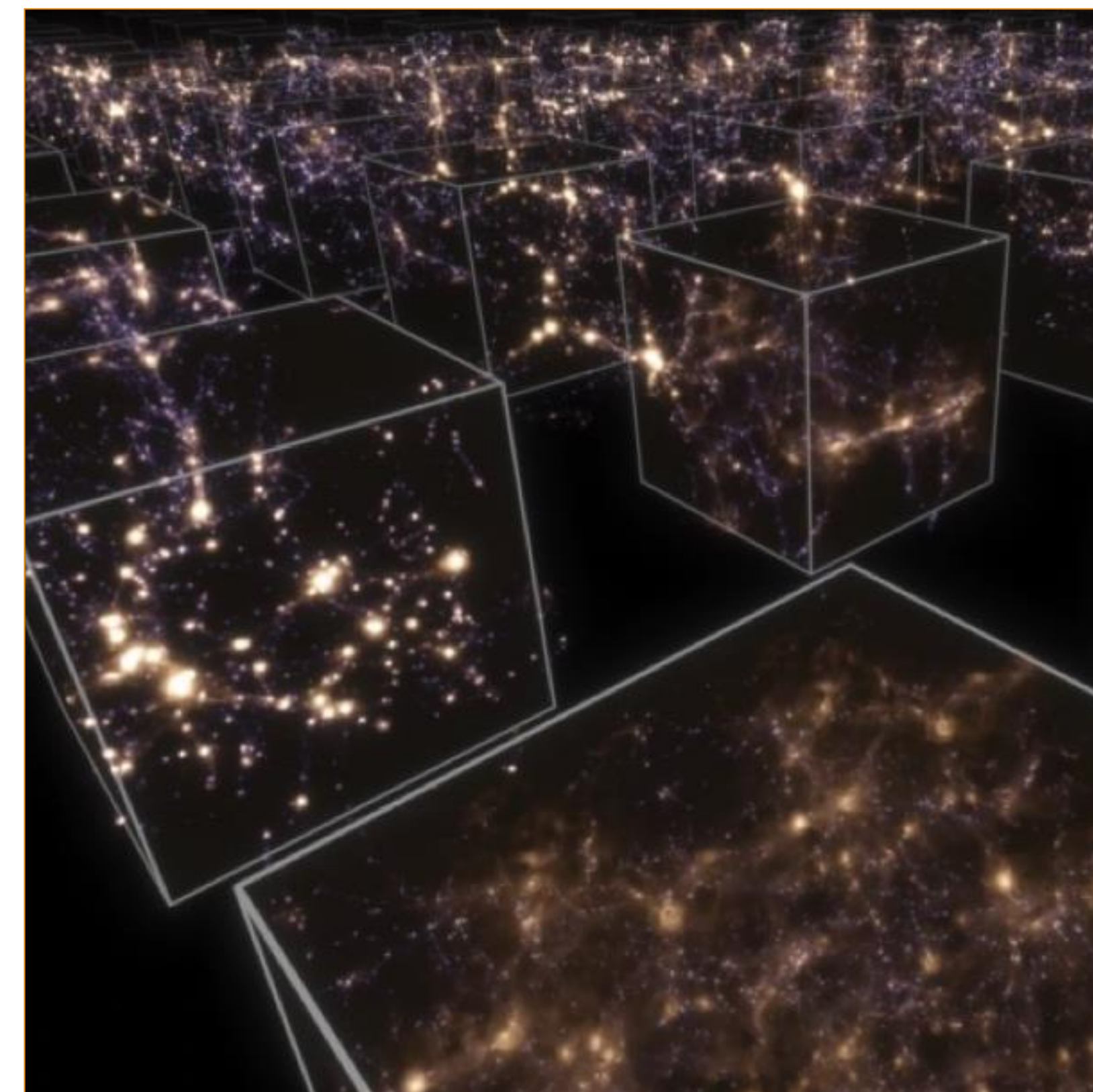
Europe's largest supercomputer powering discovery in healthcare, quantum, telecommunications and climate



Credit: Axer and Amunts | INM-1

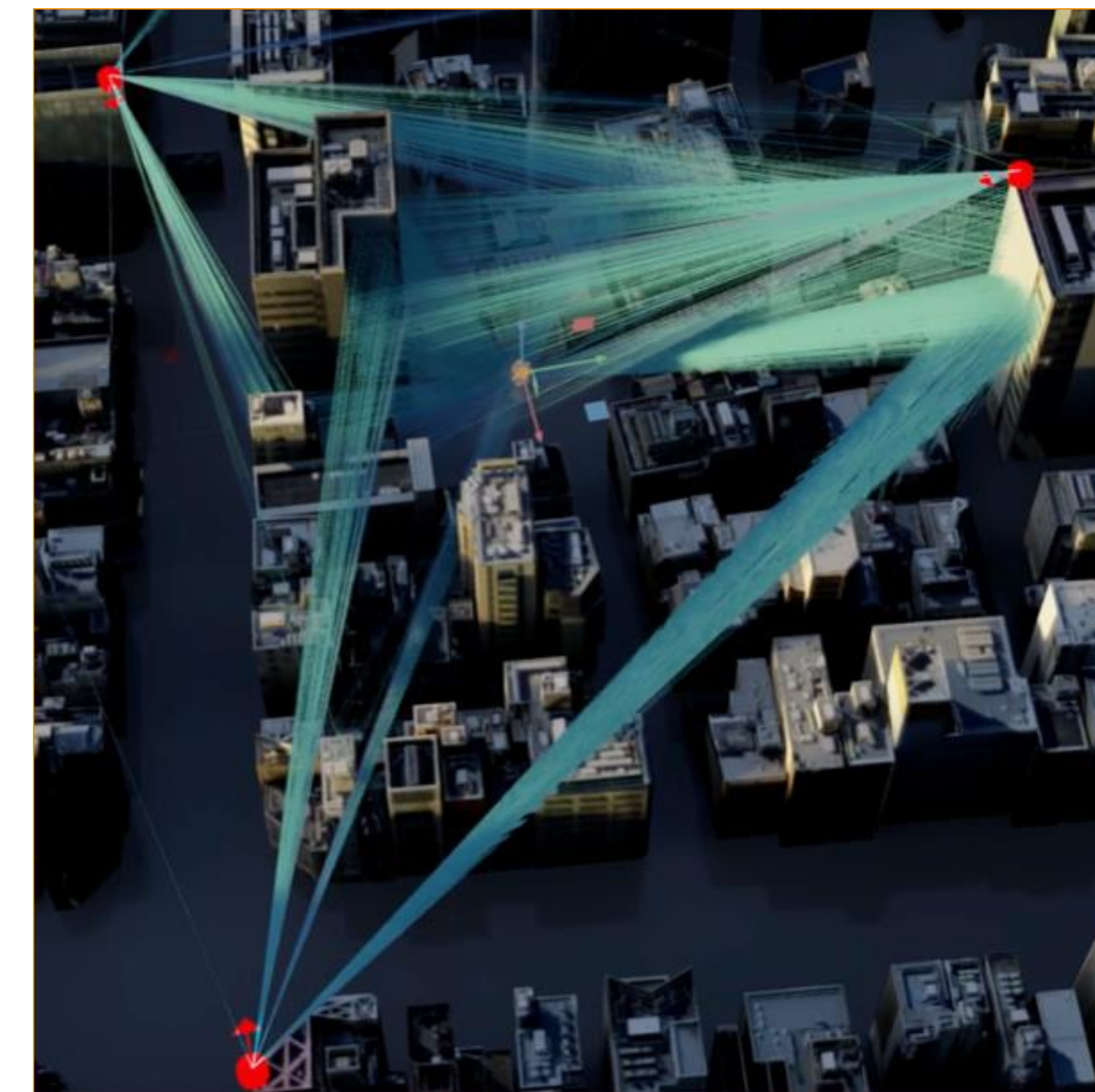
First 3-D Atlas for the Human Brain

An intelligent agent for brain research



Quantum Simulation

State-vector simulation of
Shor's algorithm at 50 qubits



6G Telecommunications

Neuromorphic computing at Ericsson



Climate Simulation

Largest and most complex climate simulation:
146 days per day at 1km resolution

High Performance, Energy-Efficient Systems in TOP500/GREEN500

NVIDIA powers 81% of Top500 and sweeps Green500 for performance and energy efficiency

Powering Worlds Fastest, Greenest Supercomputers

TOP500

The platform of choice

81%

use NVIDIA
(+17 vs SC25)

238

GPU systems
record high

89%

Of new systems
choose NVIDIA

Green500

Energy Efficiency

#1

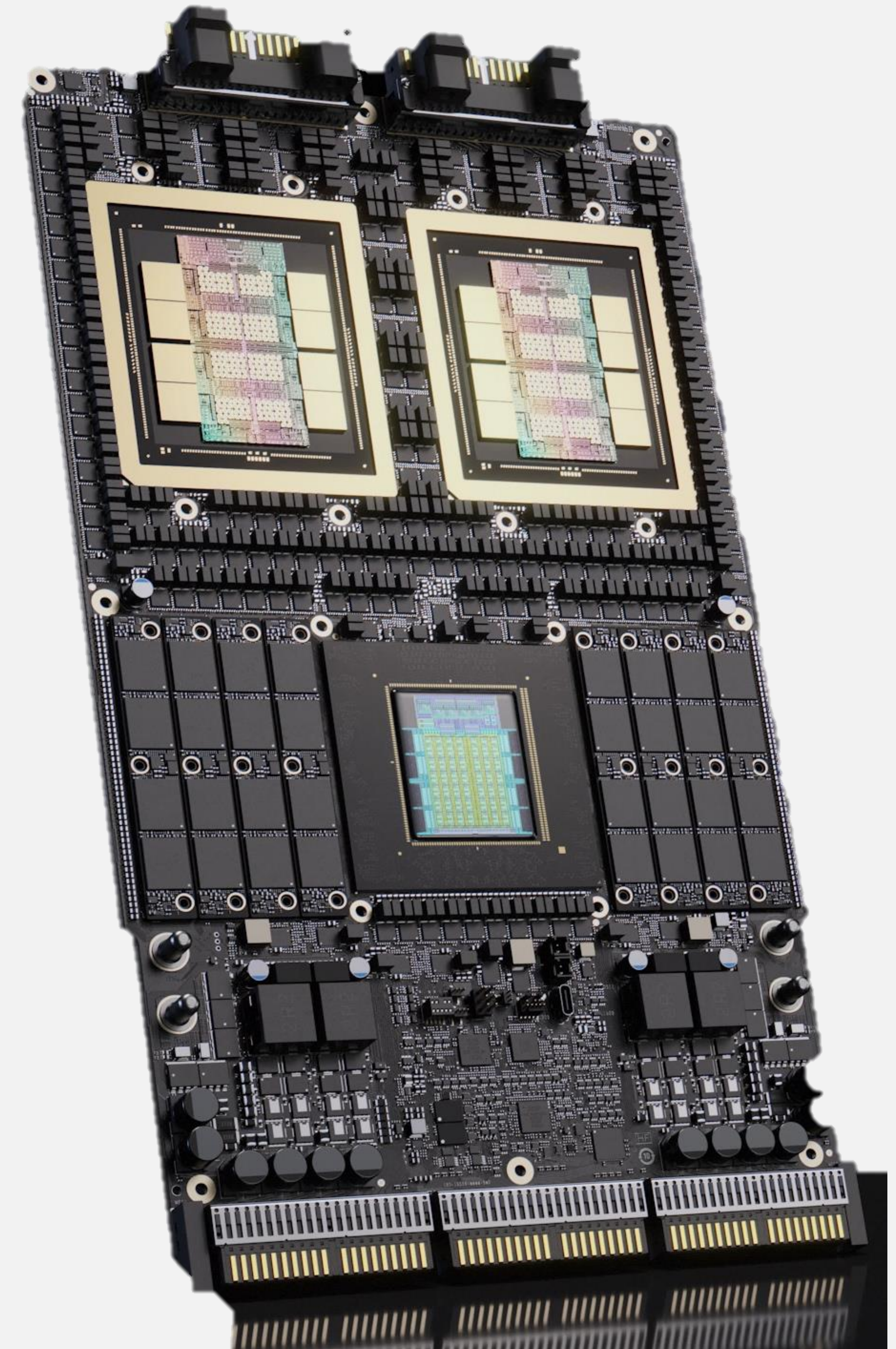
Green500
GH200 73GF/W

8

of the
Top 8 powered by
NVIDIA Hopper

68%

of the top 50
Green500 are
NVIDIA



20 Years of CUDA: The Foundation of Modern Science

Every Industry | One Architecture | CUDA Everywhere

7M

Developers

600M

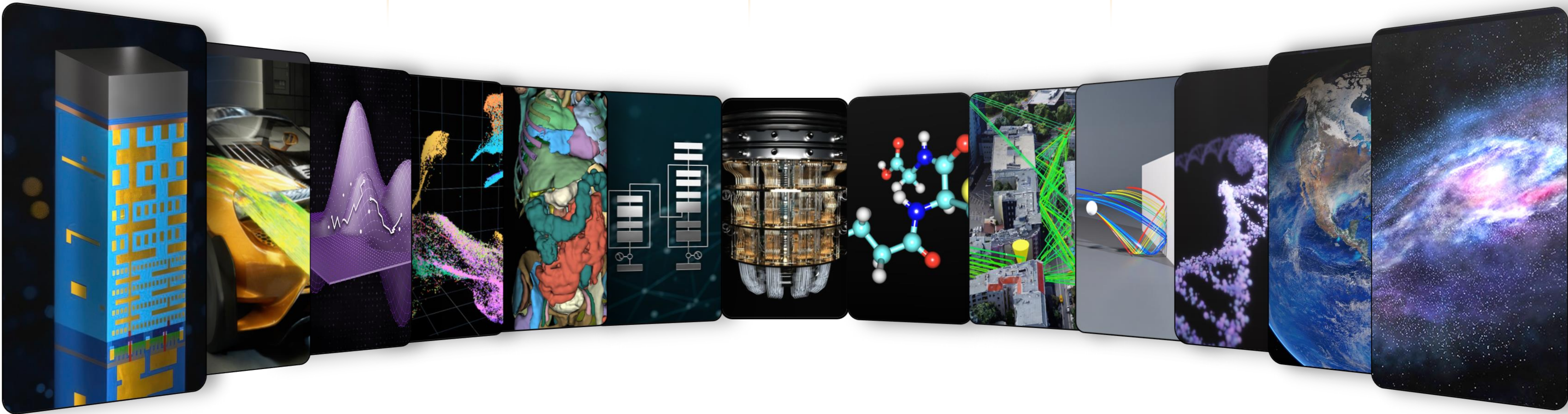
Downloads

6K

Accelerated
Applications

1K

SDKs and Models



cuLitho
Computational
Lithography

cuDSS
cuSPARSE
cuFFT
AmgX
Computer-Aided
Engineering

cuOpt
Decision
Optimization

cuDF
cuVS
cuML
cuGraph
Data Science
and Processing

MONAI
Medical
Imaging

Megatron
Dynamo
NIXL
cuDNN
CUTLASS
Deep
Learning

cuQuantum
CUDA-Q
Quantum
Computing

cuEST
cuEquivariance
cuTensor
ALCHEMI
Quantum
Chemistry

Aerial
Sionna
Holoscan
5G/6G
DAQIRI
Signal
Processing

Warp
PhysicsNeMo
Physics

Parabricks
Genomics

Earth-2
Weather
Analytics

cuPyNumeric
cuPhoton
Numerical
Computing

NVIDIA NGC

A portal of enterprise services, software, and support for AI, digital twins, and high-performance computing.
<https://catalog.ngc.nvidia.com/>

Container

 NVIDIA **NVIDIA cuQuantum Appliance**

The NVIDIA cuQuantum Appliance is a highly performant multi-GPU multi-node solution for quantum computing.

Latest (26.03-x86_64) Signed Scanned

OverviewTagsLayersSecurity Scanning

NVIDIA cuQuantum Appliance

The NVIDIA cuQuantum Appliance is a highly performant multi-GPU multi-node solution for quantum computing. It contains NVIDIA's cuStateVec, cuTensorNet, cuDensityVec, tensor network, analog quantum dynamics, Pauli simulator, and more. The cuQuantum Appliance provides the following simulators:

- IBM's Qiskit Aer frontend via cusvael, NVIDIA's distributed backend solver.
- An optimized multi-GPU Google Cirq frontend via qsim.

Container

 NVIDIA **CUDA-Q**

With a unified and open programming model, CUDA Quantum is a first-of-its-kind, open-source platform for integrated quantum-classical programming.

Get Container

Container

 NVIDIA **vLLM**

vLLM is a fast and easy-to-use library for LLM inference and serving. The NVIDIA vLLM is a community-driven project and contains a validated set of libraries that enable and optimize GPU performance.

Latest (26.06-py3) Signed Scanned

OverviewTagsLayersSecurity ScanningRelated Collections

vLLM

vLLM is a fast and easy-to-use library for LLM inference and serving. The NVIDIA vLLM is a community-driven project and contains a validated set of libraries that enable and optimize GPU performance.

NGC Containers are one of the easiest ways to get started with vLLM. The NVIDIA vLLM NGC Container is optimized for GPU acceleration and contains a validated set of dependencies included, providing an easy place to start developing applications for natural language processing (NLP), recommenders, and code generation.

Container

 SISSA **Quantum ESPRESSO**

Quantum ESPRESSO is an integrated suite of Open-Source computer codes for electronic-structure calculations at the nanoscale based on density-functional theory, plane waves, and pseudopotentials.

Latest (qe-7.3.1)

OverviewTagsLayersSecurity ScanningRelated Collections

Quantum ESPRESSO

Quantum ESPRESSO is an integrated suite of Open-Source computer codes for electronic-structure calculations at the nanoscale based on density-functional theory, plane waves, and pseudopotentials.

Quantum ESPRESSO has evolved into a distributed project. The Quantum ESPRESSO distribution includes the codes needed to perform more advanced tasks, plus several third-party codes contributed by Researchers active in the field of electronic-structure calculations, contributing their codes or by implementing new features.

Container

 NVIDIA **PyTorch**

PyTorch is a GPU accelerated tensor computational framework. Functionality can be extended with common NumPy and SciPy. Automatic differentiation is done with a tape-based system at the functional and neural network layer level.

NVIDIA AI Enterprise Supported

Latest (26.06-py3) Signed Scanned

OverviewTagsLayersSecurity ScanningRelated Collections

PyTorch

PyTorch is an optimized tensor library for deep learning using GPUs and CPUs. Automatic differentiation is done with a tape-based system at both a functional and neural network layer level. This functionality brings a deep learning framework and provides accelerated NumPy-like functionality. NGC Containers started with PyTorch. The PyTorch NGC Container comes with all dependencies included, providing an easy place to start developing common applications, such as conversational AI, natural language processing (NLP), vision, and more.

The PyTorch NGC Container is optimized for GPU acceleration, and contains a validated set of dependencies included, providing an easy place to start developing applications for natural language processing (NLP), recommenders, and code generation.

Container

 NVIDIA **NVIDIA HPC SDK**

The NVIDIA HPC SDK is a comprehensive suite of compilers, libraries and tools essential to maximizing developer productivity and the performance and portability of HPC applications.

Get Container

Container

 NVIDIA **PhysicsNeMo**

NVIDIA PhysicsNeMo is an open-source framework for building, training, and fine-tuning Physics-ML models.

Latest (26.06) Signed Scanned

OverviewTagsLayersSecurity ScanningRelated Collections

What is PhysicsNeMo?

NVIDIA PhysicsNeMo is a toolkit for developing AI enabled physics-ML applications.

With NVIDIA PhysicsNeMo, we aim to provide researchers and industry specialists, with various tools that will help accelerate your development of such models for the scientific discipline of your need.

Visit [NVIDIA PhysicsNeMo](#) for more information.

Visit [PhysicsNeMo Documentation](#) for documentation.

Running PhysicsNeMo Using Docker

If you have Docker 19.03 or later, a typical command to launch the container with an interactive bash terminal is:

Publisher

 NVIDIA

Latest Tag

26.06

Updated

June 27, 2026

Compressed Size

15.18 GB

Multinode Support

Yes



NVIDIA Brev

Scan the QRCode to gain US\$10 Credit

NVIDIA

GPUsAgentsLaunchablesTeamBillingDocs

Launchables

Deploy pre-configured GPU environments with one click. Share and discover turnkey templates for AI workflows and GPU-accelerated software.

Explore

My Launchables

quantum vllm tensorrt hpc pytorch physicsnemo

21 results found

AI-Powered-Physics-Bootcamp

by iauyeung-0

This Bootcamp will provide researchers hands-on approaches on how to use NVIDIA PhysicsNeMo, a...

\$0.70/hr

308

68

CUDA-Q

by jinsungk-0

No description available

\$3.67/hr

117

55

PyTorch NGC

by sbhavani-0

Useful for testing Transformer Engine

\$1.90/hr

14

6

QDW 2026 Workshop Materials

by mcansarihan-0

Shared QDW 2026 workshop environment and materials with Docker Compose, Palace, Qiskit...

\$0.33/hr

1940

456

Optimizing Inference with NVIDIA TensorRT-LLM

by jrodge

No description available

\$5.03/hr

60

3

Run Llama3 Inference with TRT-LLM

by amaddipoti

No description available

\$5.03/hr

60

3

Deploying NVIDIA Nemotron-3-Nano with TensorRT-LLM

Deploying NVIDIA Nemotron-3-Nano NVFP4 with vLLM

Deploying NVIDIA Nemotron 3 Nano Omni with vLLM

NVIDIA

\$10 Brev Credit

~300 NTD

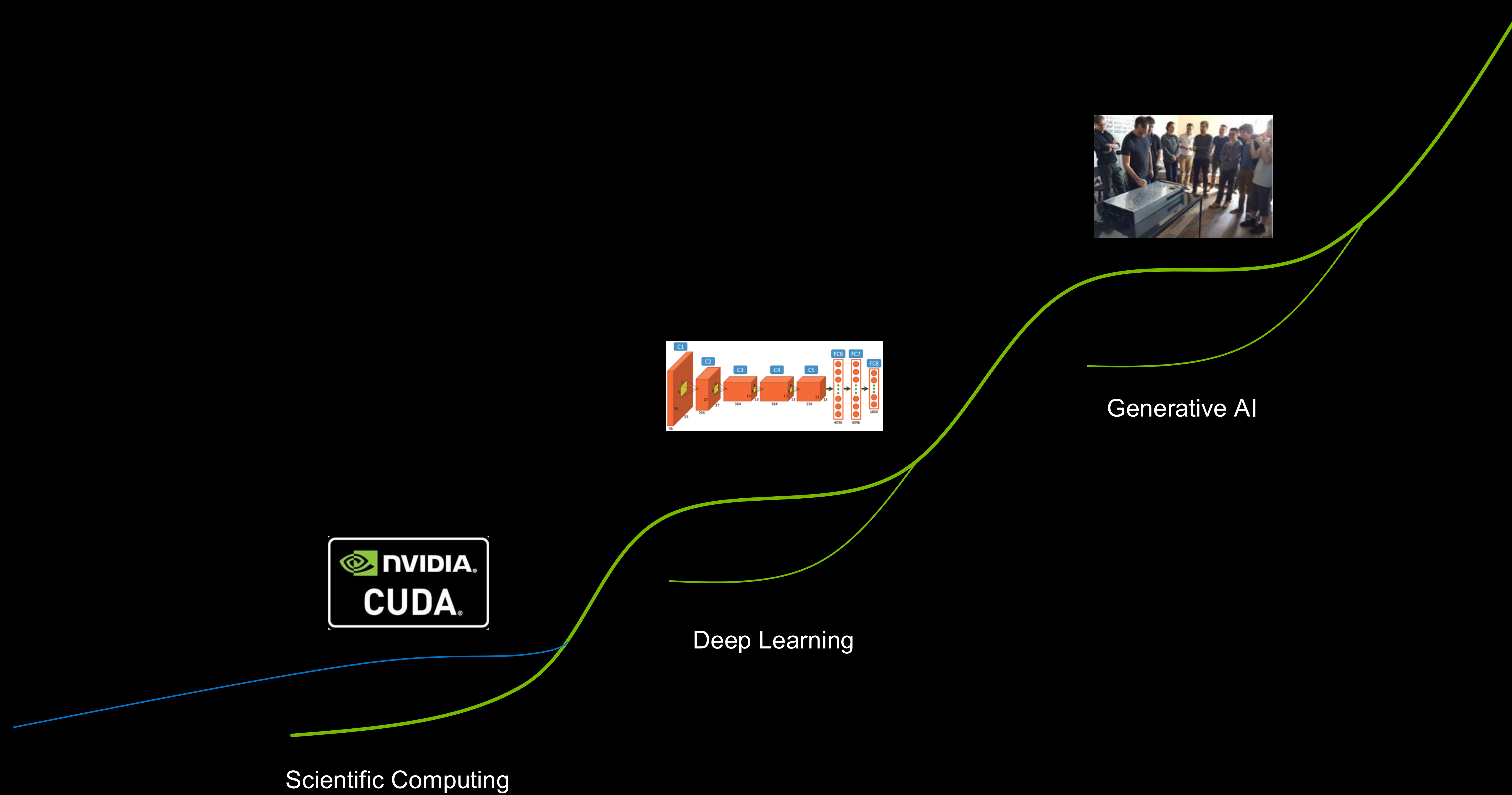
8

 NVIDIA

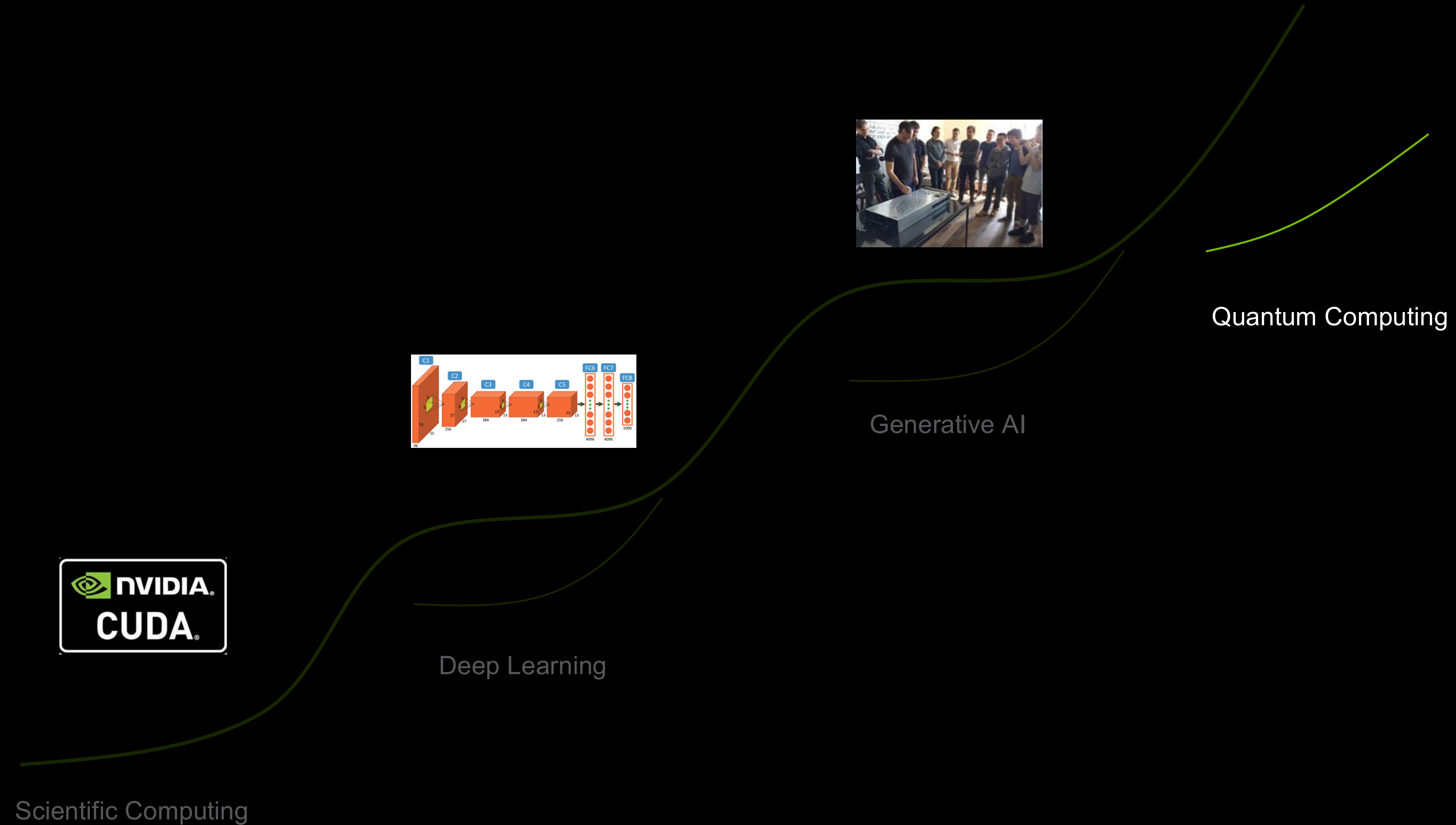
Building Quantum-GPU Supercomputing



NVIDIA's History of Enabling Computing Revolutions



NVIDIA's History of Enabling Computing Revolutions



Qubits and Coins

A simplified yet informal illustration

Bit

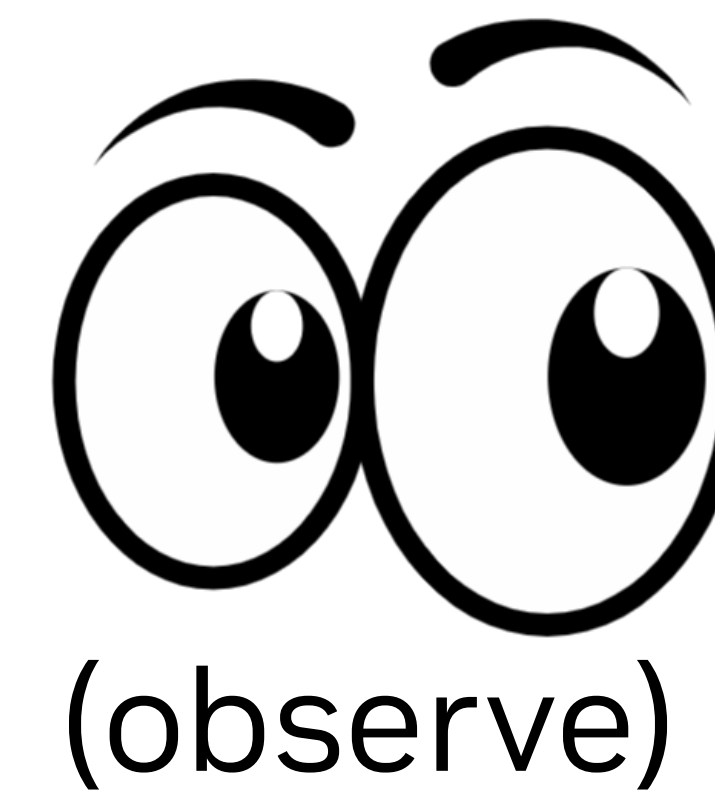


or

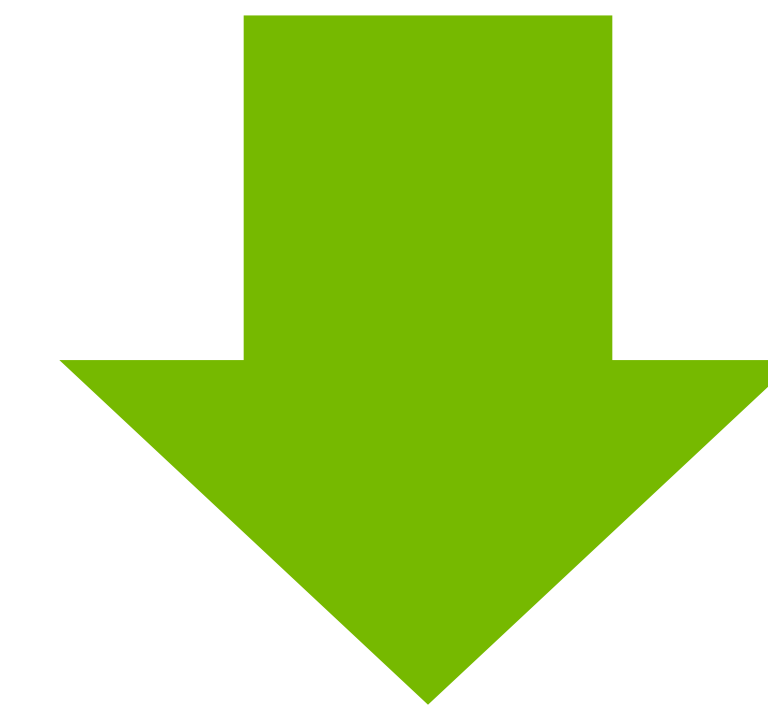
**During computation &
Computation completed**

Qubit

During computation



(observe)



Collapse



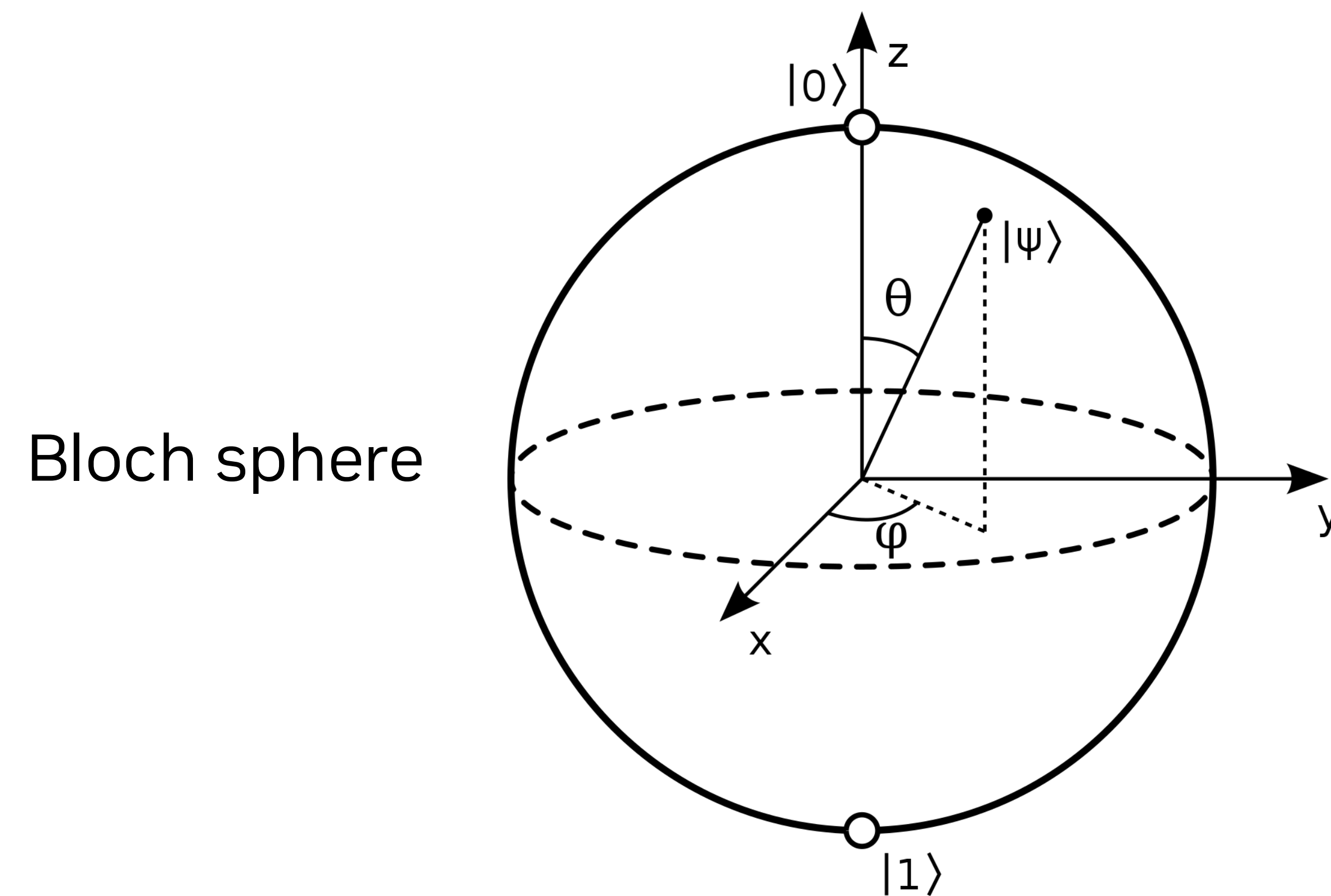
or

Computation completed

Qubits and Coins

Superposition – Let's dive into math

Single Qubit



Superposition of both 0 and 1

$$|\Psi\rangle = \cos(\theta/2)|0\rangle + e^{i\phi} \sin(\theta/2)|1\rangle$$

- Probability of measuring $|0\rangle$:

$$P(0) = |\cos(\theta/2)|^2 = \cos^2(\theta/2)$$

- Probability of measuring $|1\rangle$:

$$P(1) = |e^{i\phi} \sin(\theta/2)|^2 = \sin^2(\theta/2)$$

Since $\cos^2(\theta/2) + \sin^2(\theta/2) = 1$, this ensures a valid quantum probability distribution.

Multi Qubits

$$c_0|0\rangle + c_1|1\rangle = \begin{bmatrix} c_0 \\ c_1 \end{bmatrix}$$

$$c_{00}|00\rangle + c_{01}|01\rangle + c_{10}|10\rangle + c_{11}|11\rangle = \begin{bmatrix} c_{00} \\ c_{01} \\ c_{10} \\ c_{11} \end{bmatrix}$$

$$\vdots$$

$$c_{000}|000\rangle + c_{001}|001\rangle + c_{010}|010\rangle + c_{011}|011\rangle + c_{100}|100\rangle + c_{101}|101\rangle + c_{110}|110\rangle + c_{111}|111\rangle$$

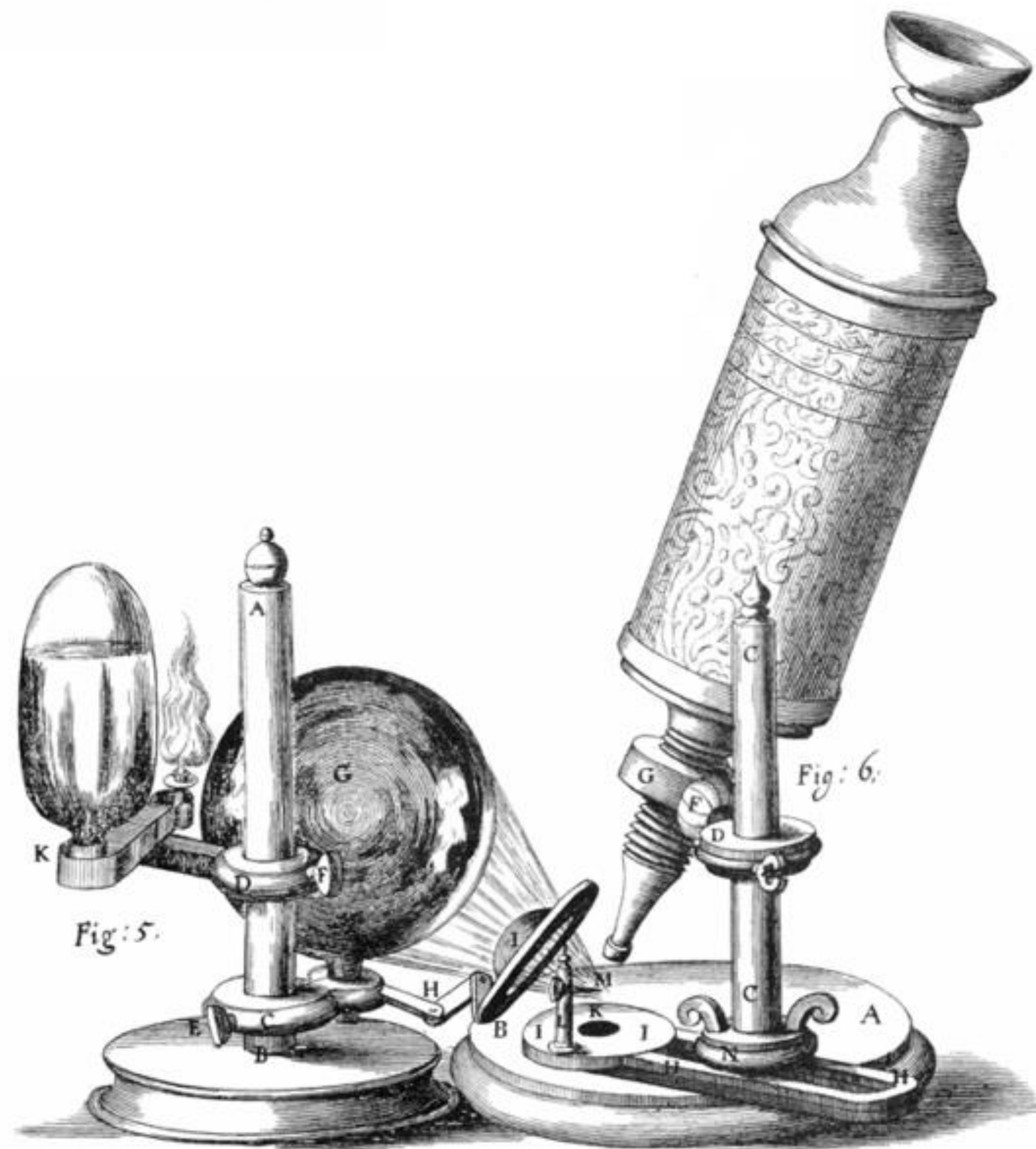
$$= \begin{bmatrix} c_{000} \\ c_{001} \\ c_{010} \\ c_{011} \\ c_{100} \\ c_{101} \\ c_{110} \\ c_{111} \end{bmatrix}$$

N qubits can represent 2^N states simultaneously

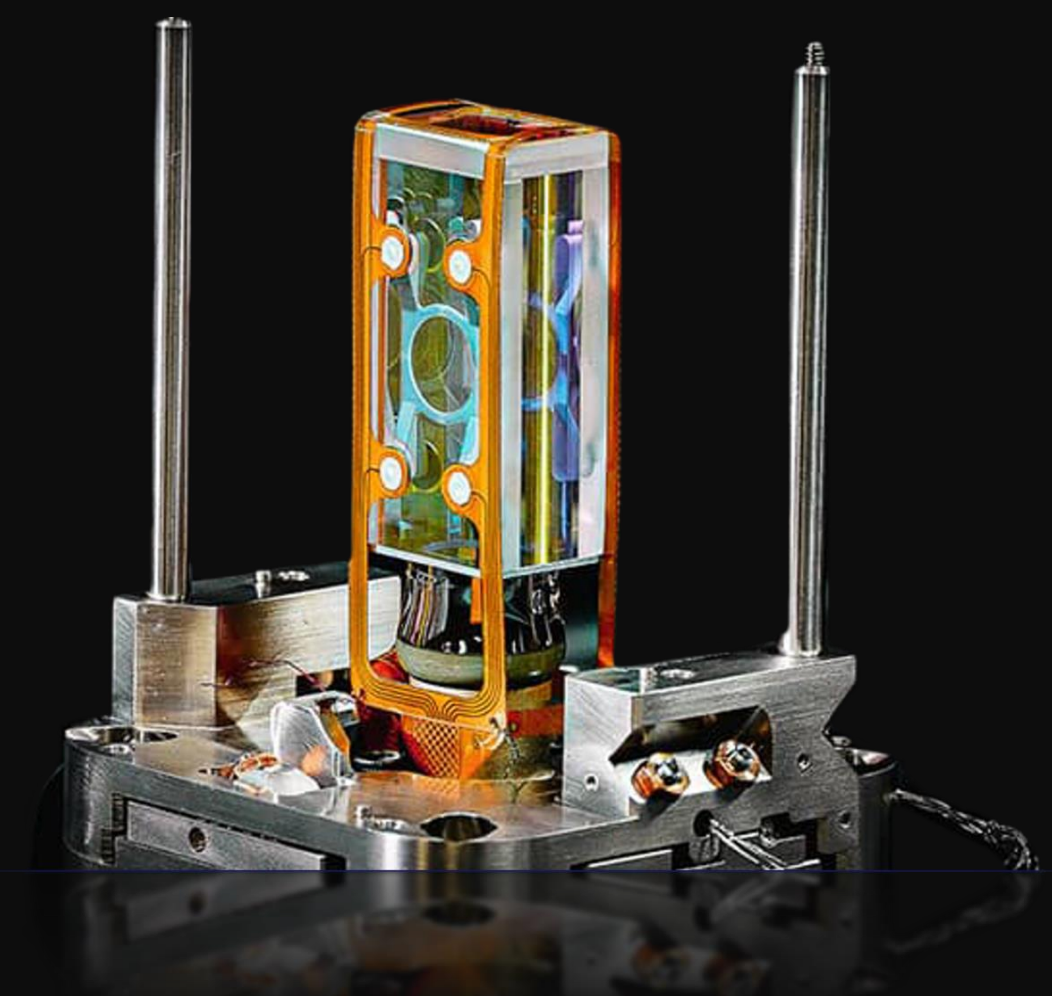
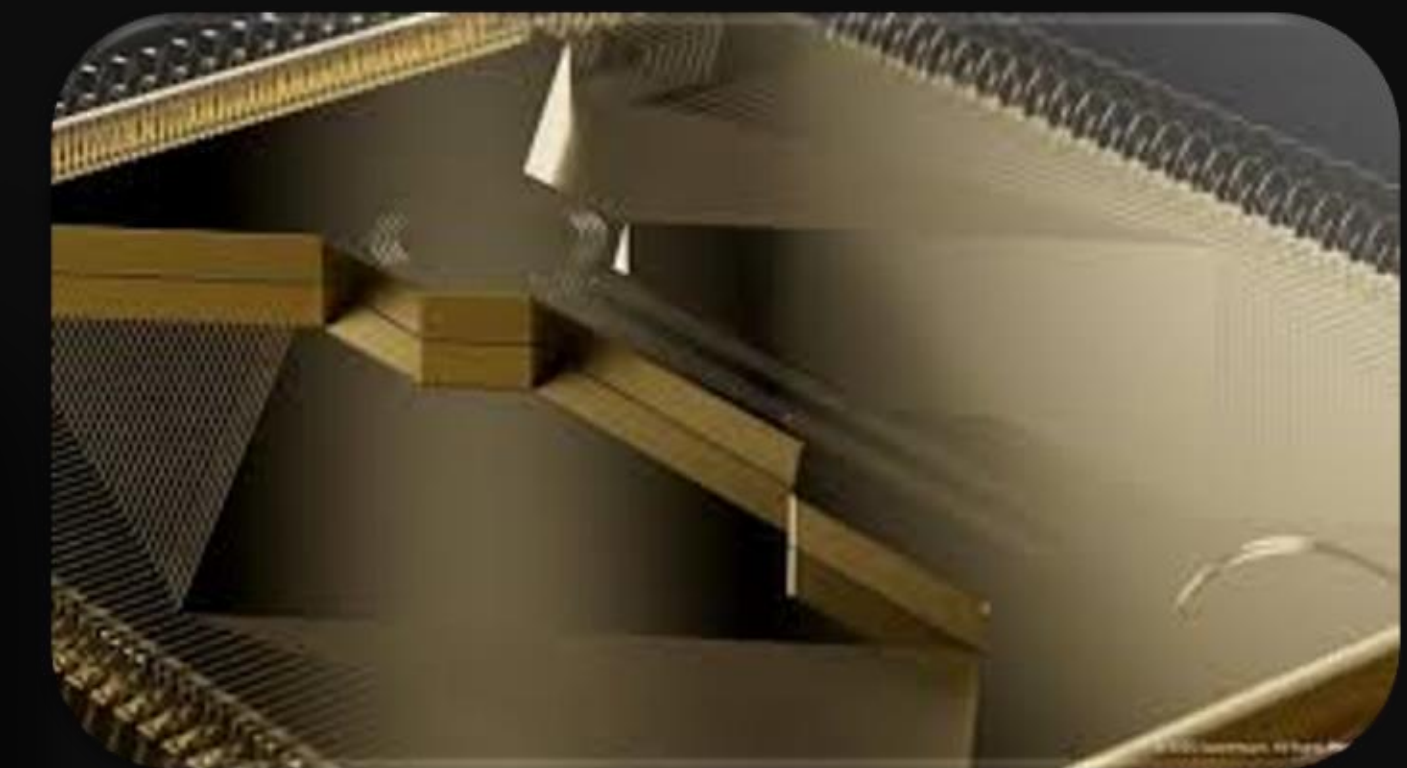
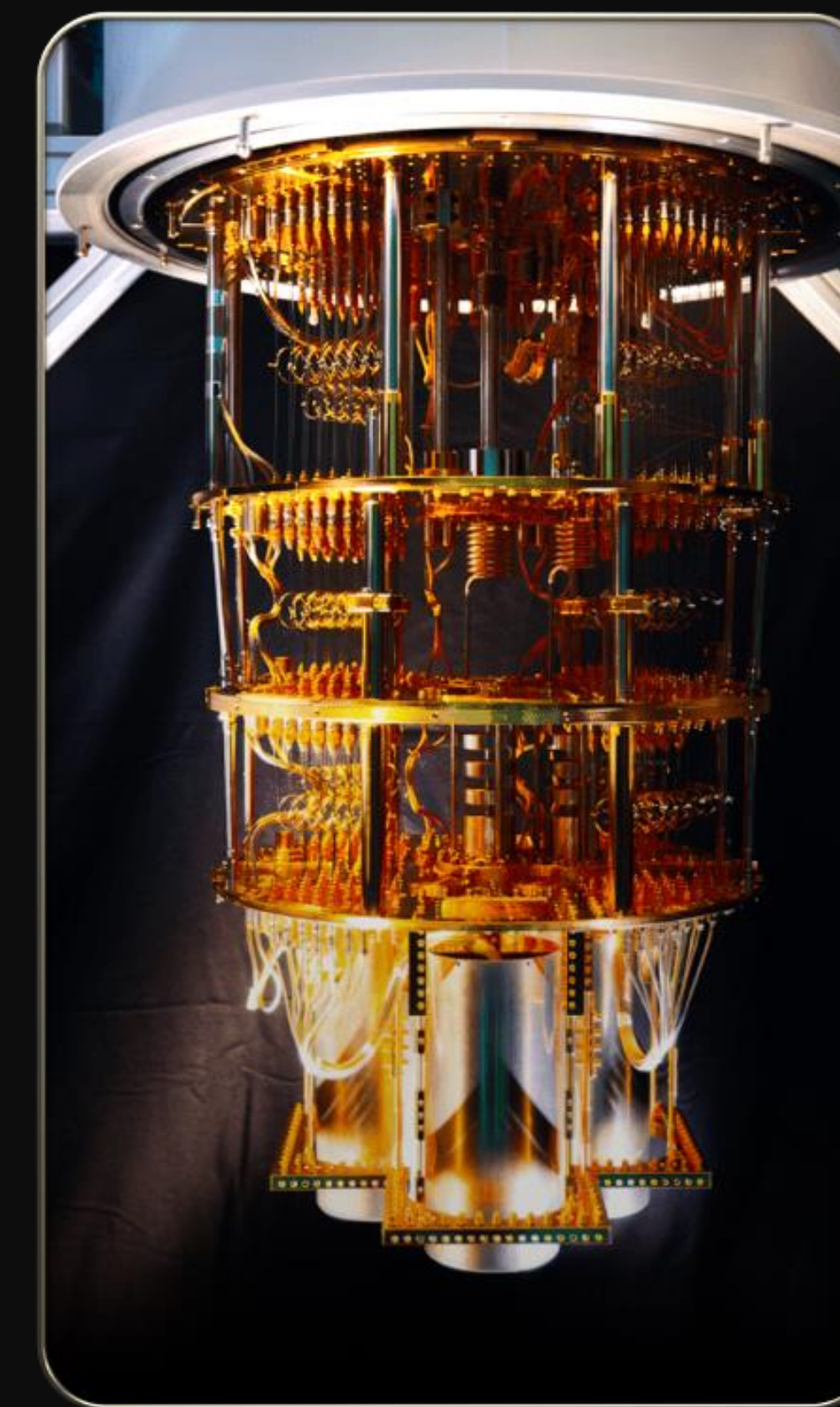


Qubit is more informative than classical bit!

The Next Great Instrument for Scientific Discovery



Robert Hooke's Microscope, 1664



Quantum Processors, today

Quantum Computing's Potential Impact



Drug discovery



EV chemistry



Materials

AI and Quantum

AI to Advance Quantum Computing



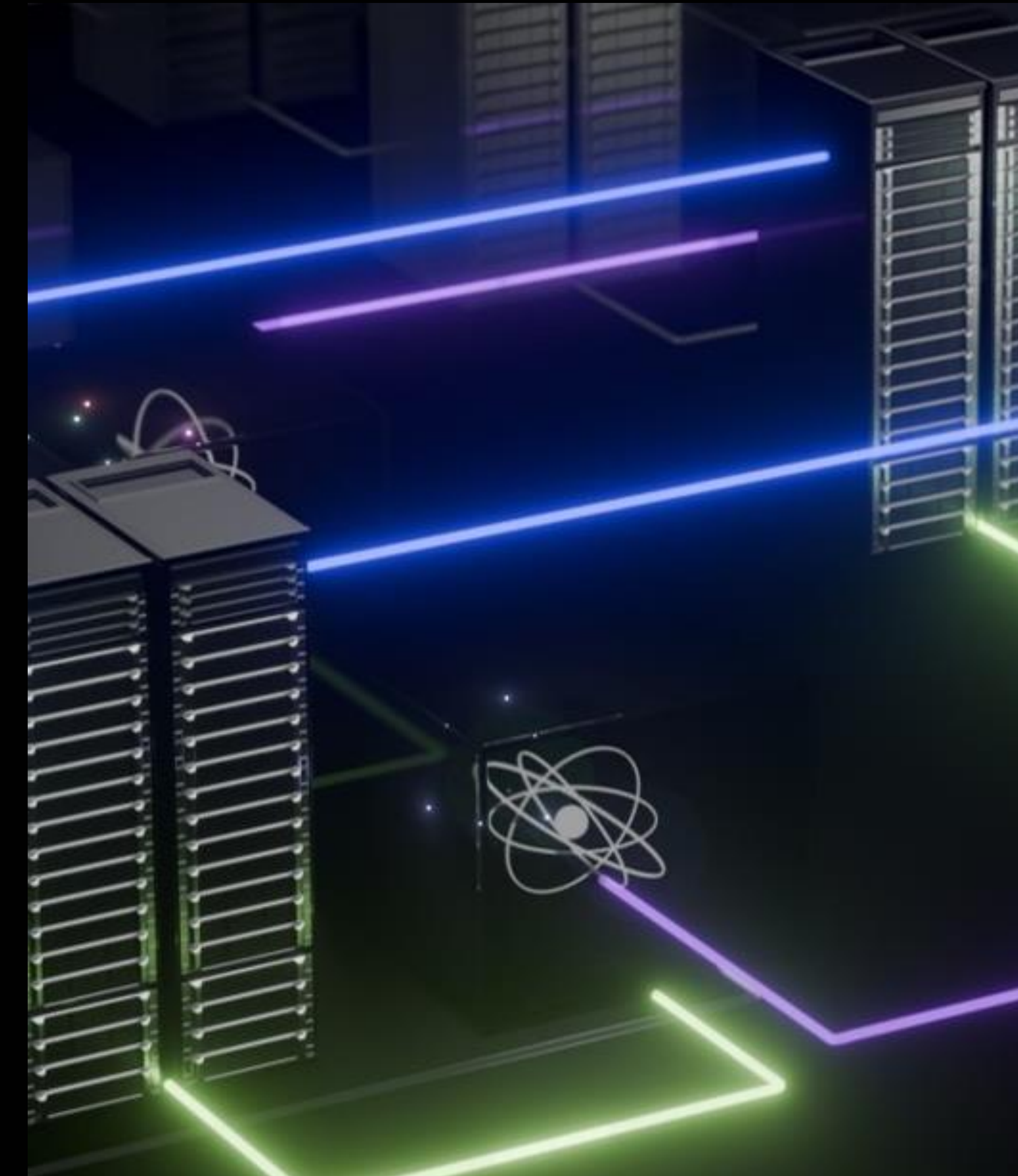
Algorithms

QEC

Calibration

Digital Twins

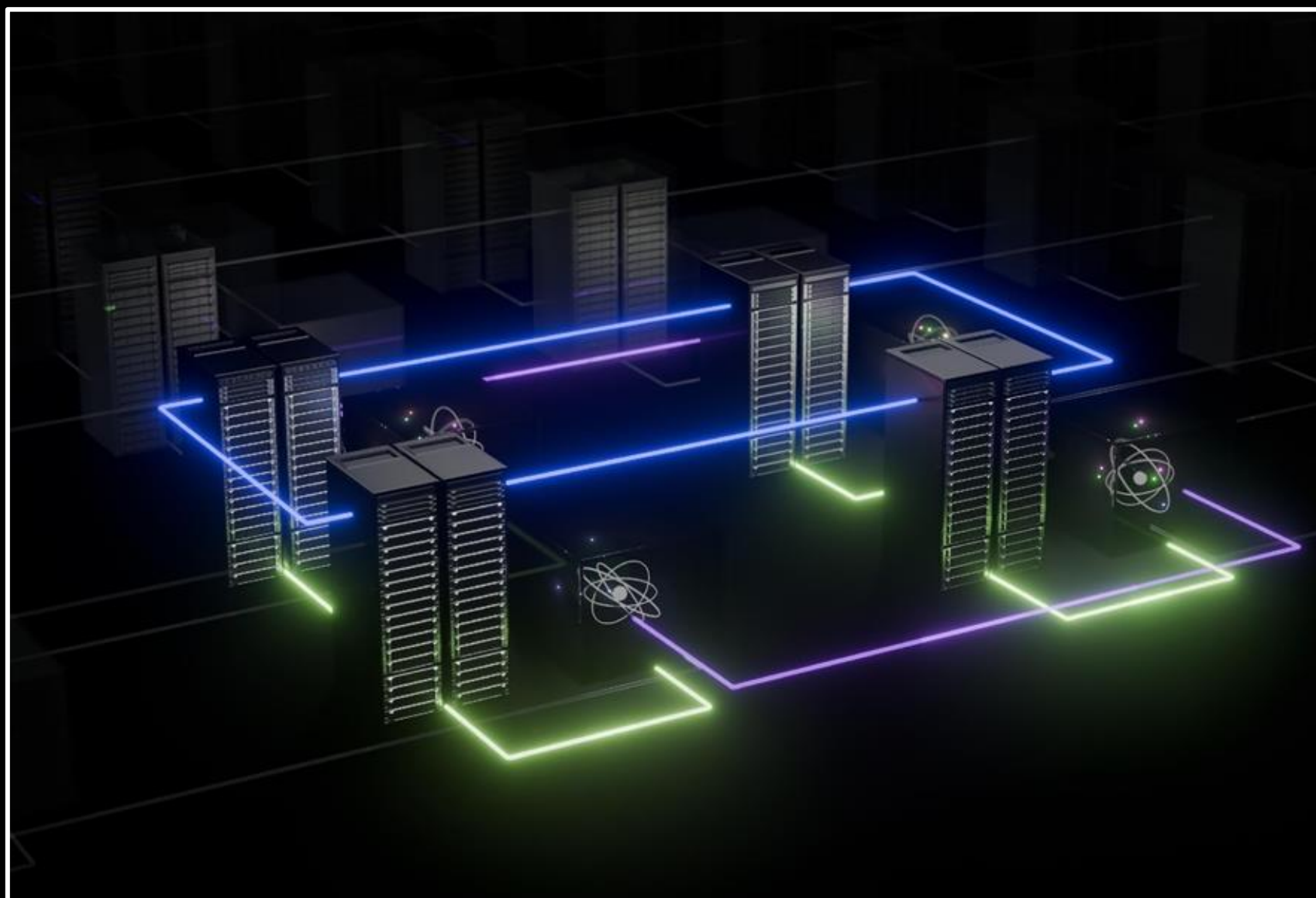
Quantum Computing to Advance AI



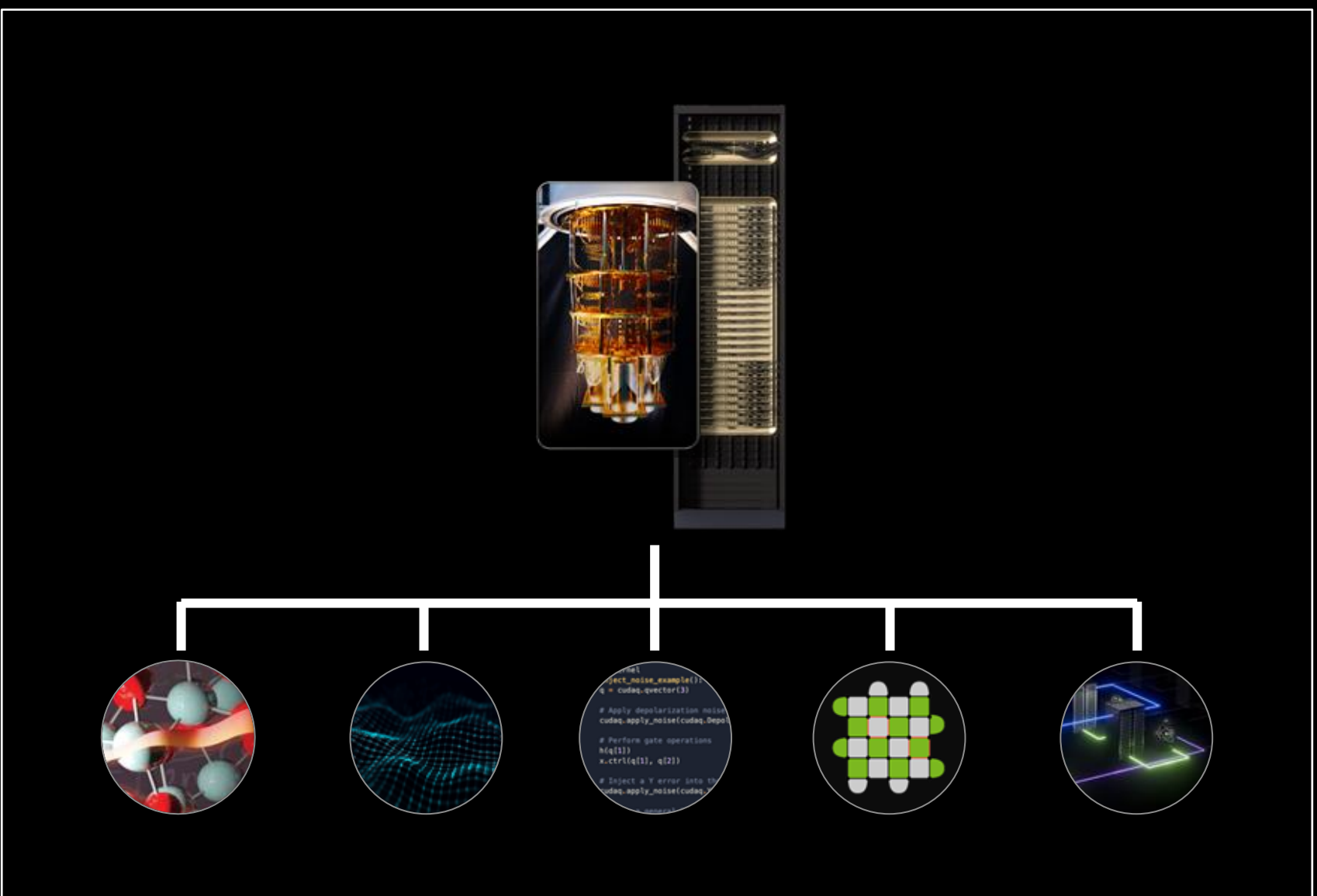
Hybrid Applications

Quantum Data
Generation

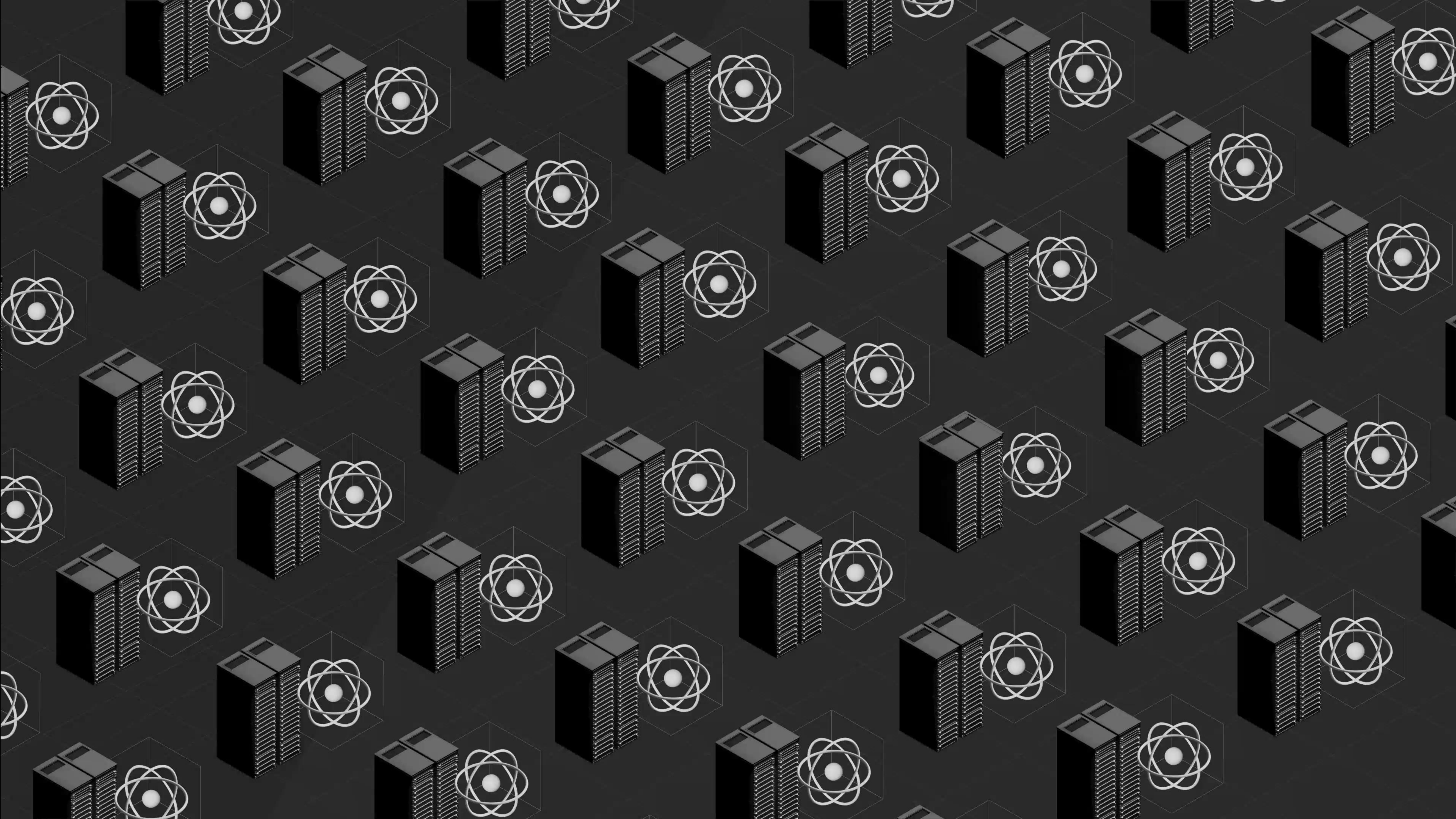
Expand Frontier of
Accelerated Computing



Running useful quantum applications and hardware requires **AI supercomputing**



Developing useful quantum applications and hardware requires **AI supercomputing**



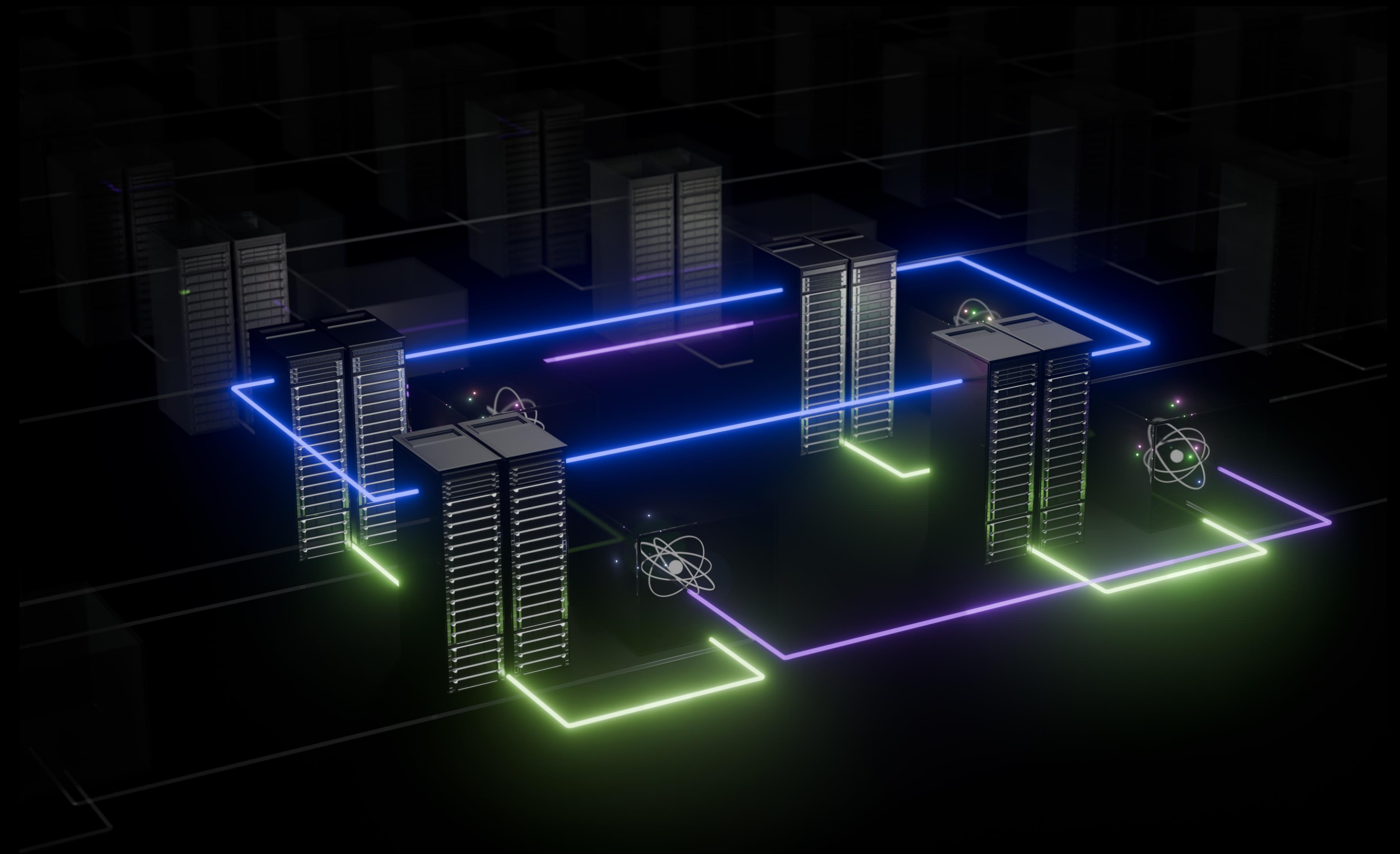
Quantum-GPU Supercomputing

All valuable quantum computing will be **hybrid**

GPUs and AI will prove essential

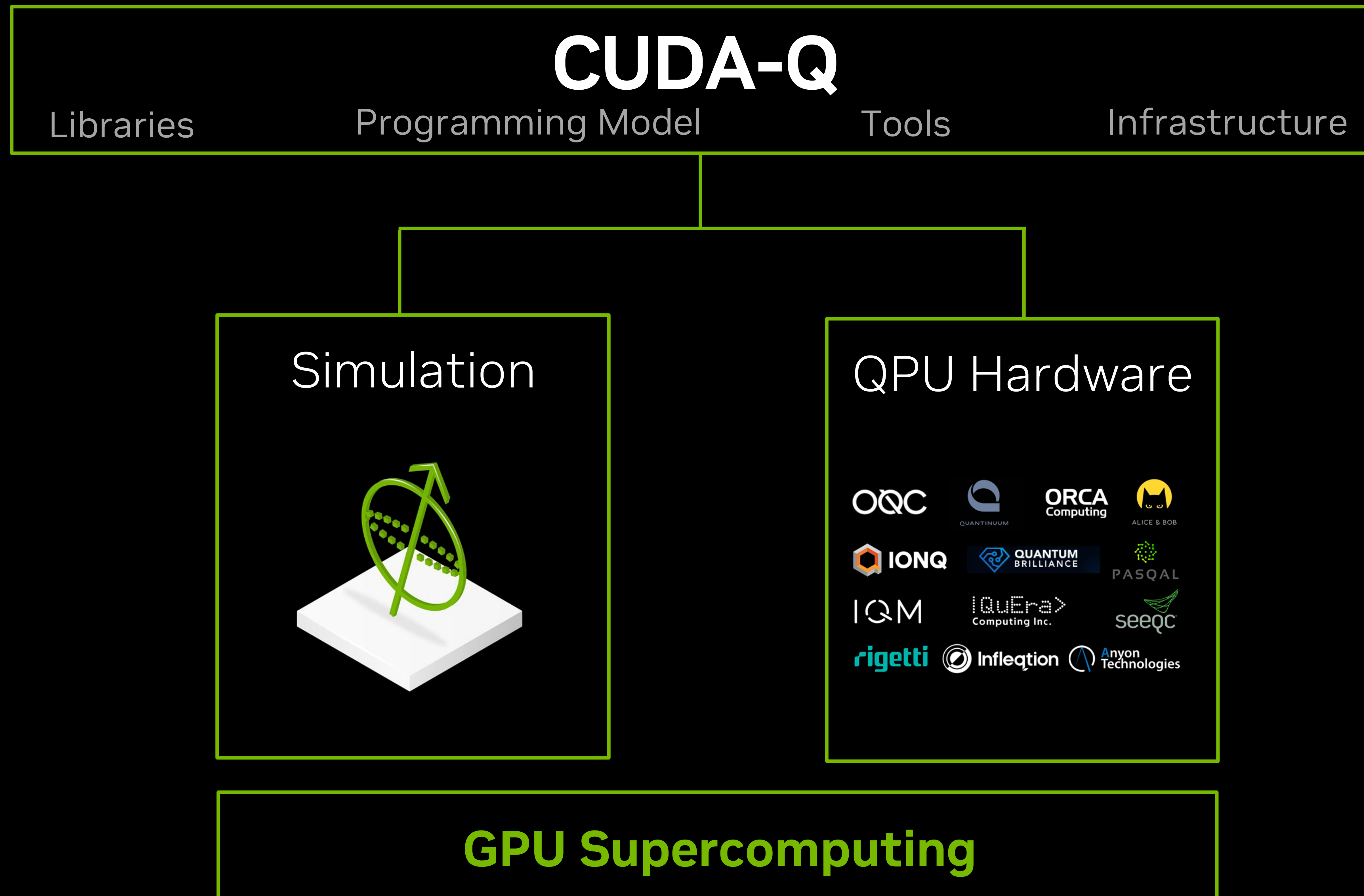
Petascale compute needed for useful Fault Tolerant Quantum Computers

Developer ecosystem is critical



CUDA-Q

The platform for accelerated quantum computing



The NVIDIA Quantum Platform

Bringing AI Supercomputing to Enable Useful Quantum Computing

CUDA-QEC and CUDA-Q Solvers

Turnkey Quantum Research and Application Development

CUDA-Q

Open-Source Software Platform for Accelerated Quantum Supercomputers

cuQuantum

Accelerated Quantum Simulation/Emulation on GPU

NVQLink

Reference Architecture for Low Latency Integration

The NVIDIA Quantum Platform

Bringing AI Supercomputing to Enable Useful Quantum Computing

CUDA-Q

Open-Source Software Platform for Accelerated Quantum Supercomputers



The NVIDIA Quantum Platform

Bringing AI Supercomputing to Enable Useful Quantum Computing

CUDA-Q

Open-Source Software Platform for Accelerated Quantum Supercomputers

cuQuantum

Accelerated Quantum Simulations



Cirq



PENNYLANE



Qiskit



Quest



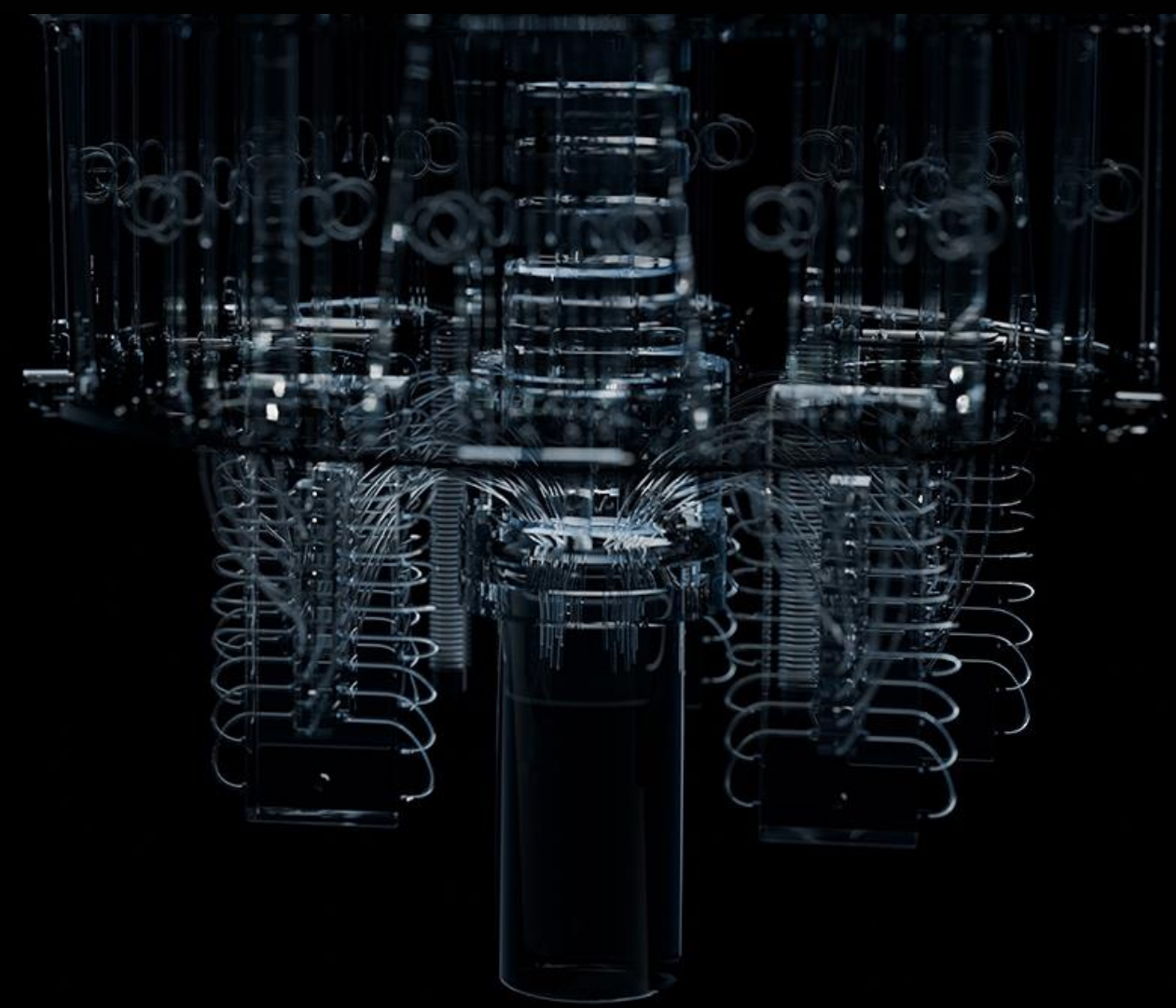
Qulacs

TKET™

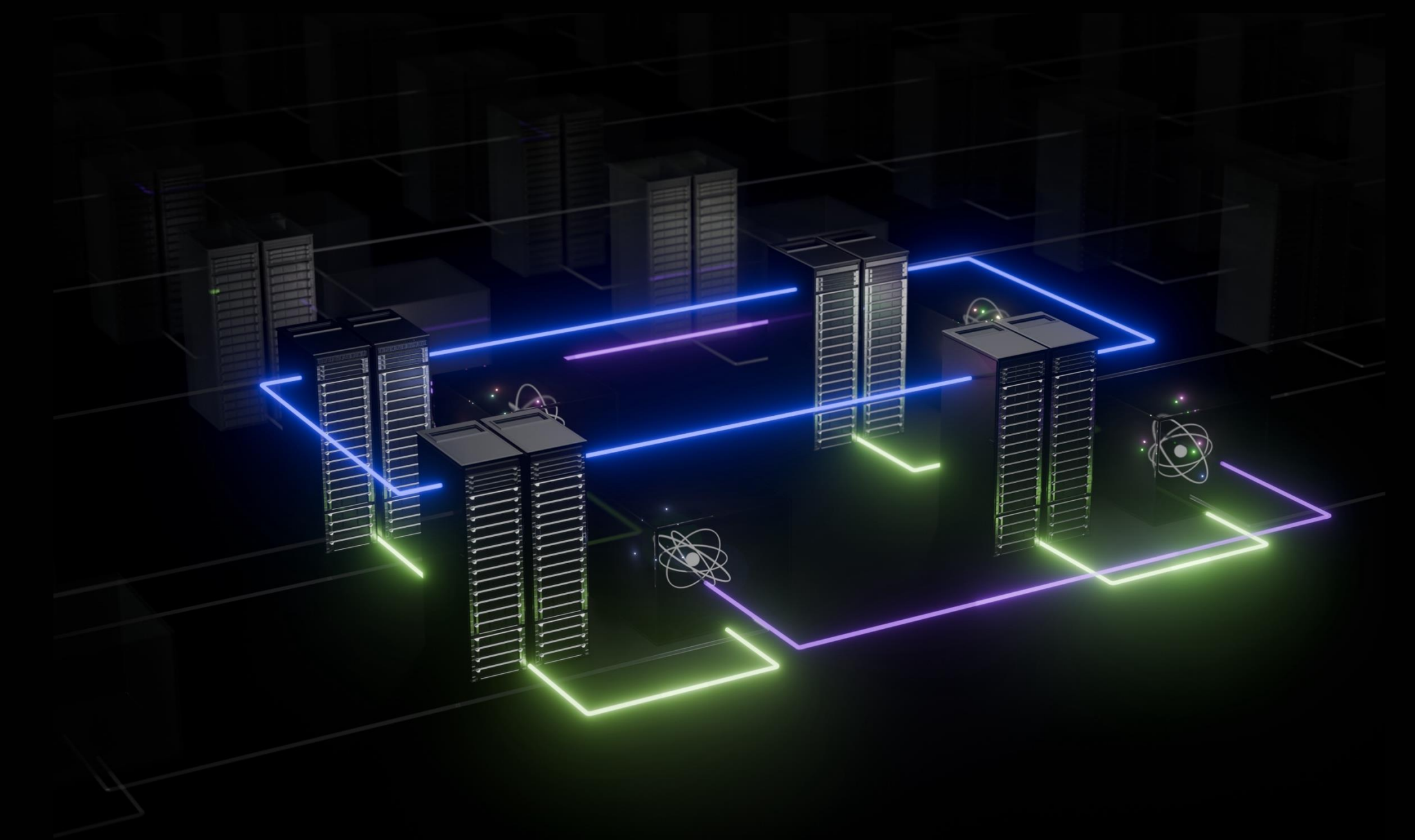


Torch
Quantum

The Path from Qubits to Supercomputers



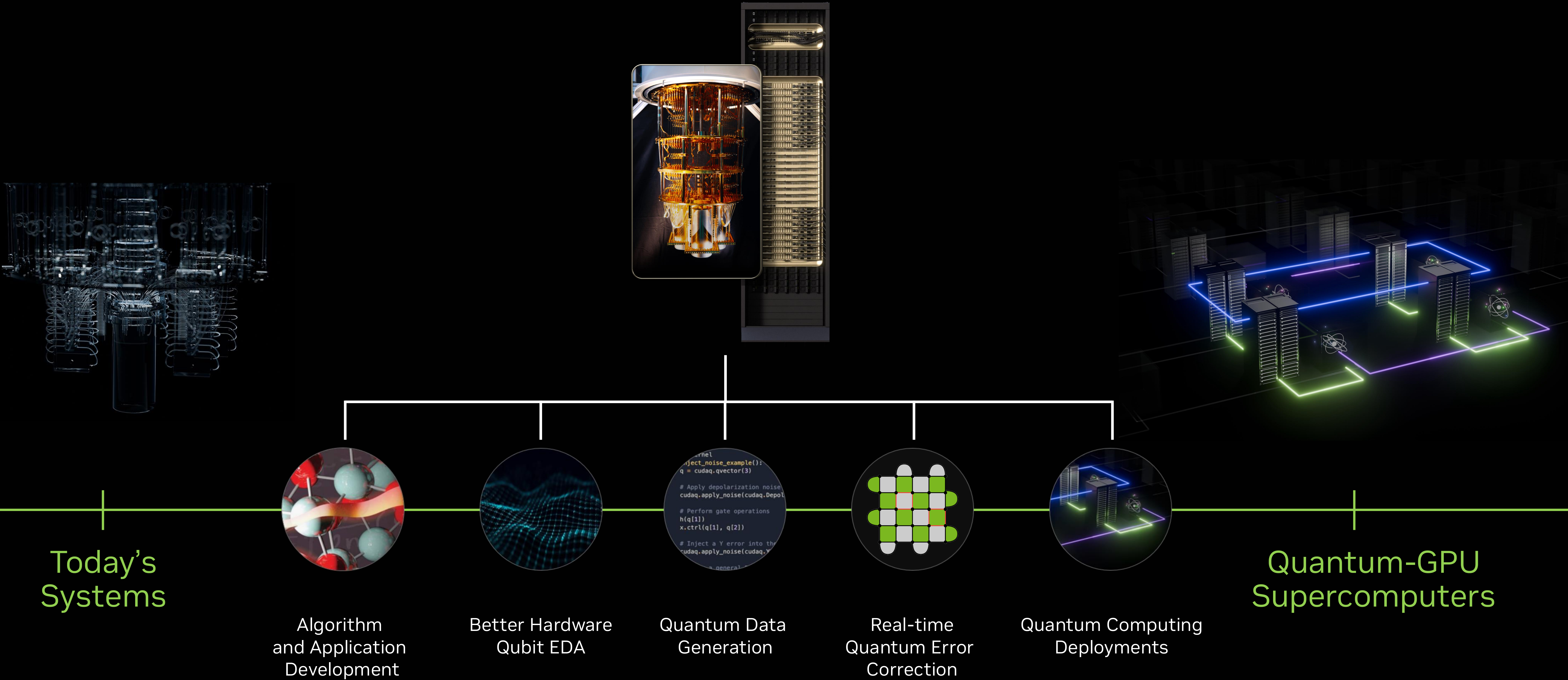
Today's
Systems



Quantum-GPU
Supercomputers

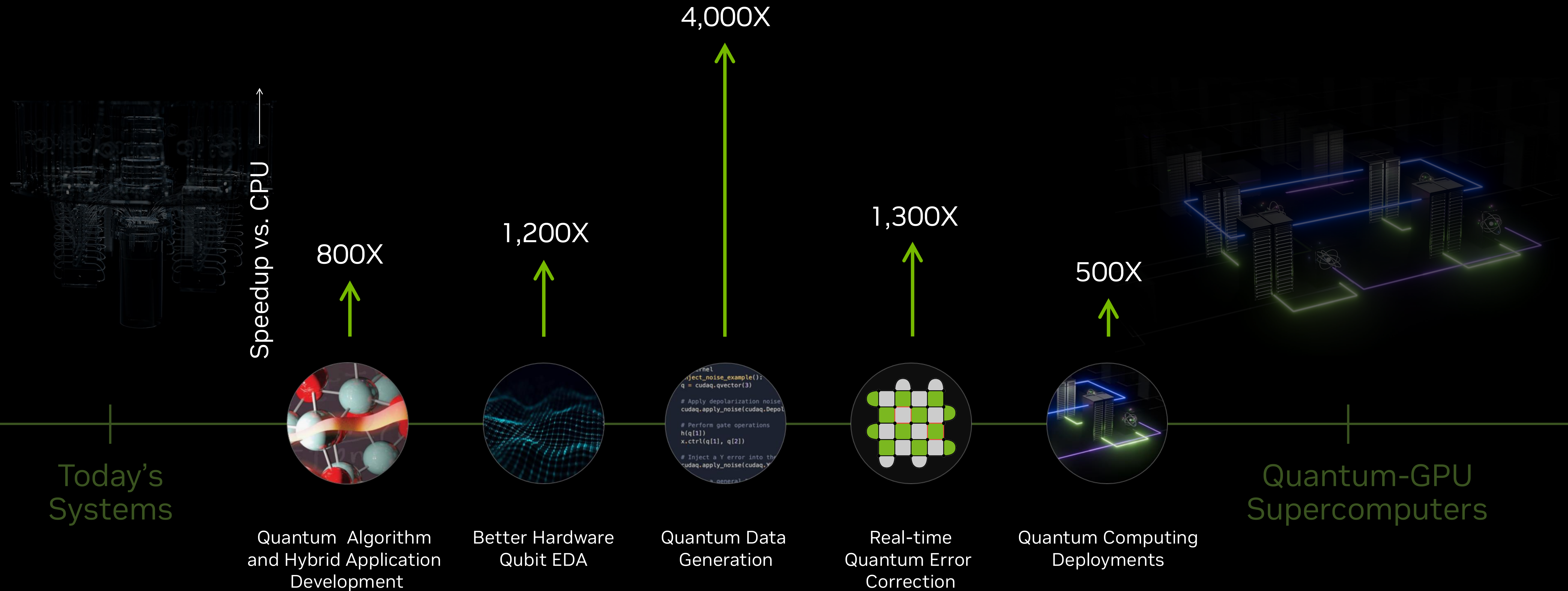
The Path from Qubits to Supercomputers

AI to the Rescue



The Path from Qubits to Supercomputers

AI to the Rescue



Alexeev, Y., Farag, M.H., Patti, T.L. *et al.* Artificial intelligence for quantum computing. *Nat Commun* **16**, 10829 (2025). [arXiv:2411.09131](https://arxiv.org/abs/2411.09131)

DGX SuperPOD Successes for Quantum

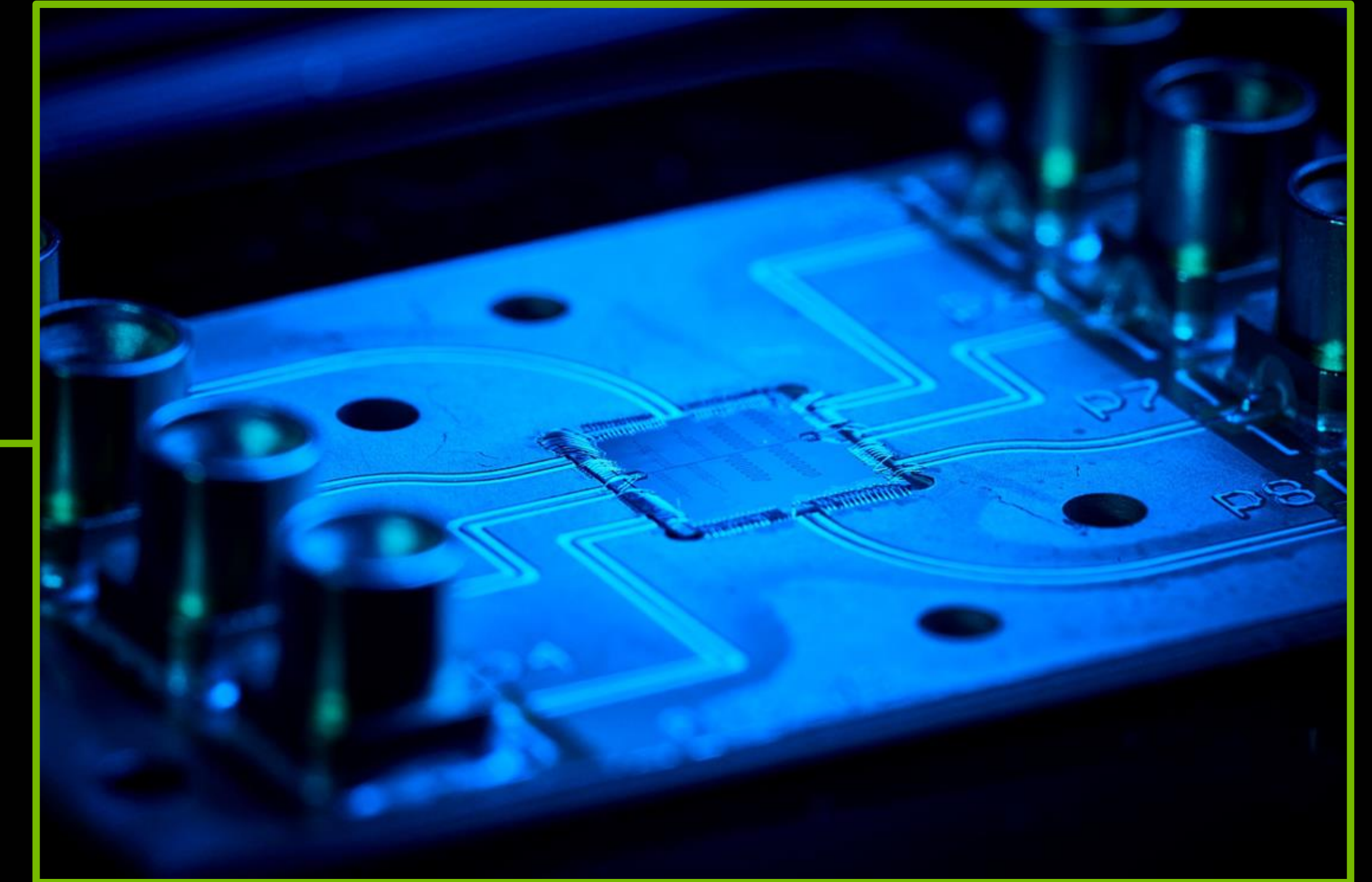
“Gefion will be an essential tool to support our full stack quantum computing research as part of the NQCP - NNF’s 12- year initiative...”

- Peter Krogstrup, CEO NNF QC Program

Gefion

**Novo Nordisk
Foundation**

H100 + Infiniband



“ABCI-Q will let Japanese researchers explore quantum computing technology to test and accelerate the development of its practical applications.”

- Masahiro Horibe, deputy director of AIST

ABCI-Q

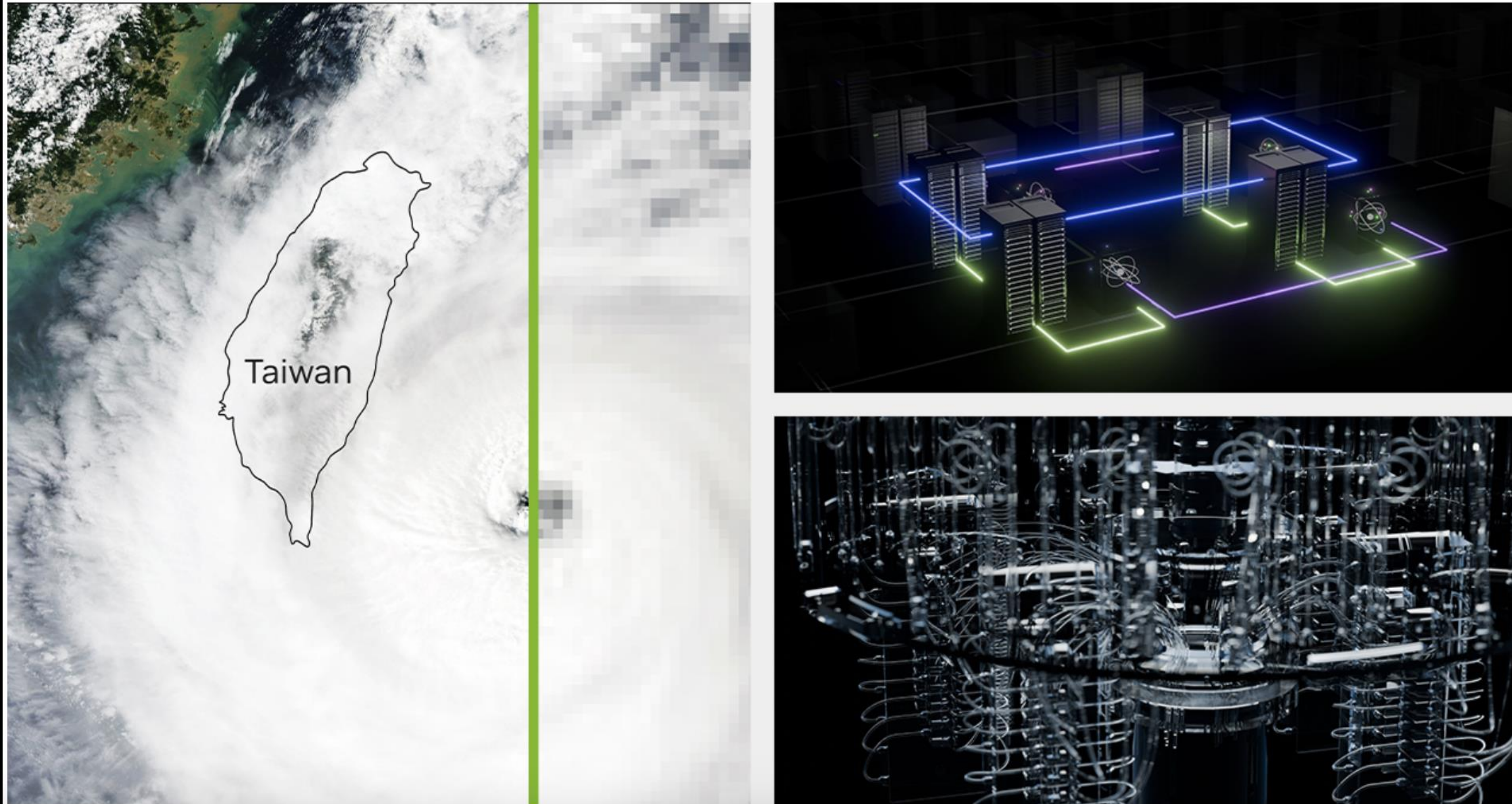
AIST and Fujitsu
>2,000 H100 + Infiniband



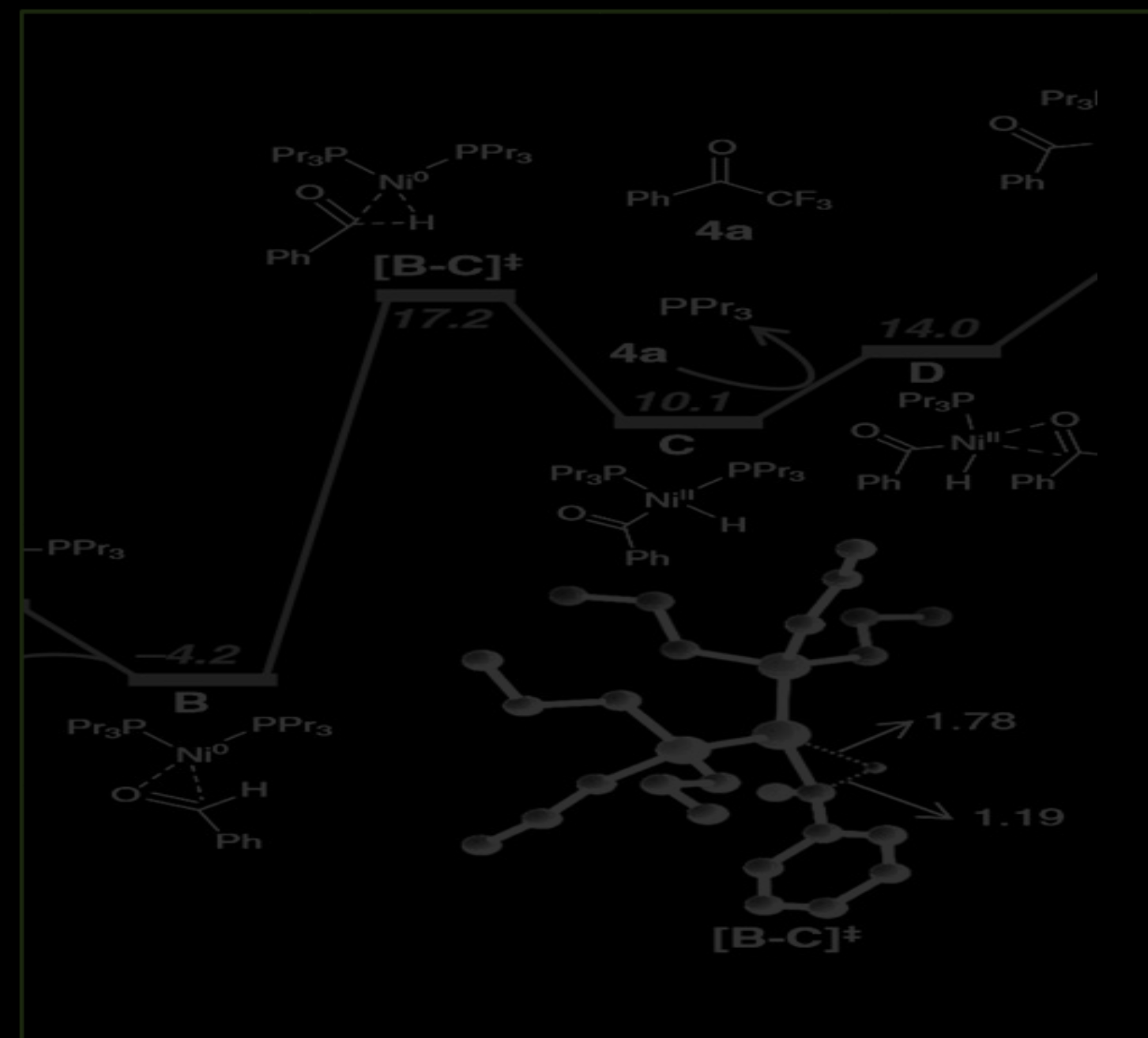
NVIDIA-Powered Supercomputer to Enable Quantum Leap for Taiwan Research

Built by ASUS, the National Center for High-Performance Computing's new AI supercomputer will support climate science, quantum research and the development of large language models.

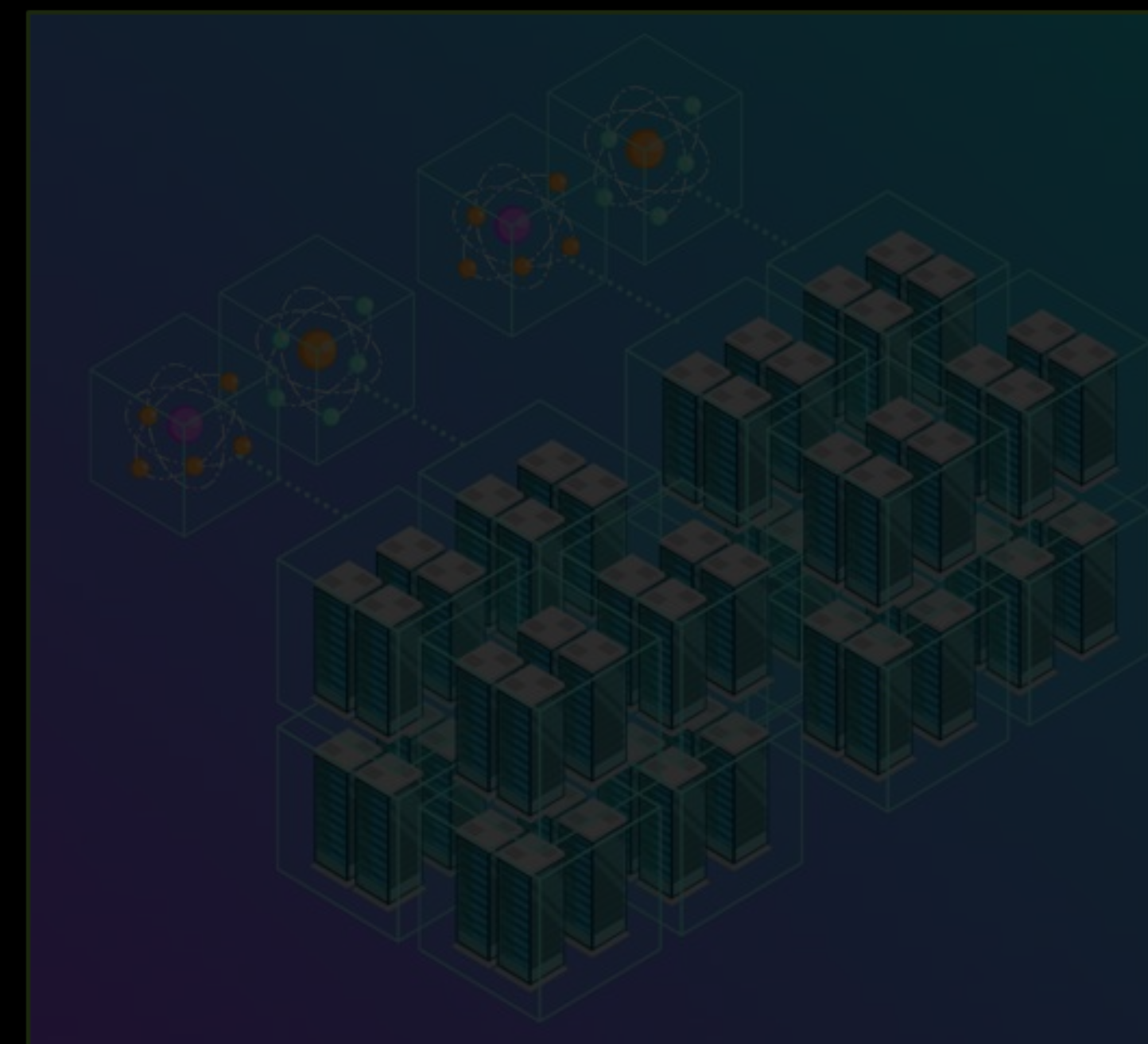
May 18, 2025 by [Dion Harris](#)



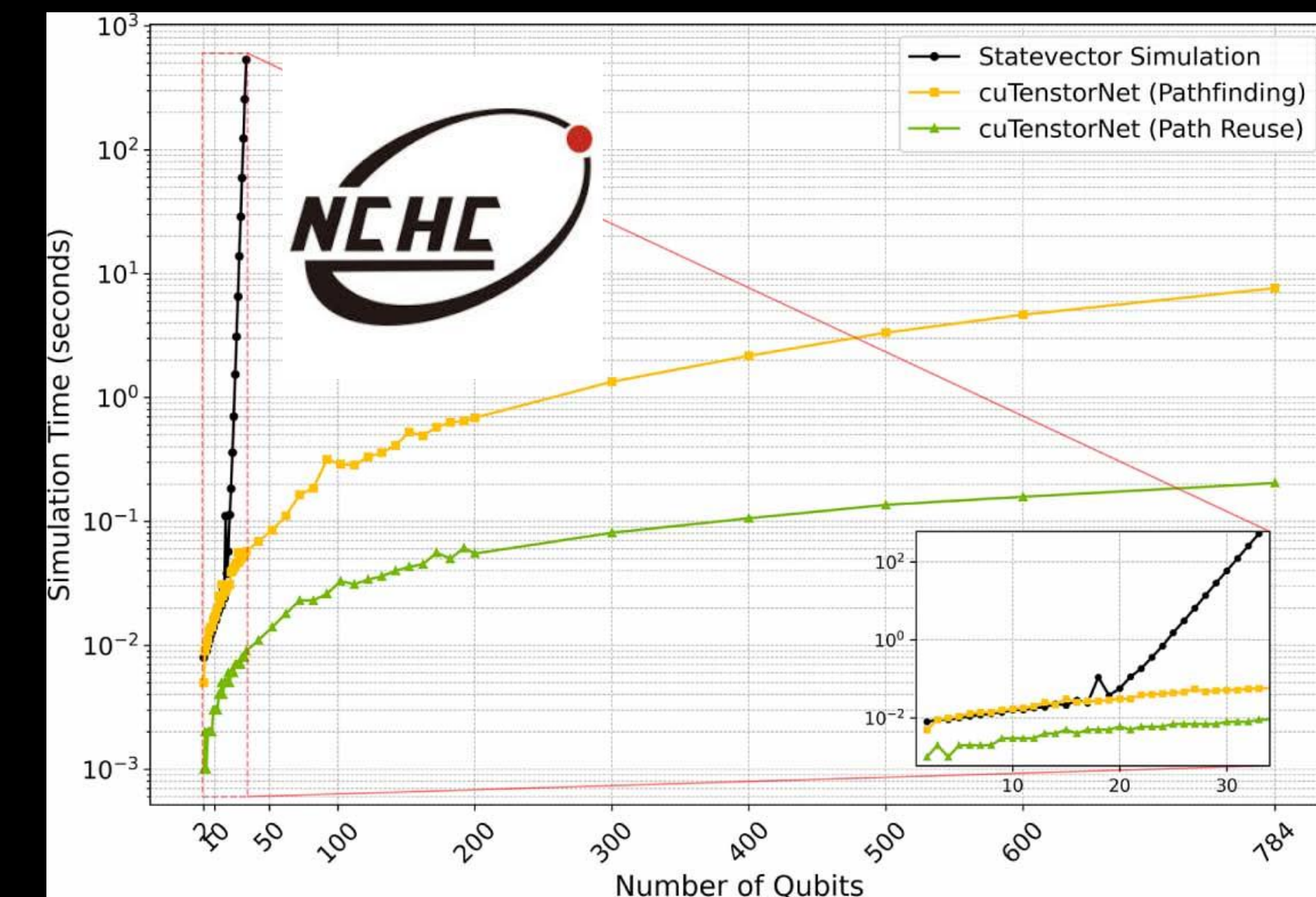
Better Algorithms with Large-Scale Simulations and AI



IonQ/AWS/AstraZeneca
AF-QMC simulation for
electronic structure modeling



HPE Labs
Distributed QC with
Adaptive Circuit Knitting



NCHC
> 1000-qubit simulation to
validate QML approaches

Qubits

Better QPUs

Better
Algorithms

Scaling QEC

Accelerated Quantum
Supercomputers

Chen et. al. cuTN-QSVM: cuTensorNet-accelerated Quantum Support Vector Machine with cuQuantum SDK.
<https://arxiv.org/html/2405.02630v2> (2024)

CUDA-Q Python Bindings

Core C++ compiler platform exposed via builder API
JIT compile Quake representation at runtime

- Simple ways to create quantum kernels

```
import cudaq

cudaq.set_target('nvidia')

@cudaq.kernel
def kernel():
    # A single qubit initialized to the
    # ground / zero state
    qubit = cudaq.qubit()

    # Apply the Pauli x gate to the qubit
    x(qubit)

    # Measurement operator
    mz(qubit)

    # Sample the qubit for 1000 shots to
    # gather statistics
    result = cudaq.sample(kernel,
        shots_count=1000)
```

```
import cudaq

cudaq.set_target('quantinuum', machine='H2-2')

@cudaq.kernel
def kernel(angles: List[float]):
    # Allocate a qubit that is initialized to
    # the  $|0\rangle$  state.
    qubit = cudaq.qubit()

    # Apply parameterized gates to the qubit
    rx(angles[0], qubit)
    ry(angles[1], qubit)

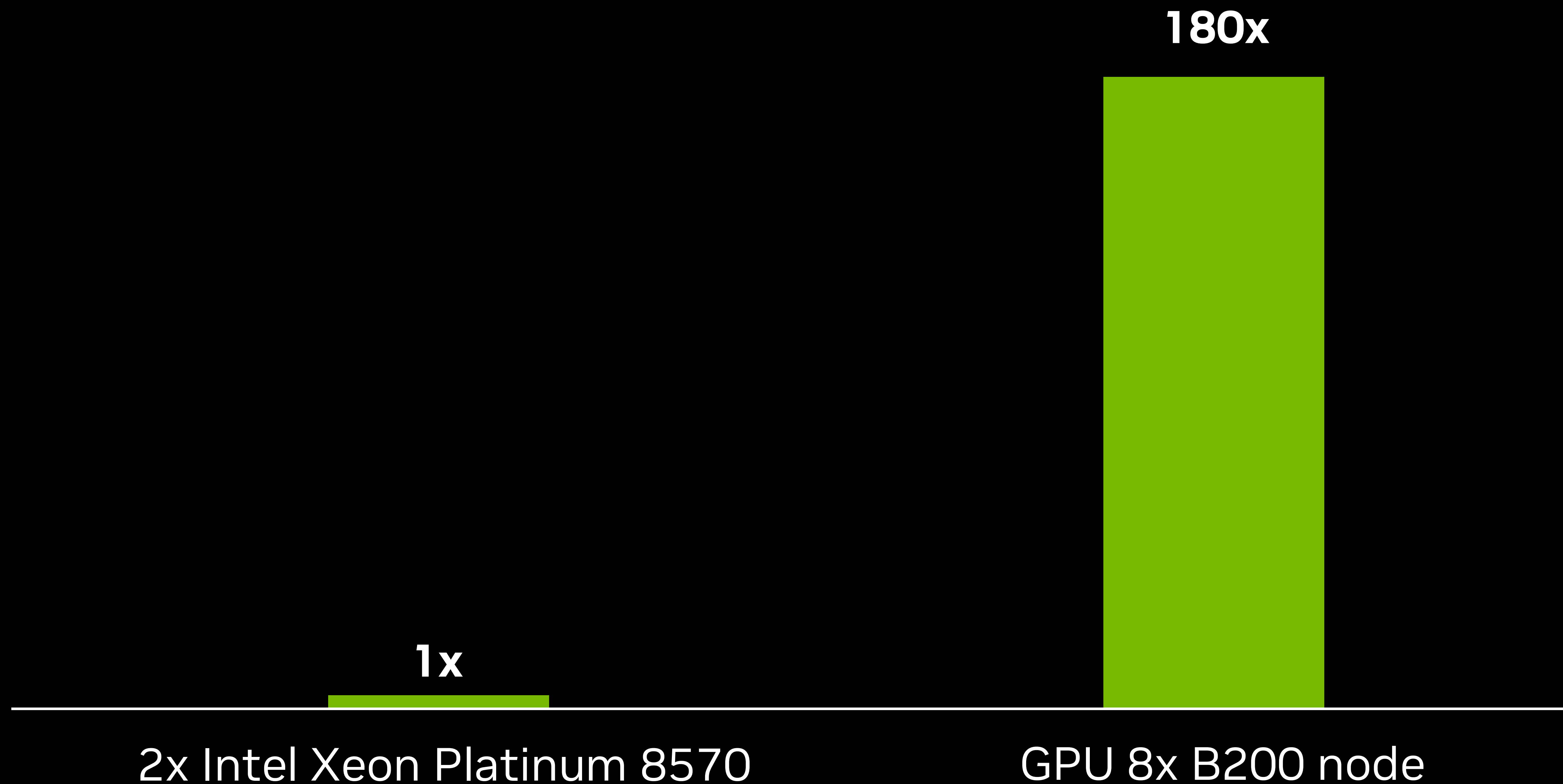
    # Gate parameters which initialize the qubit
    # in the zero state
    initial_parameters = [0, 0]

    # Draw the circuit diagram for the kernel
    print(cudaq.draw(kernel, initial_parameters))
```

CUDA-Q GPU-Acceleration

CPU's are not enough for the simulations needed to develop quantum computing

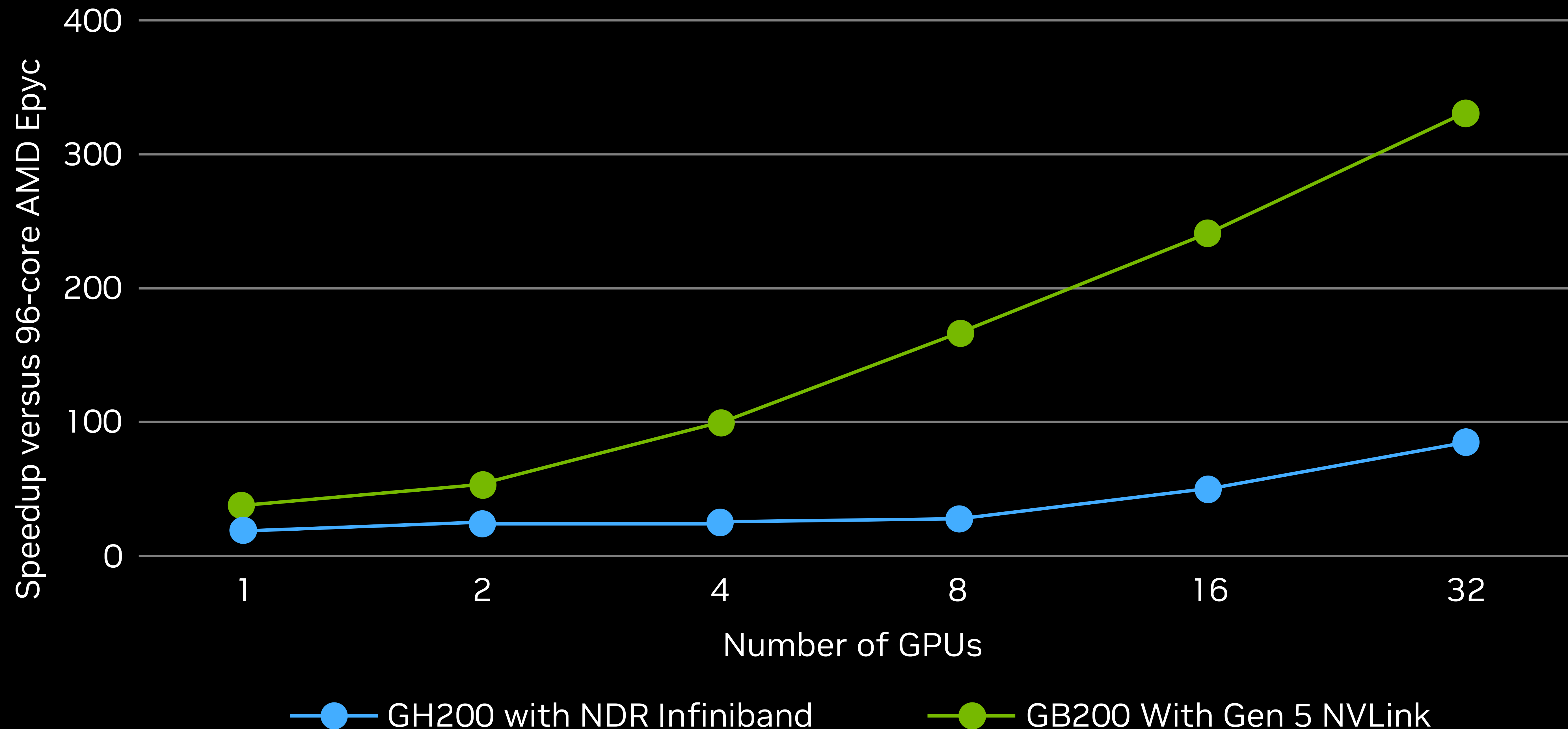
Speedup for 35 Qubit Quantum Volume Calculation



CUDA-Q GPU-Scaling

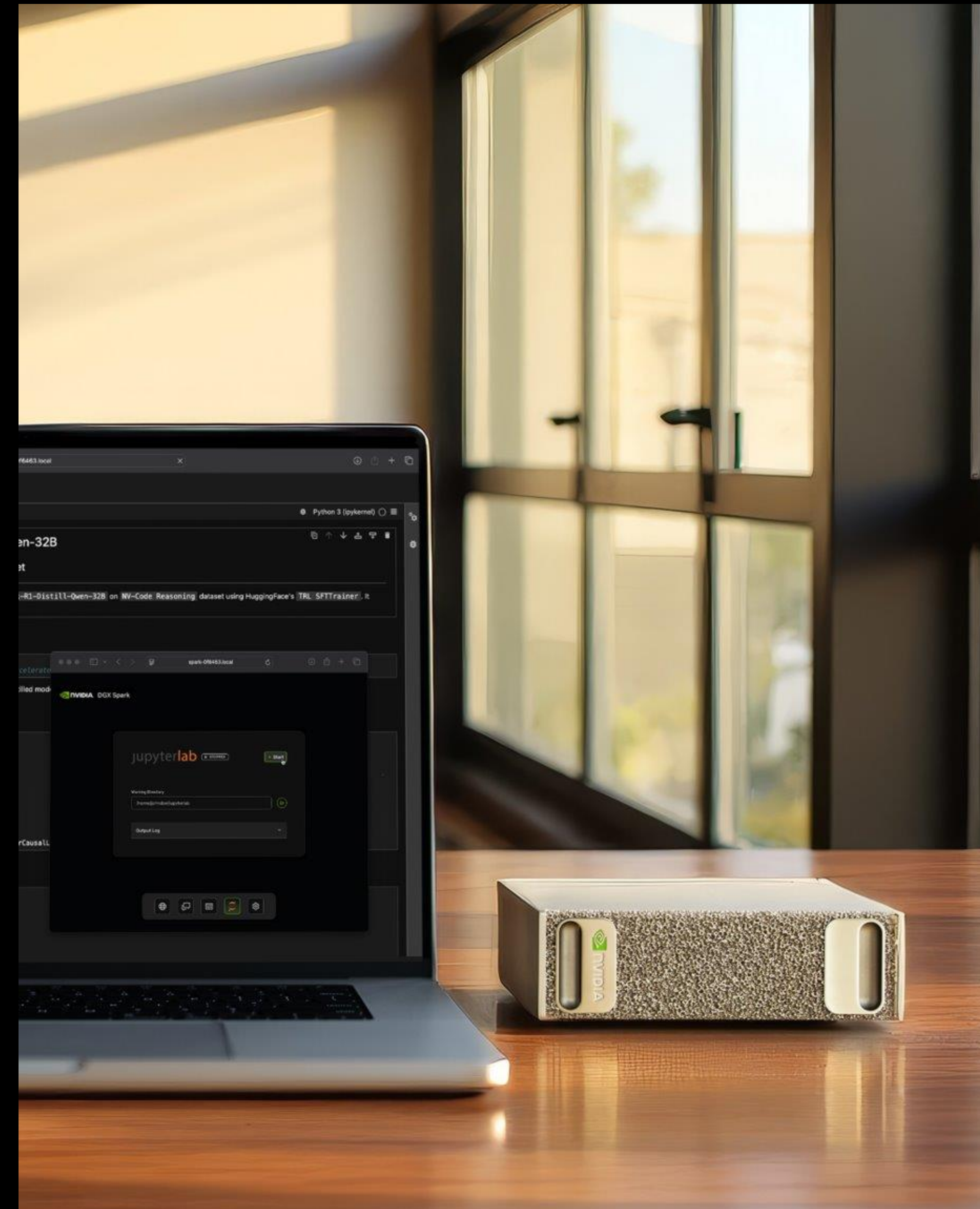
CUDA-Q allows researchers to trivially scale simulations from single GPUs to supercomputers

GPU-Scaling Speedup for Qubit Trotterization Circuit



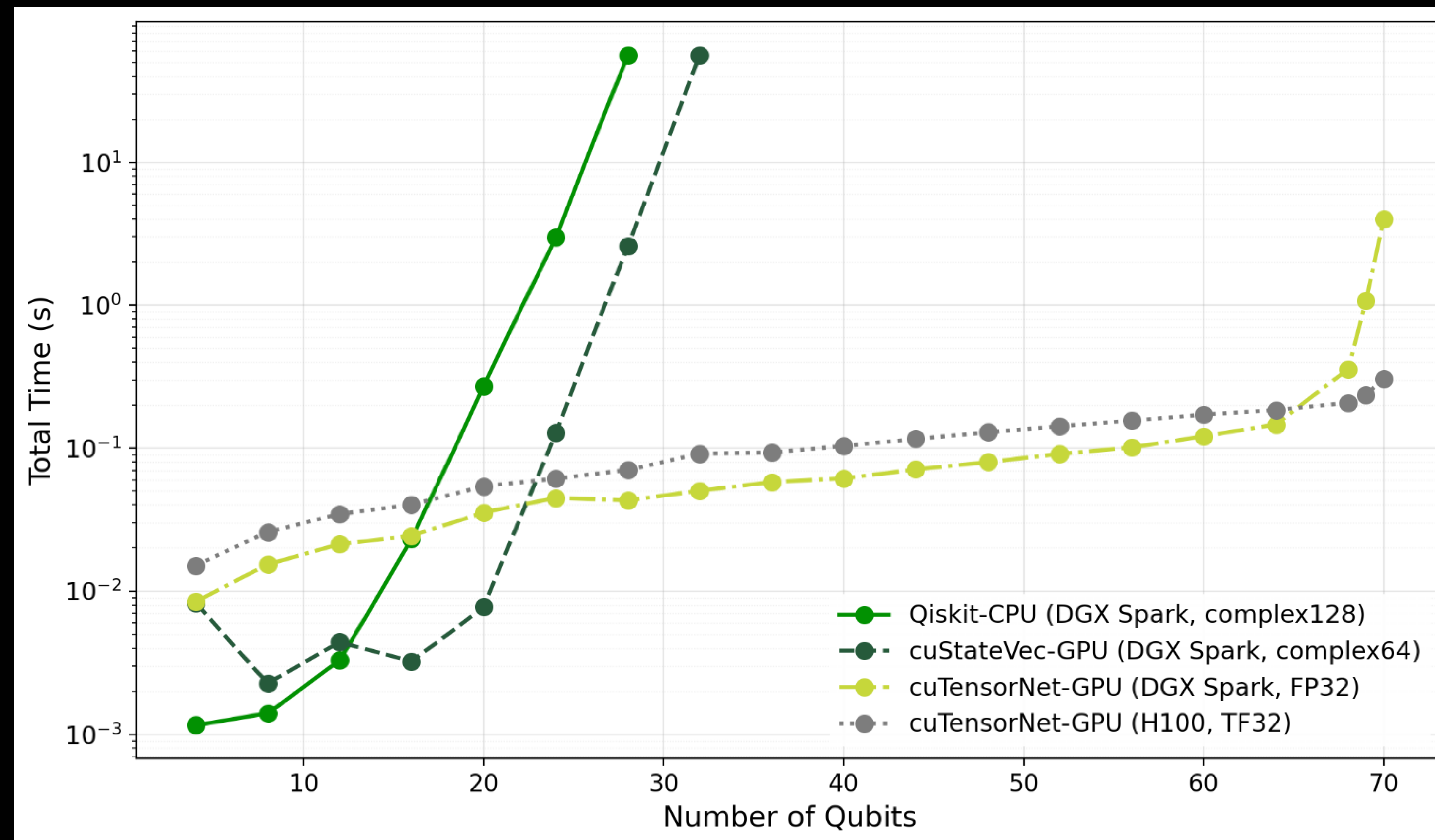
DGX Spark for Quantum Computing

- NVIDIA **Grace Blackwell Architecture**
- **QPU simulation**
 - Full state vector simulation up to 32 qubits
 - Tensor network simulation for larger systems
- Train and run **AI models for quantum** (AI decoders, quantum EDA, circuit design and more)
- Compact, power efficient form factor
- Configure as a simulated **desktop QPU** or as a **GPU resource in a hybrid quantum-GPU setup**



DGX Spark for Quantum Computing

The supercomputer in the lab



FAST-VQE benchmarking on DGX Spark with cuQuantum (cuTensorNet) acceleration

- NVIDIA **GB10 Grace Blackwell superchip**
- **1 PFLOP of FP4** AI performance
- **128GB** coherent, unified system memory

- NVQLink-compatible **ConnectX-7 Smart NIC**
- 150mm L x 150mm W x 50.5mm H

NVIDIA Open Quantum Platform

200+

NVIDIA Quantum
Partners

>90%

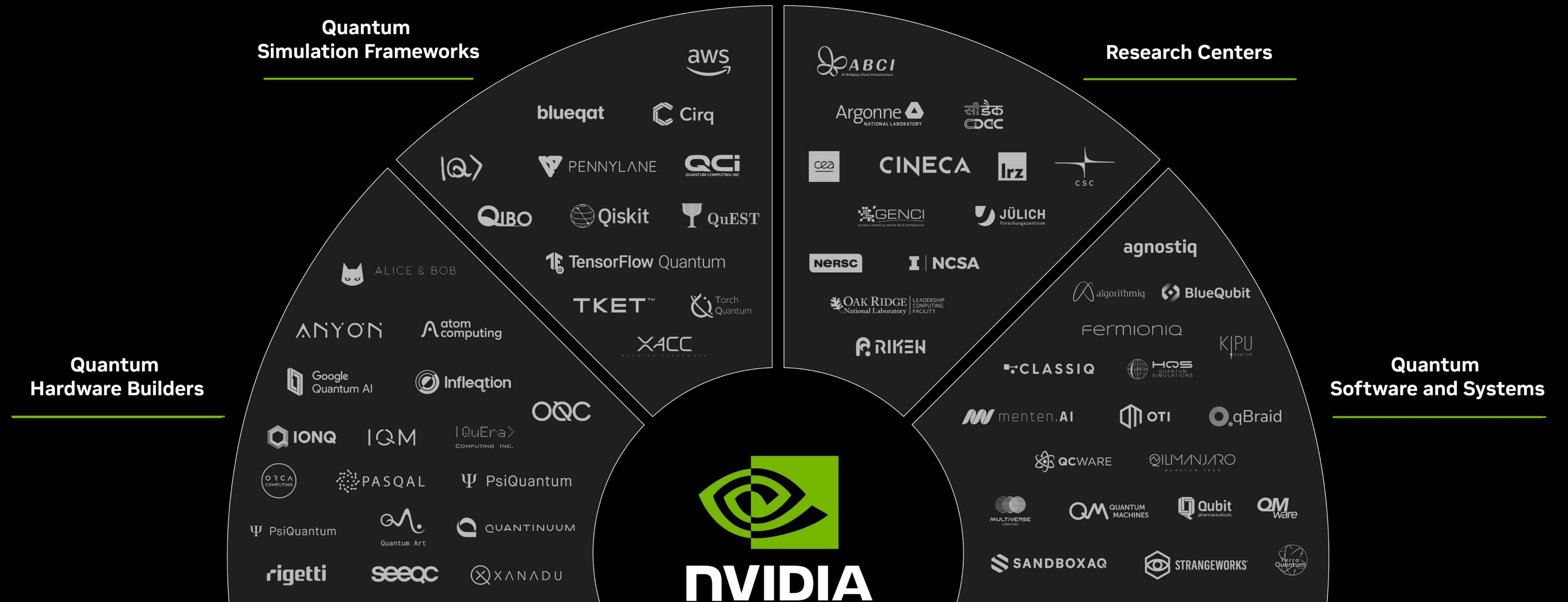
Largest Startups
Working with NVIDIA

>80%

QPUs Integrating
NVIDIA Software

100%

Leading Quantum
Development Frameworks
Accelerated



CUDA-Q Now Available

The hybrid quantum computing platform

CUDA-Q Overview

<https://developer.nvidia.com/cuda-q>

CUDA-Q Academic

Educational resources for CUDA-Q
(<https://github.com/NVIDIA/cuda-q-academic>)

CUDA-Q Applications

<https://nvidia.github.io/cuda-quantum/latest/using/tutorials.html>



Find out More

CUDA-Q Agentic Bootcamp + Onboarding Guide



會員中心 服務介紹 操作說明 常見問題

註冊 登入

← 上一則 → 下一則 ↶ 回列表

"SINICA NCHC CUDA-Q Agentic Coding Bootcamp" 活動公告

 2026-07-08 15:30:00 課程資訊

SINICA NCHC CUDA-Q Agentic Coding Bootcamp

Overview / 活動說明

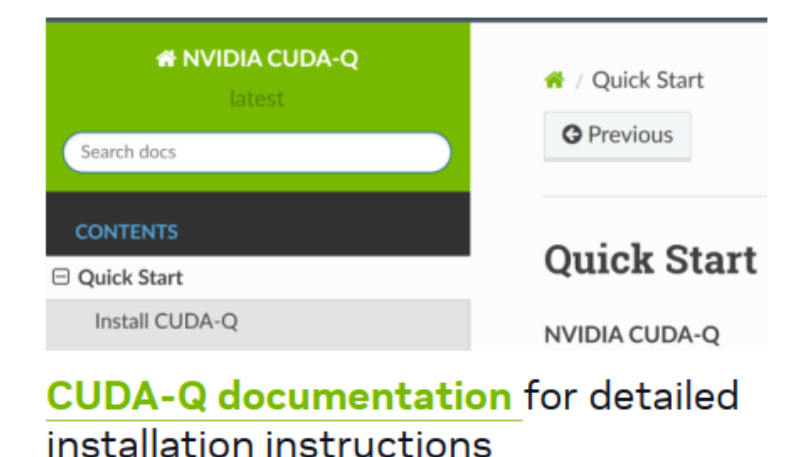
Together with NVIDIA and the OpenACC organization, the Academia Sinica, Research Center for Critical Issues and National Center for High-performance Computing (NCHC) will host the in-person SINICA NCHC CUDA-Q Agentic Coding Bootcamp on August 7.

SINICA NCHC CUDA-Q Agentic Coding Bootcamp is a hands-on training program designed to help developers explore the latest NVIDIA quantum computing and hybrid quantum-classical workflows from quantum programming fundamentals to advanced application development with agentic AI. Through expert-

Installation Guide & Quick Start

Start your CUDA-Q journey

- **Installation by Using PyPI:**
 - Required GPU architect: *Turing, Ampere, Ada, Hopper, Blackwell*
 - Ensure that the [CUDA Toolkit](#) is installed (version >= 12.x)
(Blackwell supported for CUDA 13.x / Driver 580.65.06+ only)
 - [Driver](#) version is updated (version >= 560.35.05+)
 - Install CUDA-Q directly [via PyPI](#) (pip version >= 24.0)
- **Installation by Using Container [Recommended]:**
 - If you are using Windows, WSL (Windows Subsystem for Linux) is needed, and it should be WSL2. Follow [WSL Installation Guide](#)
 - Install Docker by visiting [Docker's website](#)
 - [Pull/Run the container image](#)
 - Container flags can be found on the [NGC page](#), for example: `nvcr.io/nvidia/quantum/cuda-quantum:cu13-0.15.0`
- **Get Started:**
 - Learn [CUDA-Q basics](#) for easy hands-on
 - [Official GitHub repository](#) for full understanding



\$10 Brev Credit
~300 NTD

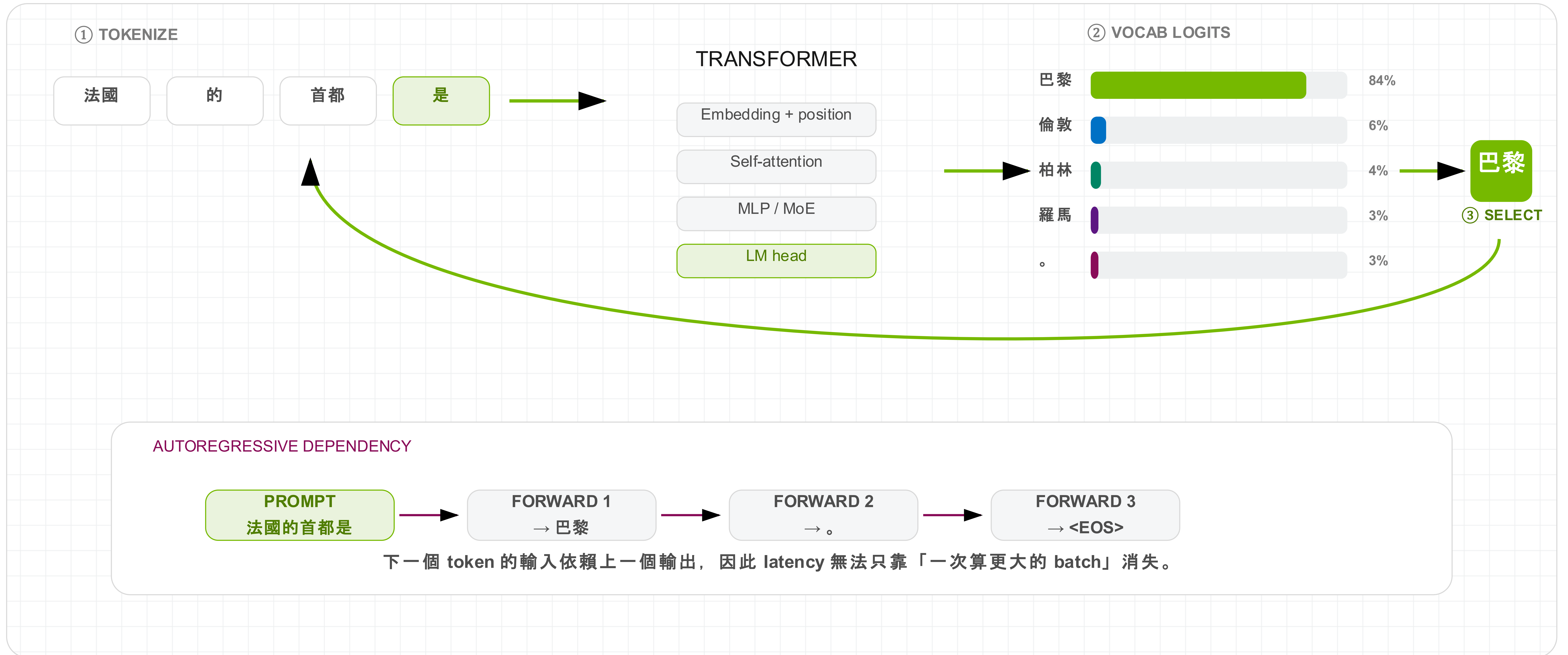


Accelerating LLM Inference



LLM 推論原理：一次預測一個 Token

模型產生 logits、選出 token、接回輸入；這個序列相依性決定了 decode 的基本成本



KV Cache 與 Prefill / Decode

KV Cache 避免重算歷史 K/V；代價是 decode 每一步都必須讀取更長的歷史記憶體

PREFILL prompt tokens 可平行處理

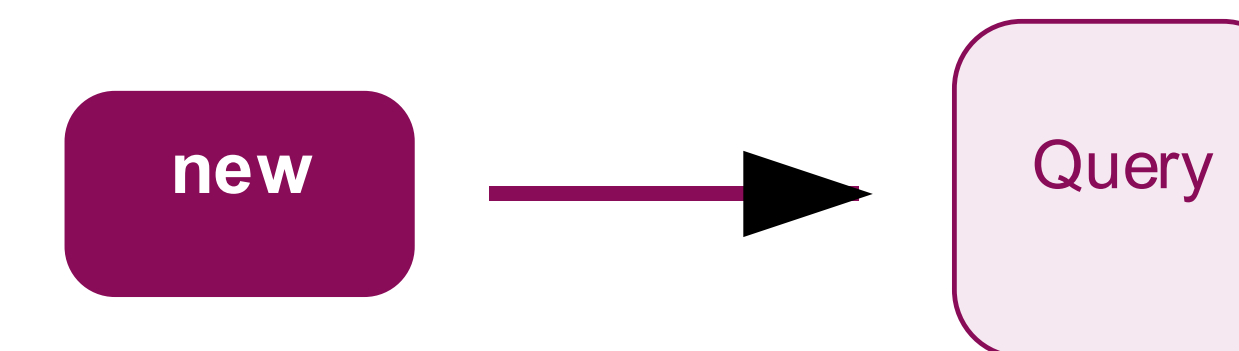


建立每一層的 K / V

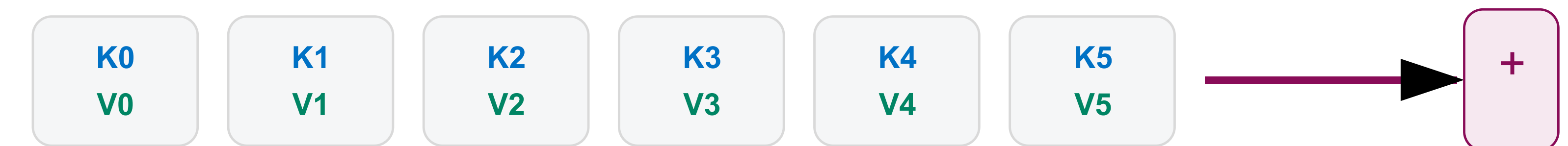


一次大矩陣運算：compute-friendly

DECODE 每輪只有 1 個新 token



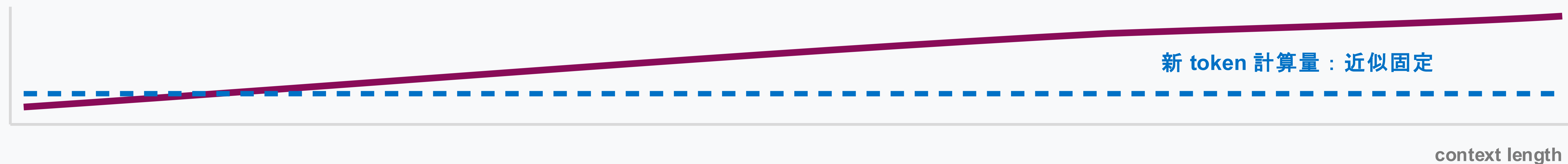
新 token 產生 Query，再讀取全部歷史 K / V



小矩陣 + 大量 KV 讀取：bandwidth-bound

每個 decode step 的資料流

HBM bytes



速度提升來自六層優化

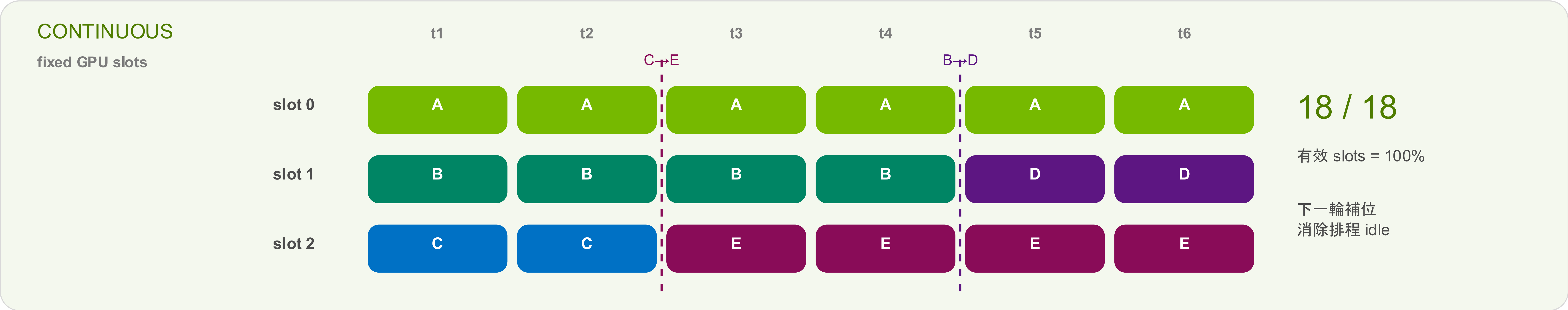
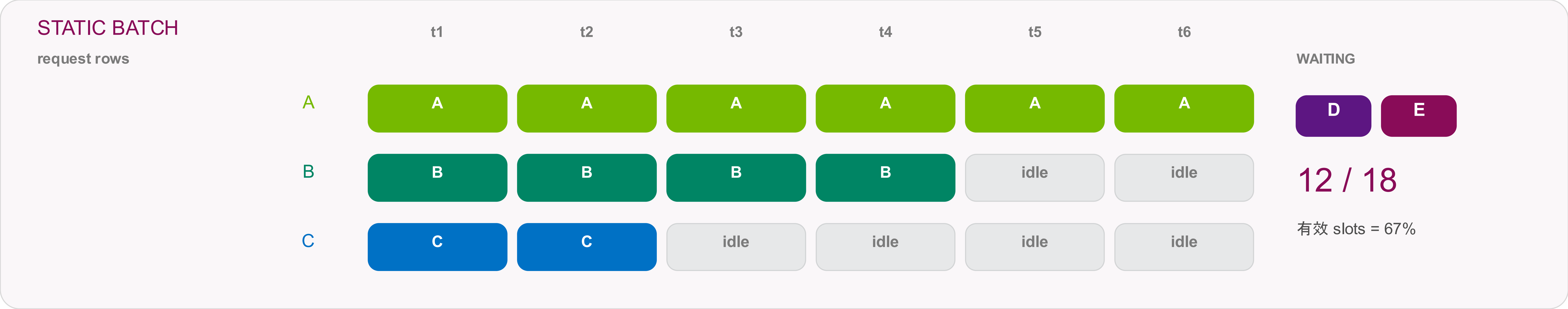
每一層都對應一種可量測的系統浪費；最終效益會互相影響

層級	消除的浪費	核心技術	觀察指標
01 排程	SCHEDULING 不同長度造成 idle slots	Continuous batching · Chunked prefill	GPU utilization / tok/s
02 KV 記憶體	KV MEMORY 連續配置、碎片、重複 prefix	Paged KV · Prefix cache	KV usage / cache hit
03 執行核心	KERNELS HBM 往返、launch 與同步	Fusion · FlashAttention · CUDA Graph	kernel time / launch gaps
04 低精度	PRECISION 每個參數與 KV 的 bytes 過多	FP8 · NVFP4 · quantized KV	model bytes / GEMM throughput
05 解碼演算法	DECODING 每個 token 都要一次昂貴 target step	Speculative · MTP · EAGLE	accepted tokens / target pass
06 跨 GPU	SCALING Prefill / Decode 特性混在同一資源池	P/D disaggregation · TP / EP	TTFT / TPOT / scaling efficiency

Serving engine 的價值：同時最佳化「模型計算」與「系統排程」，而非只換一個更快的 GEMM。

Continuous Batching：完成一個，下一輪立刻補位

同一組 request 長度不同；用固定 GPU slot 泳道看清楚 D / E 如何接手



完成 request 釋放 slot；scheduler 在下一個 decoding iteration 重新組 batch。

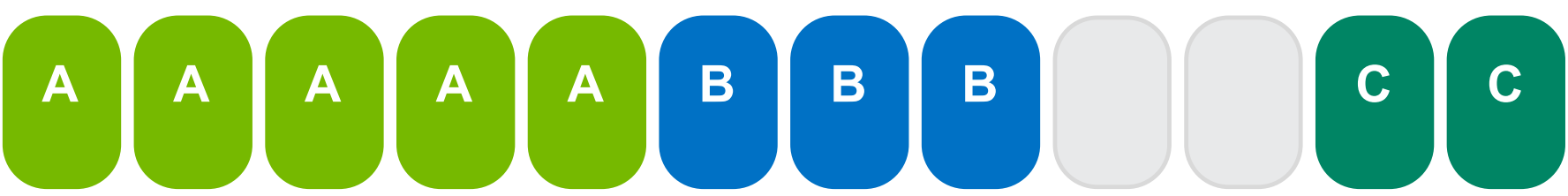
Paged KV + Prefix Cache：先解決配置，再重用計算

把兩個概念分開看：page table 決定 KV 放哪裡；prefix hash 決定哪些 KV 不必重算

1 CONCEPTUAL BASELINE

教學示意 · 非 vLLM / TRT runtime

GPU memory



A 需要再增加一個 block



系統只能在三者中取捨



結果：KV 容量利用率下降
同一張 GPU 能服務的 requests 變少

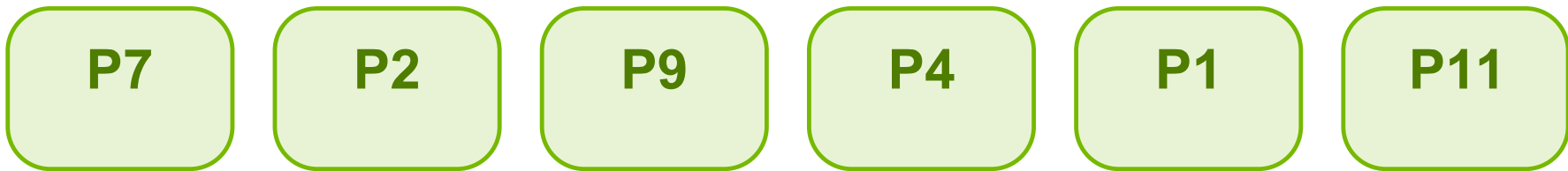
2 PAGED KV

logical blocks → block table → pages

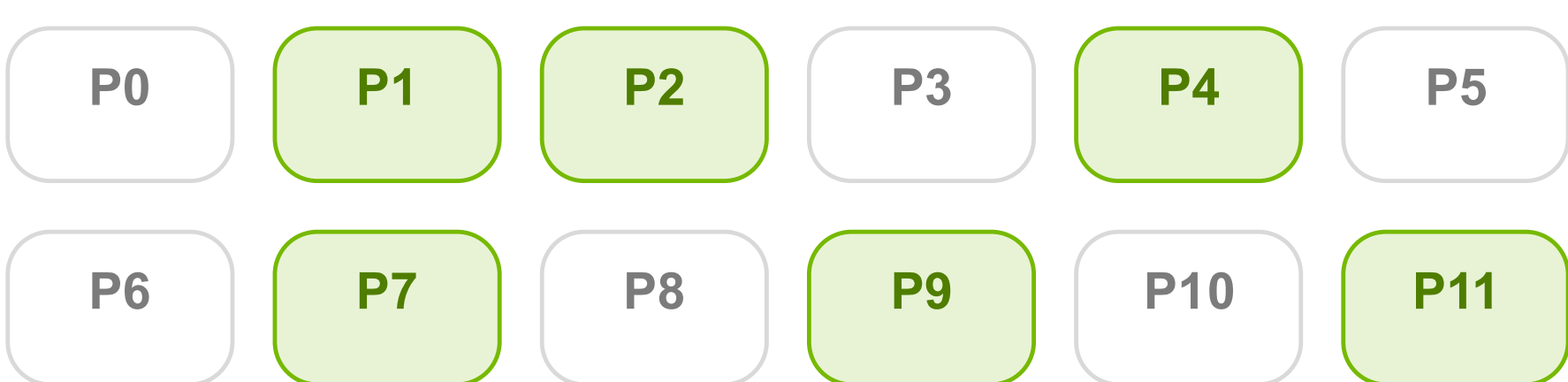
Req A logical blocks



block table



physical page pool (可散落)

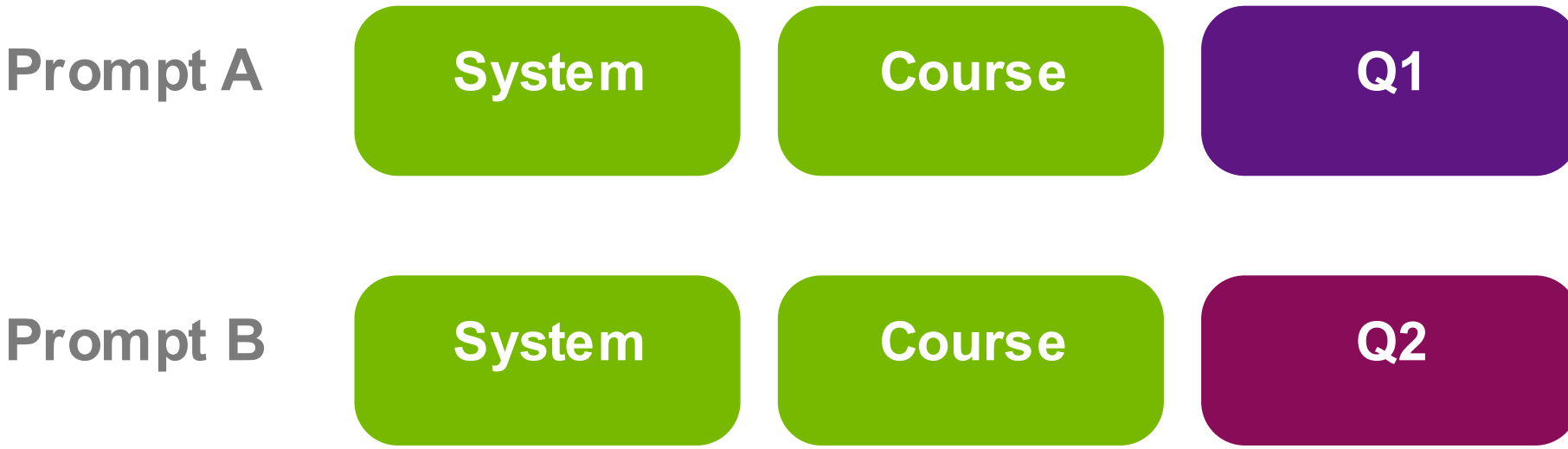


A 成長時拿任一 free page (P11)
舊 pages 不搬動；完成後可逐頁回收

解決「配置與碎片」

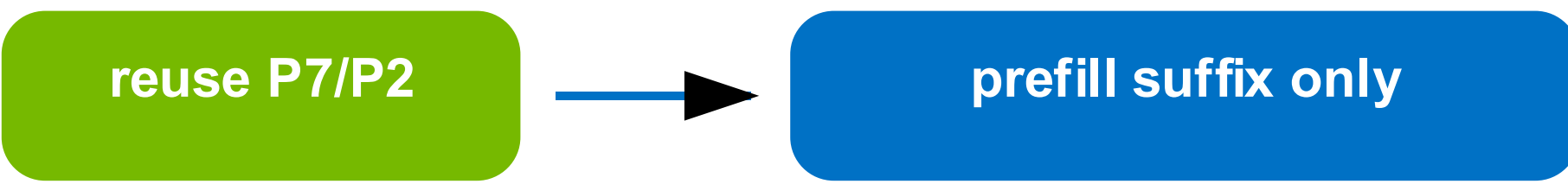
3 PREFIX CACHE

same token prefix → same KV pages



hash(parent, tokens, extra keys)
→ pages P7, P2 refcount = 2

cache hit 的動作



跳過重複的 prefix prefill
前提：tokens / model / cache key 相容

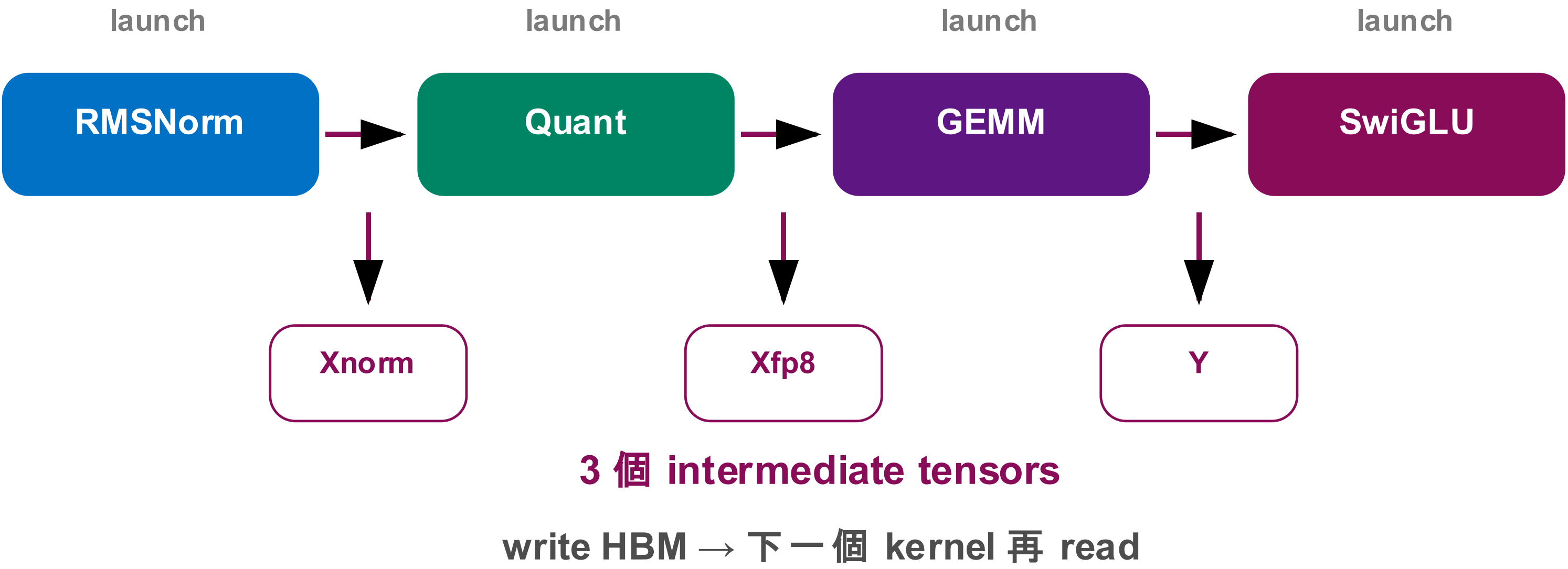
Paged KV 省「記憶體容量」；Prefix Cache 省「重複計算」。兩者相關，但不是同一個最佳化。

Kernel Fusion：少搬中間張量，通常比少幾個 FLOPs 更關鍵

Decode 常受 HBM bandwidth 與 launch latency 限制；融合的核心是把資料留在片上

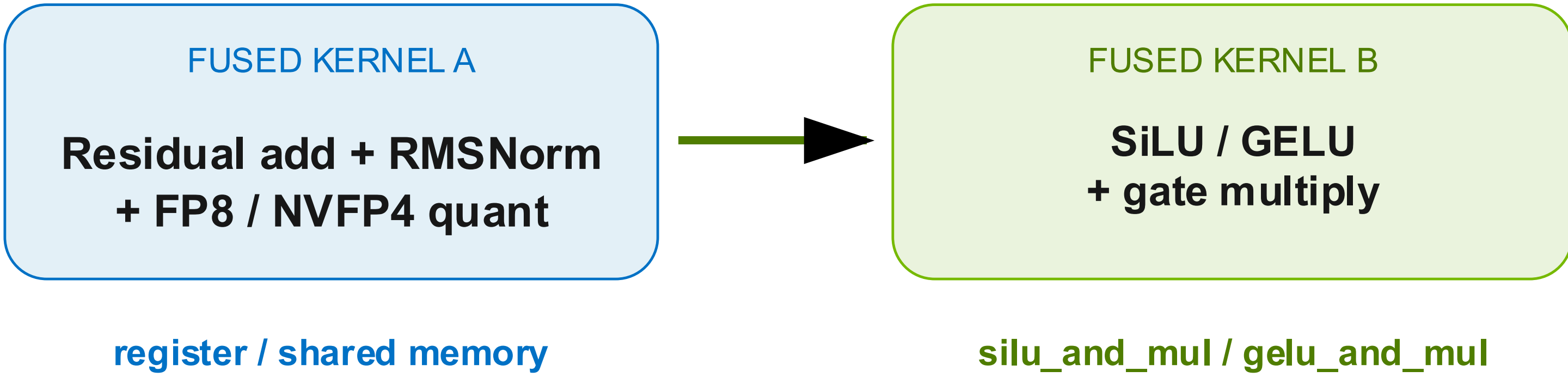
UNFUSED

每個 op 都寫回 HBM

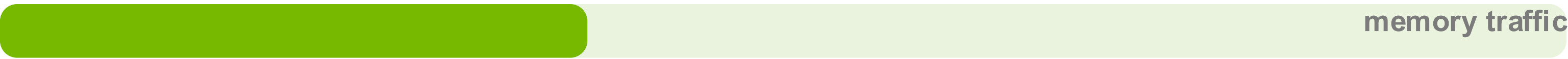


FUSED

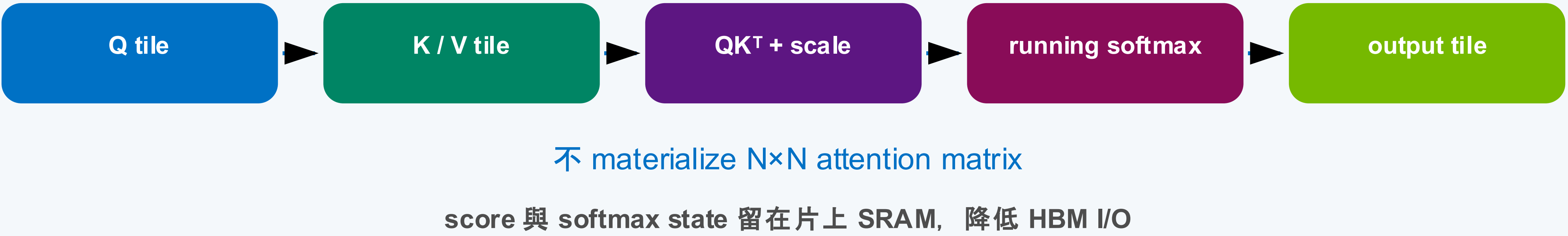
只在必要邊界落回 HBM



較少 HBM round trips · launch · global sync



FlashAttention：演算法 + kernel fusion



CUDA Graph ≠ Kernel Fusion



實作例：fused_add_rms_norm_quant、silu_and_mul；GEMM 仍可能是獨立 tuned kernel。

FP8 與 NVFP4：少 Bytes、更多 Tensor Core MACs

低精度同時影響權重頻寬、KV 容量與 GEMM；scaling granularity 決定速度與誤差的平衡

BF16

16-bit value



2 bytes / element

精度基準

FP8

8-bit value + scale



約 1 byte / element

Hopper / Blackwell

recipe 可為 per-tensor / row / block scaling

NVFP4

4-bit value + block scale

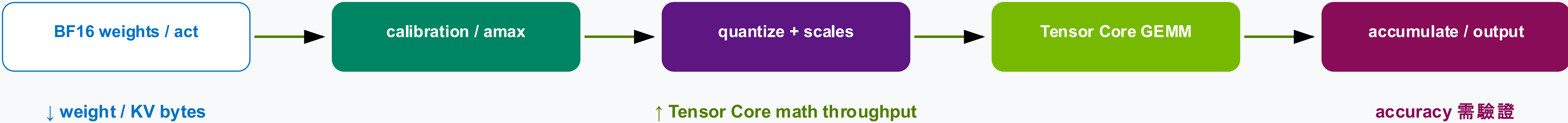


約 0.5 byte / element*

W4A4 native : Blackwell

W4A16 : vLLM Marlin · scale group = 16

LOW-PRECISION DATA PATH



*實際 footprint 含 scales / metadata；不是保證 2× 或 4×。需用目標模型與資料集做 calibration、perplexity / task eval。

Speculative Decoding : 先 Draft, 再由 Target 一次驗證

不跳過 Target ; 把多個 serial target steps 改成低成本 draft + 一次 batched verification

1 DRAFT

小模型提出多個候選 token

context → d₁ → d₂ → d₃ → d₄

4 cheap autoregressive draft forwards

Draft 很便宜, 但不是免費
draft cost 仍會影響淨加速

2 TARGET VERIFY

一次 forward 計算多個位置 logits

context d₁ d₂ d₃ d₄



TARGET MODEL · one forward



p₁ p₂ p₃ p₄ p₅

Target 沒有被省略 ; 只是一次驗證多個位置

3 ACCEPT / REJECT

接受 prefix ; mismatch 後由 Target 接手

POSITION DRAFT TARGET RESULT

t+1	d ₁	d ₁	ACCEPT
t+2	d ₂	d ₂	ACCEPT
t+3	d ₃	x ₃	REJECT
t+4	d ₄	—	DISCARD

輸出 : d₁, d₂, x₃

GREEDY

draft token = Target argmax 才接受

第一個不一致的位置停止, 後續候選全部丟棄

SAMPLING / REJECTION SAMPLING

accept probability = min(1, p(x) / q(x))

拒絕時從 correction distribution 取樣, 保持 Target 分布

Draft 改變速度, 不應改變模型品質

淨加速取決於 acceptance rate、draft length、draft overhead 與 batch size



Speculative Decoding: Off

Write me a short story explaining speculative decoding.

Imagine



Speculative Decoding: On

Write me a short story explaining speculative decoding.

Imagine a mad

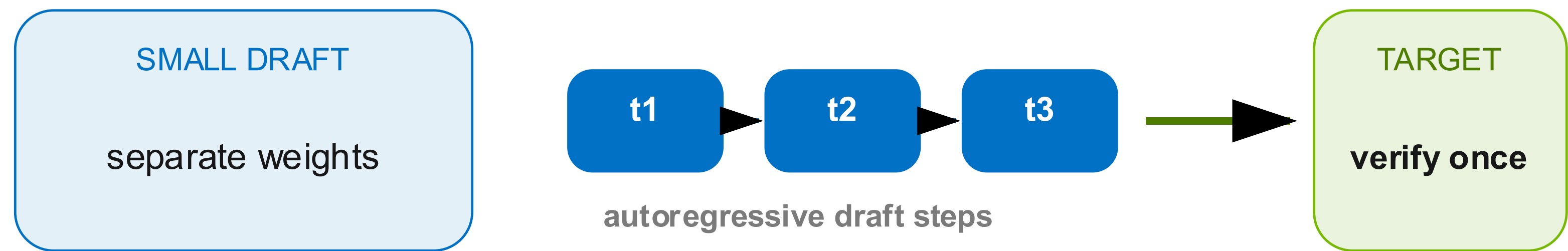
Speculative Decoding vs. MTP : 共同驗證，不同 Draft 來源

上一頁的 Target verification 相同；這一頁只比較候選 token 從哪裡來

MTP 是 Speculative Decoding 的一種 draft 機制，不是另一個驗證框架

CLASSIC DRAFT / TARGET

draft 來源：獨立的小模型

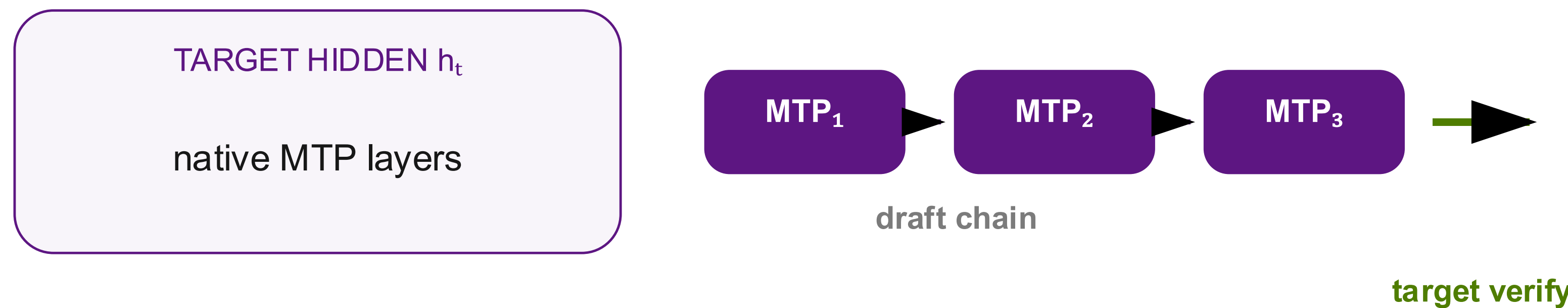


優點 可搭配多數 target (tokenizer / vocab 必須相容)

成本 需要額外模型、KV 與多個 draft forward

MTP DRAFT

draft 來源：native MTP layers (TRT 亦支援 EAGLE-style)



優點 draft 與 target 共同訓練 / 原生整合，額外成本較小

限制 模型必須提供 MTP weights；支援與 draft depth 受架構限制

共同驗證流程

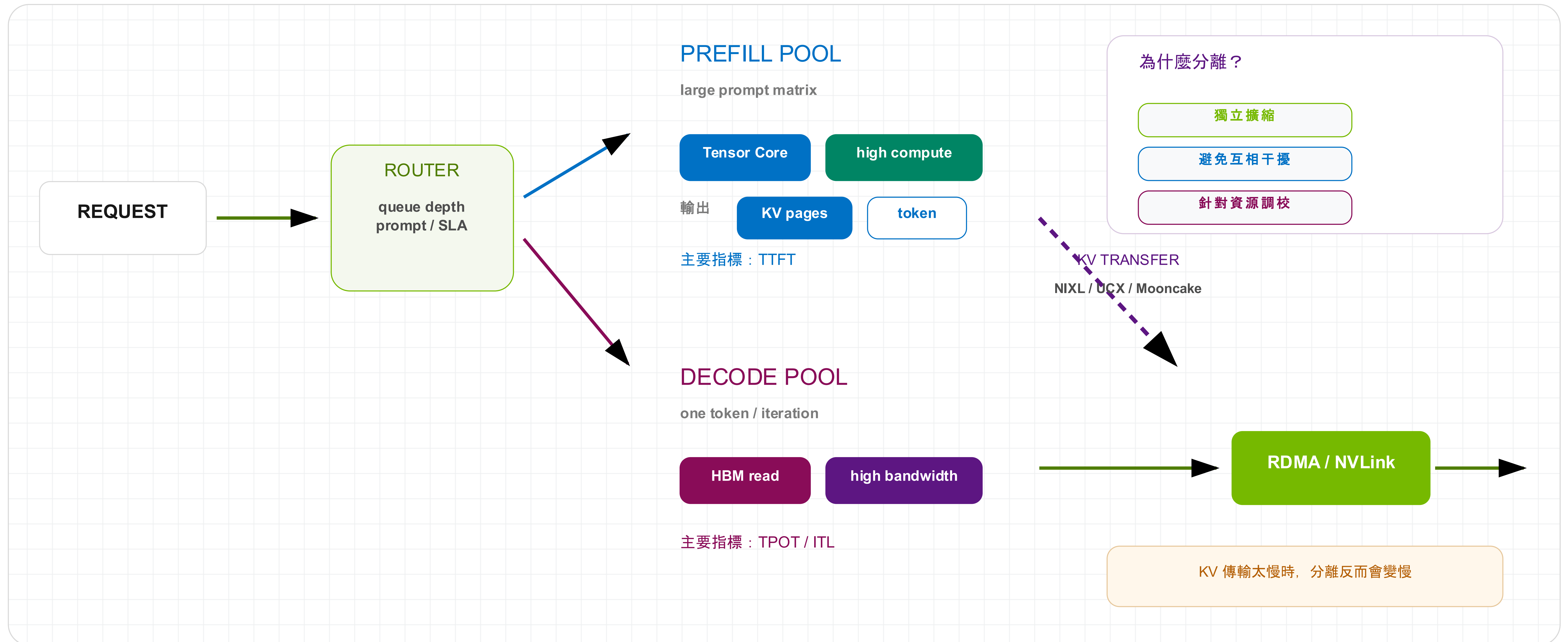


Strict / standard path 接受到第一個 mismatch；TRT MTP 可選 thinking-phase relaxed acceptance。

收益取決於 acceptance rate、draft length、drafting overhead 與 batch size；高 batch 不一定加速。

Prefill / Decode 分離：不同 GPU 做不同工作

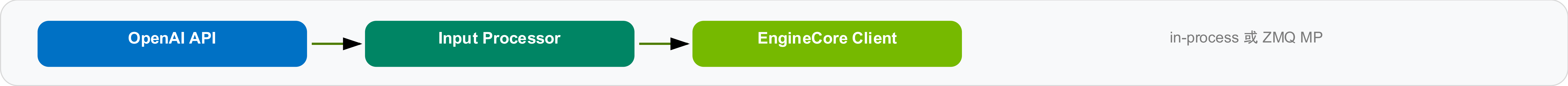
Prefill 偏 compute ; Decode 偏 memory bandwidth ; 中間以 KV pages 交接



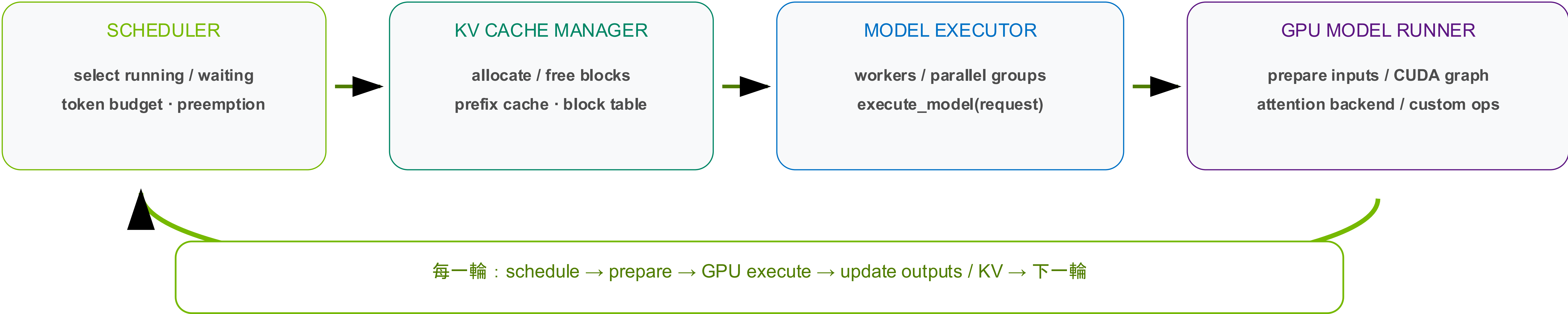
實作：TRT 預設 NIXL；vLLM connectors 對接 NIXL / Mooncake / LMCache。先量測 KV transfer 成本。

vLLM : Token-Centric Runtime

EngineCore 把 scheduler、KV cache manager 與 GPU runner 組成逐 iteration 的控制迴圈



ENGINECORE — ITERATION LOOP



token-centric : 排程與 KV 管理以「本輪能處理多少 tokens」為核心

強項

快速支援新模型 / backend ; Python runtime 易擴充

典型最佳化

continuous batching · PagedAttention · prefix cache

實作特徵

scheduler 與 KV manager 是一等公民

GPU 路徑

PyTorch + Triton / CUDA / FlashAttention backends

TensorRT-LLM : NVIDIA GPU 上的 LLM 推論函式庫

整合模型執行、In-flight Batching、Paged KV、量化、平行化與 Serving API

LLM API

trtllm-serve

trtllm-bench

同一組部署 / benchmark 入口

COMMON RUNTIME

Executor / Scheduler

KV Cache / Batching

Parallelism / NCCL

Speculative / Quant

↓ 從模型載入、排程與 KV 管理，到 optimized kernels 與 multi-GPU

MODEL RUNTIME

PYTORCH

- eager / torch.compile / CUDA Graph
- TRT-LLM ops + model implementations
- 支援模型快速整合與擴充

SERVING RUNTIME

SCHEDULER / KV

- in-flight batching / scheduler
- Paged KV · prefix reuse
- speculative decoding · streaming

GPU OPTIMIZATION

NVIDIA STACK

- FP8 / NVFP4 / quantized GEMM
- TP / PP / EP · NCCL
- CUTLASS / cuBLASLt / custom kernels

GPU FOUNDATION

CUDA / CUTLASS / cuBLASLt / attention kernels / quantized GEMM / NCCL

NVIDIA hardware–software co-design

PyTorch vs vLLM vs TensorRT-LLM

兩者都含完整 runtime ; TensorRT-LLM 現行正式路徑是 PyTorch backend

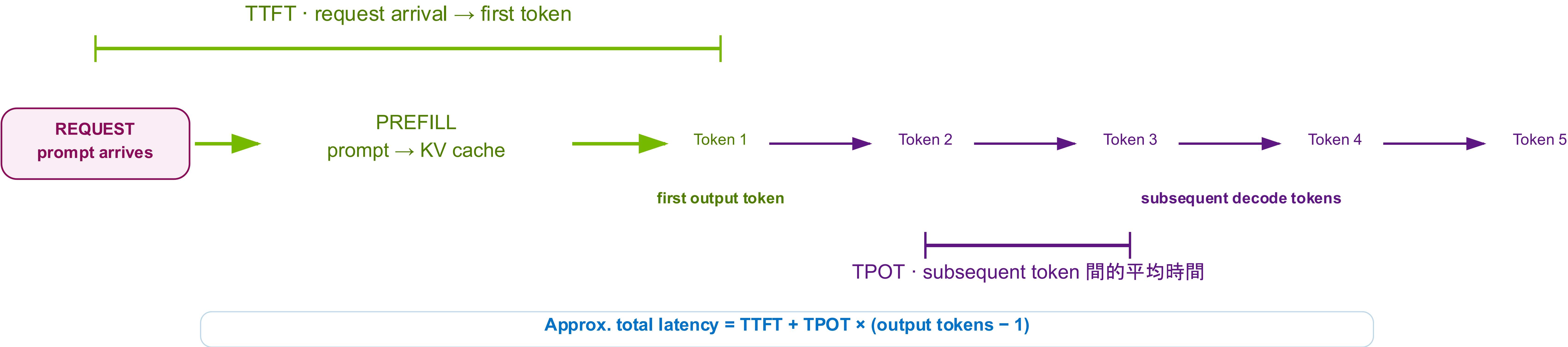
面向	PyTorch / HF	vLLM	TensorRT-LLM
目標	模型執行 / 研究	通用 LLM serving	NVIDIA serving
Batch	fixed / 自行排程	per-iteration token budget	Capacity + MicroBatch
KV	一般 tensors	paged block pool	paged KV / block reuse
精度	BF16 / FP16 / 擴充	多量化 backend	FP8 / NVFP4 深度整合
適合	彈性高，需自行服務化	多平台、模型變動快	NVIDIA GPU、runtime 深度調校

高並發、長 context、共用 prefix，最能放大 serving engine 的優勢。

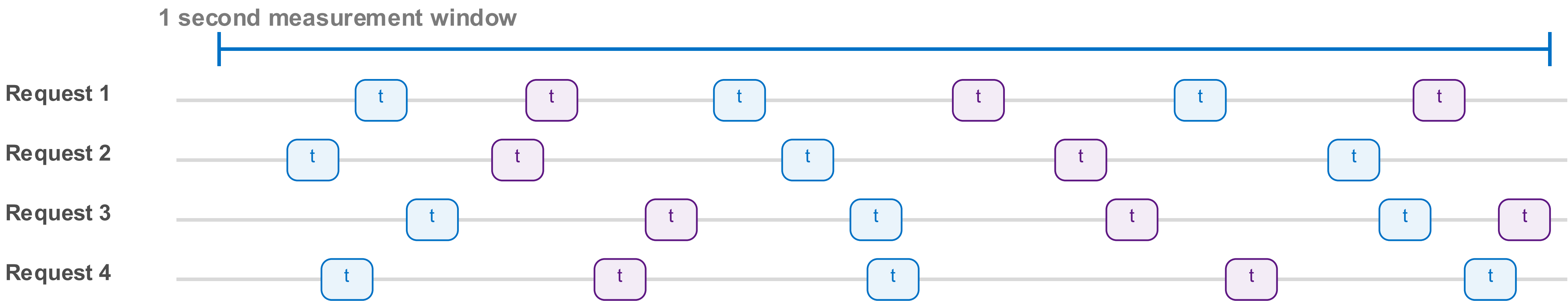
LLM Serving Benchmark：看懂 TTFT、TPOT 與 Throughput

單一 request 的 latency 與整體服務容量要分開量測

SINGLE REQUEST LATENCY



SYSTEM THROUGHPUT



Throughput

Σ output tokens / second

跨所有 concurrent requests 加總

HIGHER IS BETTER

結論：快速 LLM 推論來自多層最佳化

排程、KV 記憶體、Kernel、低精度、解碼與系統擴展必須共同最佳化

01

SCHEDULING

- Continuous batching 填補 idle slots
- Chunked prefill 平衡延遲與吞吐
- Token-centric scheduling 提高利用率

02

KV MEMORY

- Paged KV 降低配置碎片
- Prefix Cache 避免重複 prefill
- 長 context 需要容量與回收策略

03

KERNEL / PRECISION

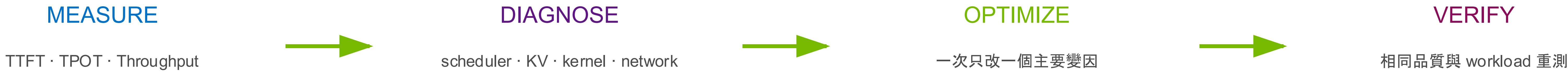
- FlashAttention / fusion 減少 HBM I/O
- CUDA Graph 降低 launch gaps
- FP8 / NVFP4 必須驗證品質

04

DECODING / SCALING

- Speculative / MTP 減少 target steps
- P/D 分離不同資源特性
- 多 GPU 通訊決定 scaling efficiency

實務落地流程



先量測瓶頸，再選擇最佳化；沒有任何單一技術能同時改善所有 workload。

Materials Science





AI & Data Science



Edge AI Inference



Machine Learning



Autonomous Vehicles



Materials Simulation



Robotics



Medical Imaging



Manufacturing



Remote Collaboration



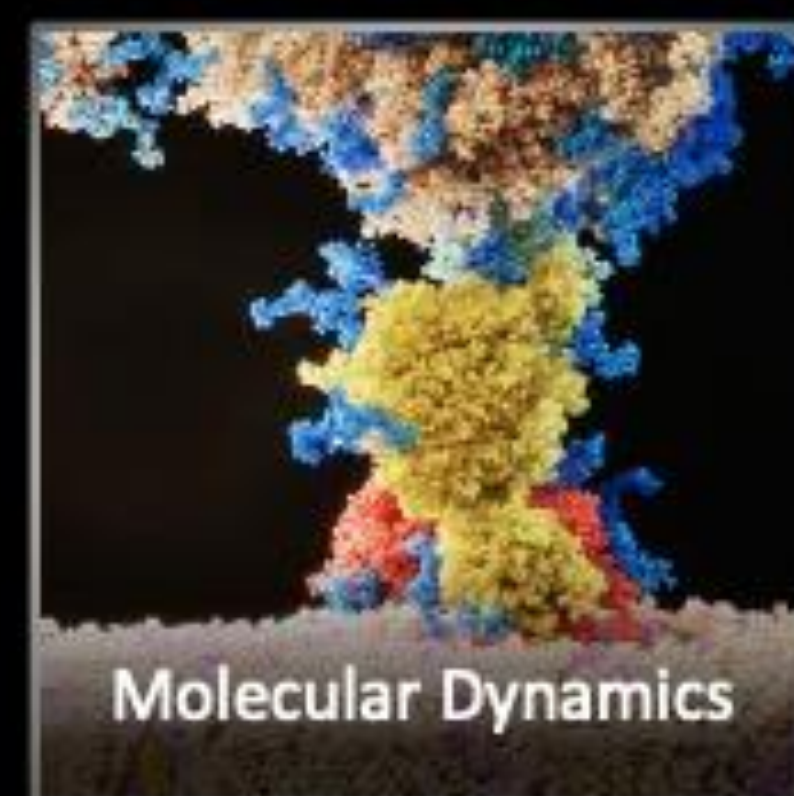
Virtual Workstations



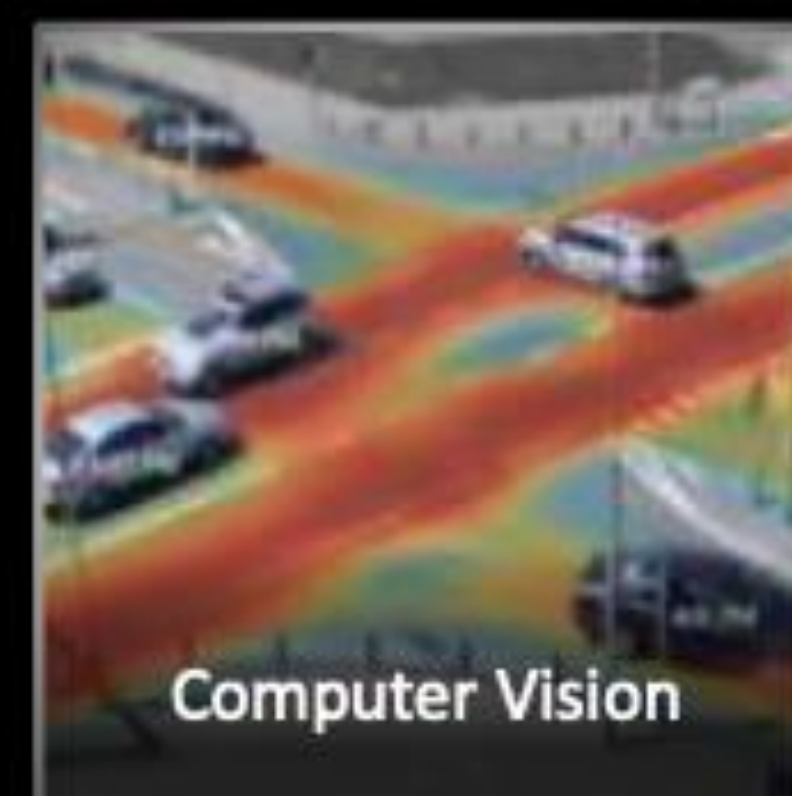
Astronomy &
Astrophysics



Collaborative Design



Molecular Dynamics



Computer Vision



Telecoms



Rendering and Ray
Tracing



Professional
Visualization



Game Development



AR / VR



Earthquake Simulation



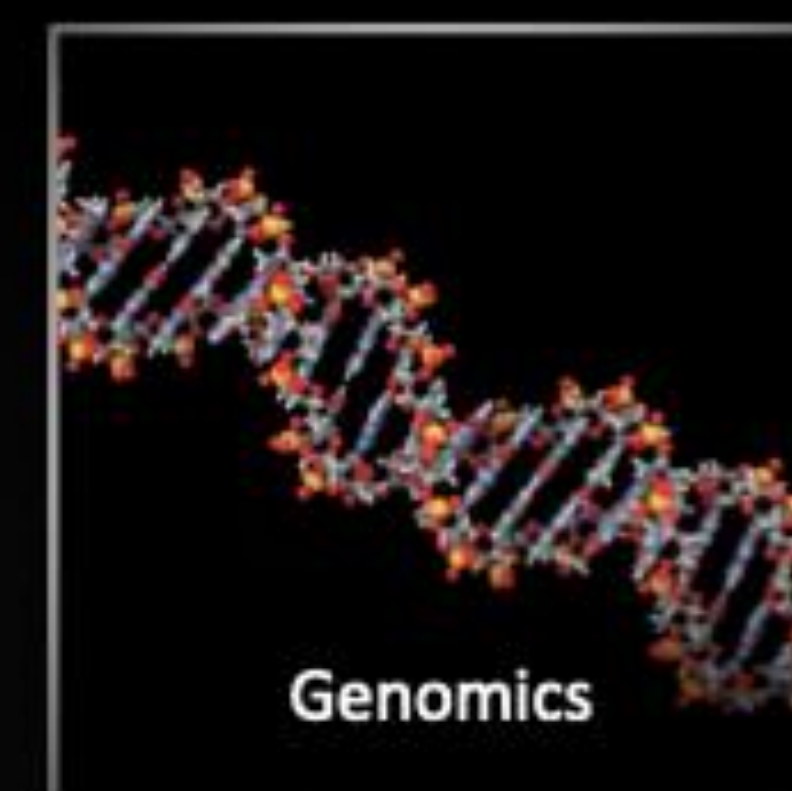
CAE Simulation



Climate and Weather



Scientific Visualization



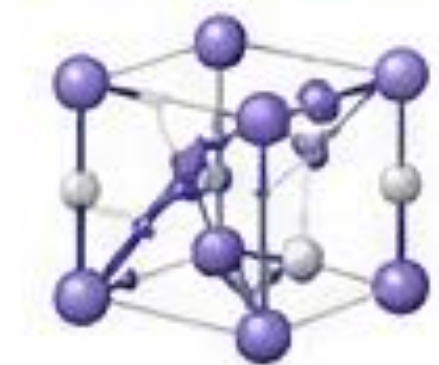
Genomics

Universe of GPU Computing

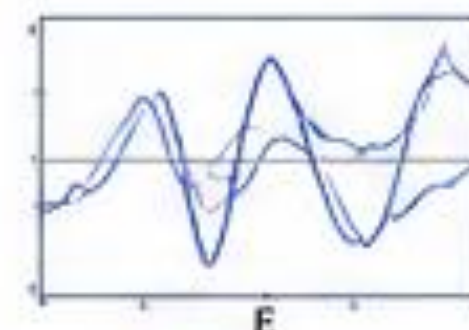
Quantum ESPRESSO: What is it and What is it for?

An open-source suite of codes for electronic-structure calculations based on Quantum Mechanics (DFT)

WHAT CAN QUANTUM ESPRESSO DO?



Structure Optimization
Find the most stable atomic arrangements.



Electronic Structure
Calculate band structure, density of states (DOS), etc.



Electronic Properties
Compute charge density, bonding, dielectric properties, magnetic properties, etc.



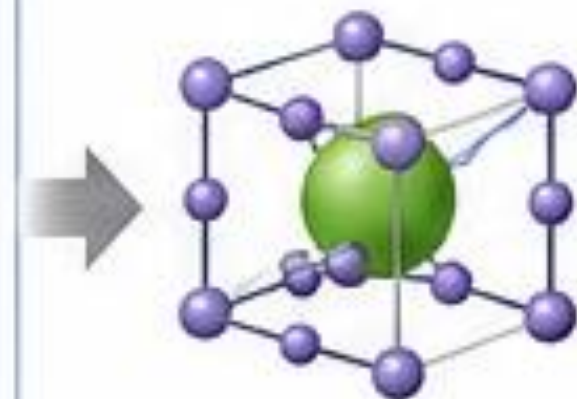
Molecular Modeling
Study reactions, adsorption, vibrations (phonons), frequencies, etc.



Materials Design & Prediction
Investigate energy, catalysis, semiconductors, magnets, polymers, and more.

THE CONCEPT: FIRST-PRINCIPLES CALCULATION (DFT)

1. Input Structure
Atomic species and positions



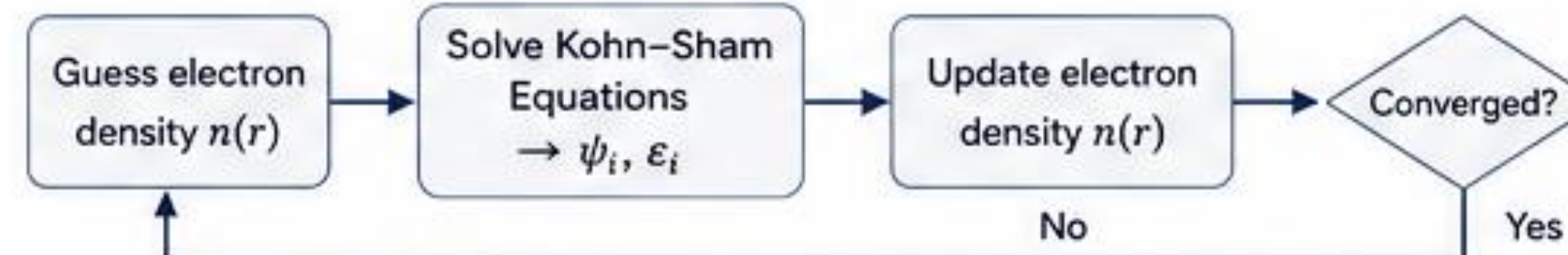
2. Build the Quantum Mechanical Model
Based on Density Functional Theory (DFT) (Kohn–Sham DFT)

Kohn–Sham Equations

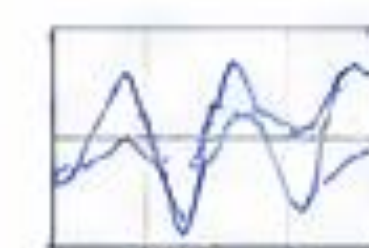
$$\left[-\frac{\hbar^2}{2m} \nabla^2 + V_{\text{ion}}(\mathbf{r}) + V_H(\mathbf{r}) + V_{\text{xc}}[n(\mathbf{r})] \right] \psi_i(\mathbf{r}) = \varepsilon_i \psi_i(\mathbf{r})$$

Goal: self-consistently solve for the electron density $n(\mathbf{r})$ and total energy E .

3. Self-Consistent Field (SCF) Loop



4. Output Results



Band Structure



Density of States (DOS)



Charge Density



Total Energy, Forces, Stress, etc.

THE ROLE OF NVIDIA HPC-SDK

A comprehensive HPC development toolkit that enables Quantum ESPRESSO to run efficiently on NVIDIA GPUs and heterogeneous systems.



High-Performance Fortran Compiler (NVFORTRAN)

Best-in-class Fortran compiler with GPU acceleration support (OpenACC / CUDA Fortran).



Optimized Math Libraries

GPU-accelerated libraries such as cuBLAS, cuFFT, cuSOLVER, NVPL (Libsci) for high-performance linear algebra and FFT.



Parallel Programming & Communication

NVIDIA SHMEM, NVSHMEM, MPI and other high-performance communication libraries for scaling to GPUs and multiple nodes.



Integration & Performance Tuning

Tightly integrated with NVIDIA GPU architectures, greatly accelerating DFT calculations and extending capabilities for large-scale simulations.

THE WORKFLOW

1. Prepare Input

- Structure files (atomic positions)
- Calculation parameters (DFT settings)
- Pseudopotentials (PSPs)

2. Run with Quantum ESPRESSO



Core code (Fortran)

- Build the model
- SCF calculation
- Produce results

3. Accelerated by NVIDIA HPC-SDK

- High-performance compilers (NVFORTRAN)
- Optimized math libraries
- GPU acceleration
- Parallel communication



4. Execute on NVIDIA GPUs

Massively parallel execution on GPU clusters / nodes



5. Get High-Quality Results

- Electronic structure
- Material properties
- Scientific insights



APPLICATION AREAS

Energy Materials



Batteries, fuel cells, photocatalysts, etc.

Semiconductors & Electronics



Device engineering, defect analysis, new material design

Catalysis & Surfaces



Adsorption, reaction mechanisms, catalyst design

Magnetic Materials



Magnetic structures, exchange interactions, spintronics

Extreme Conditions



High pressure, high temperature, planetary materials

Biomolecules & Pharmaceuticals



Molecular structure, binding energy, drug design



SUMMARY: Quantum ESPRESSO provides the state-of-the-art algorithms and physics to understand materials and molecules from first principles, while NVIDIA HPC-SDK delivers the performance and scalability on modern GPU/HPC platforms—together enabling faster simulations, larger systems, and deeper insights into the material world.

Open Source + Open Acceleration



Better Science. Faster.

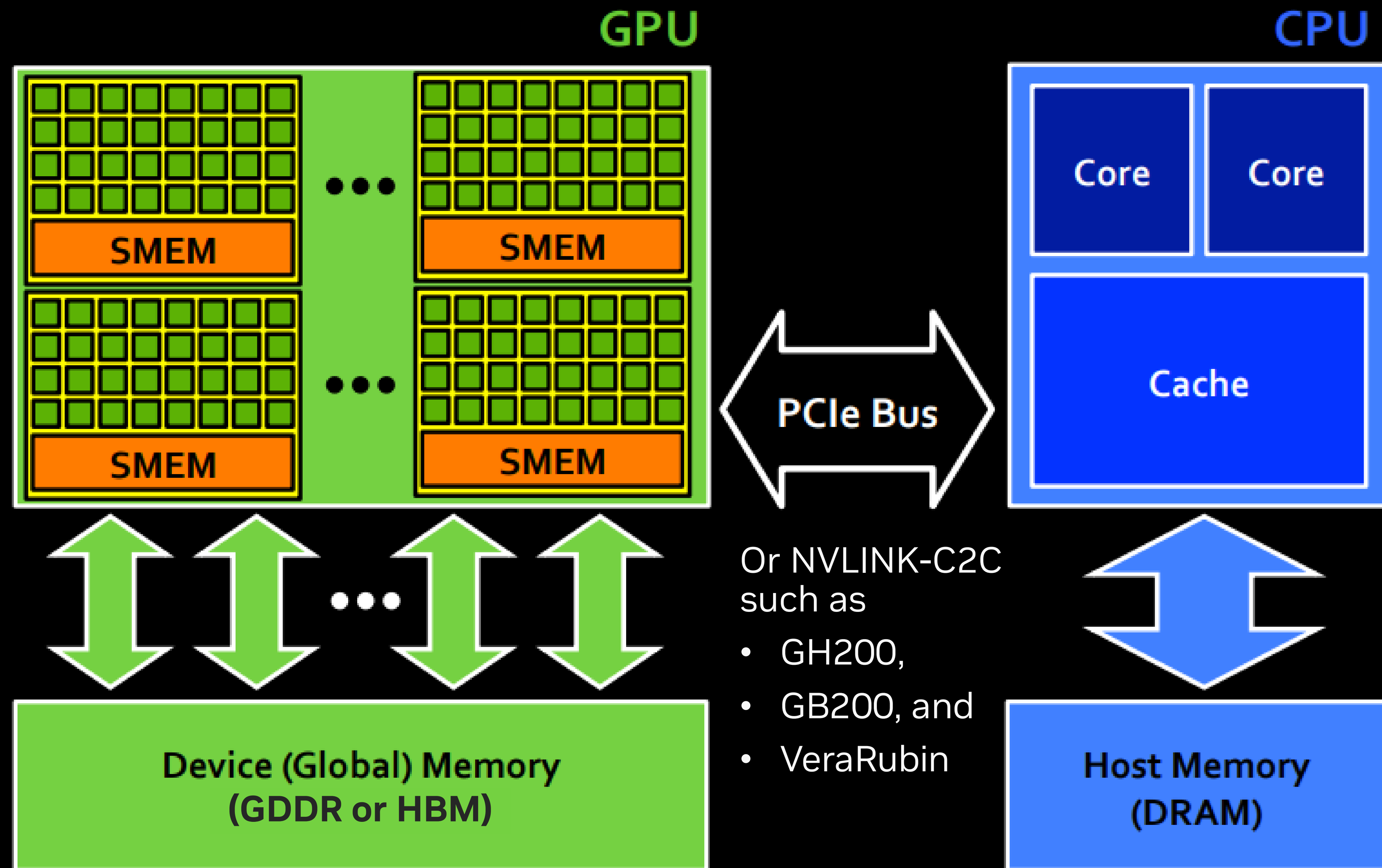
Heterogeneous Programming

GPU Strengths

- High bandwidth main memory
- Significantly more compute resources
- Latency tolerant via parallelism
- High throughput
- High performance/watt

GPU Weaknesses

- Relatively low memory capacity
- Low per-thread performance



CPU Strengths

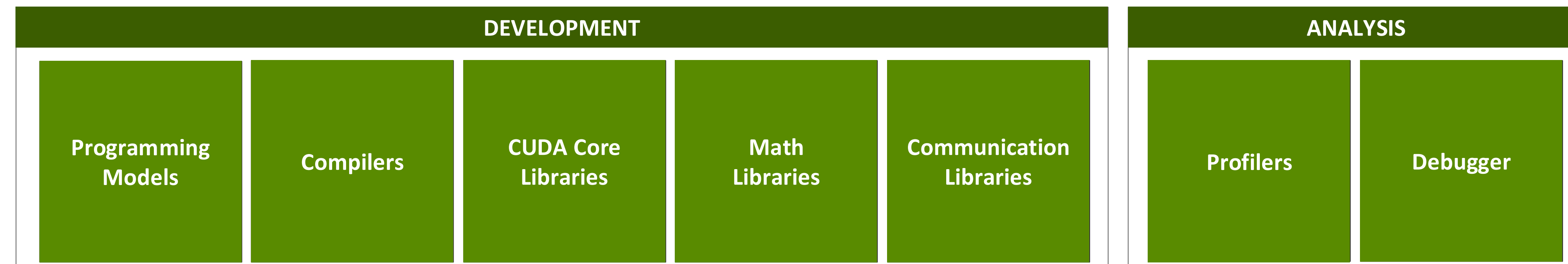
- Very large main memory
- Very fast clock speeds
- Latency optimized via large caches
- Small number of threads can run very quickly

CPU Weaknesses

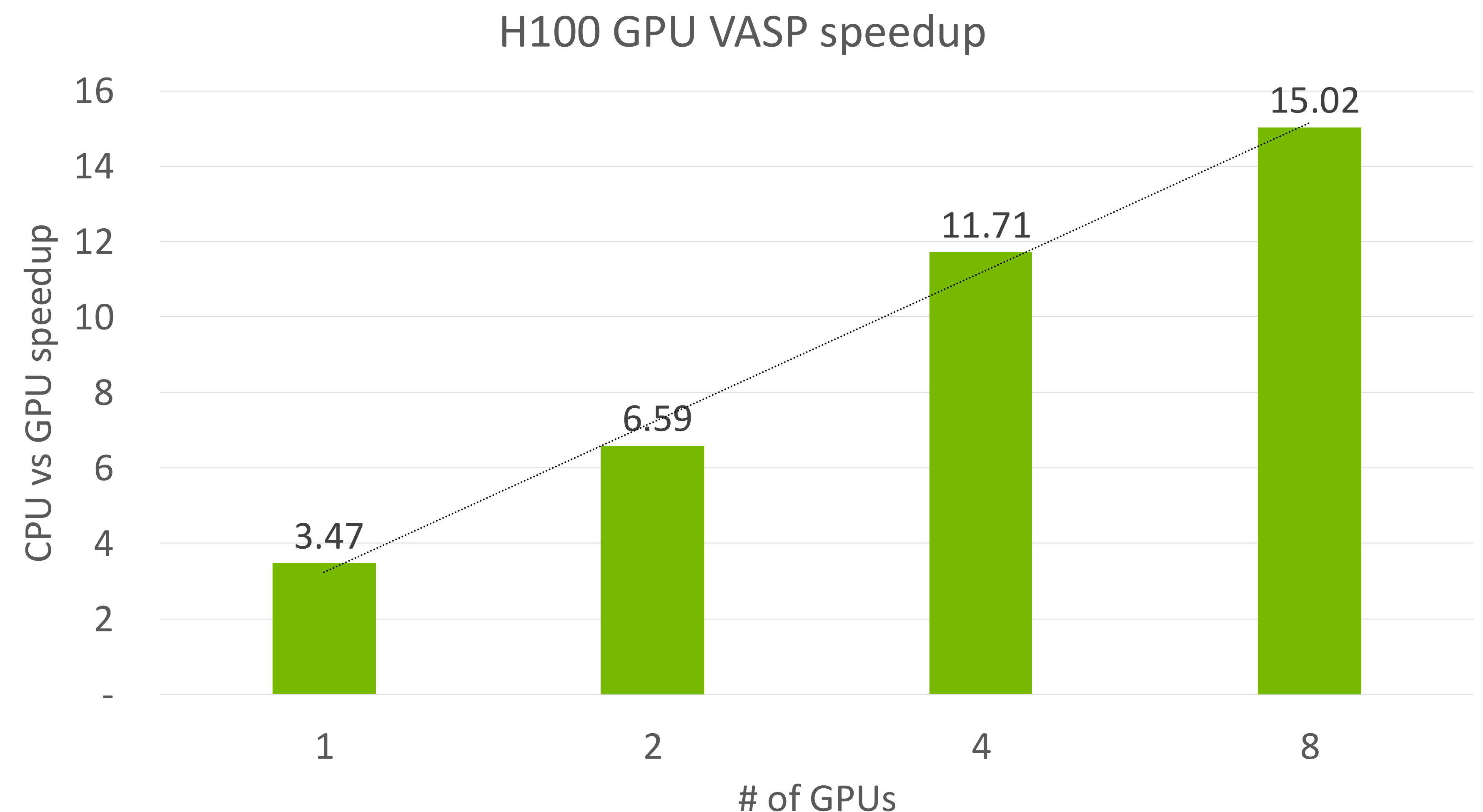
- Relatively low memory bandwidth
- Cache misses very costly
- Low performance/watt

NVIDIA HPC SDK

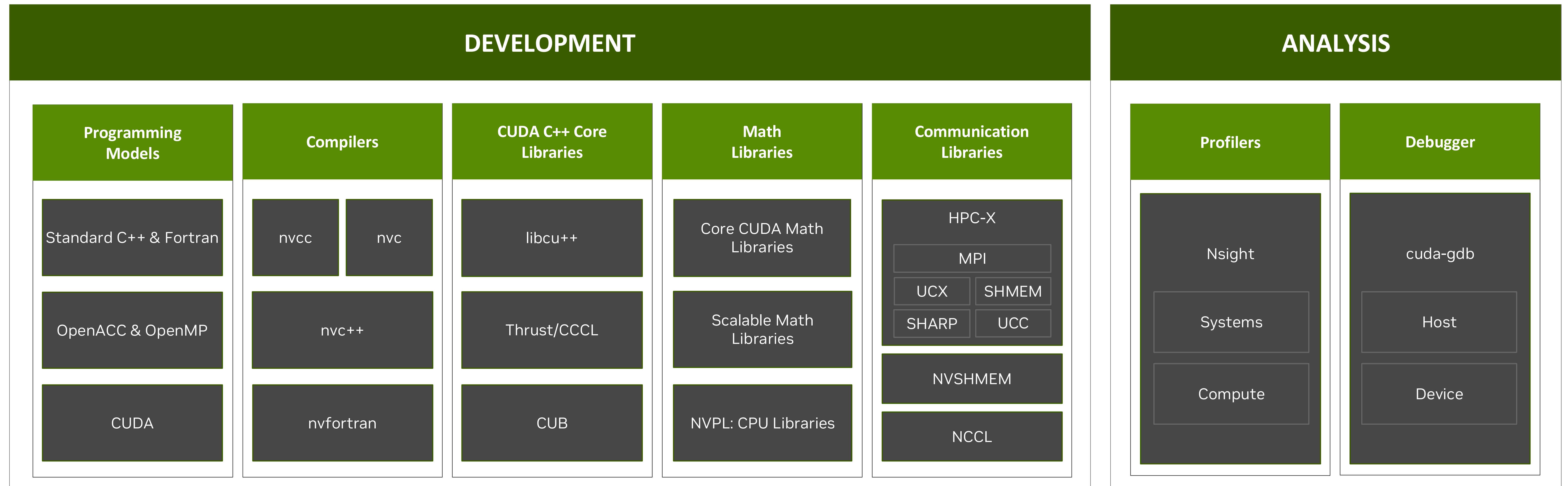
Comprehensive developer environment for users who need reliable, scalable, accelerated solutions



- Available at developer.nvidia.com/hpc-sdk, on NGC, via Spack, and in the Cloud
- Simplified deployment, access, and use of developer environment software to accelerate applications
- Guaranteed compatible software that is performance-tested on top HPC applications
- Out-of-the-box optimizing compilers, programming choice, libraries, debugging and profiling insight, all tuned for NVIDIA GPUs, CPUs, and interconnect

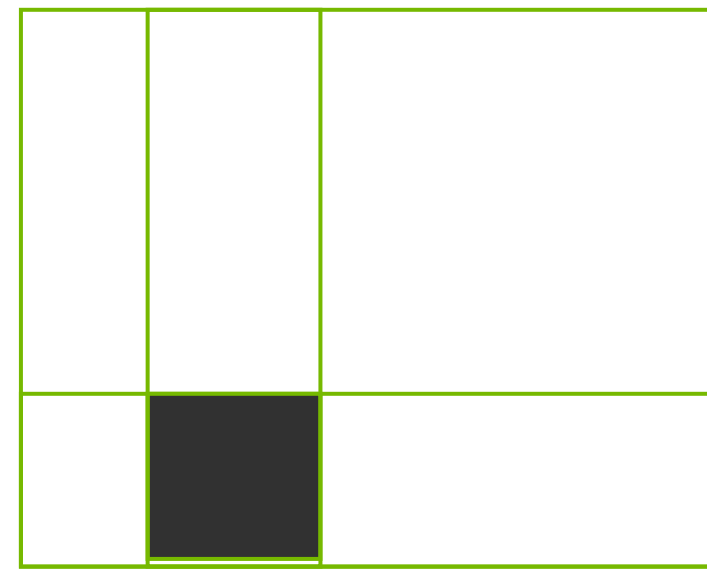


NVIDIA HPC SDK Components



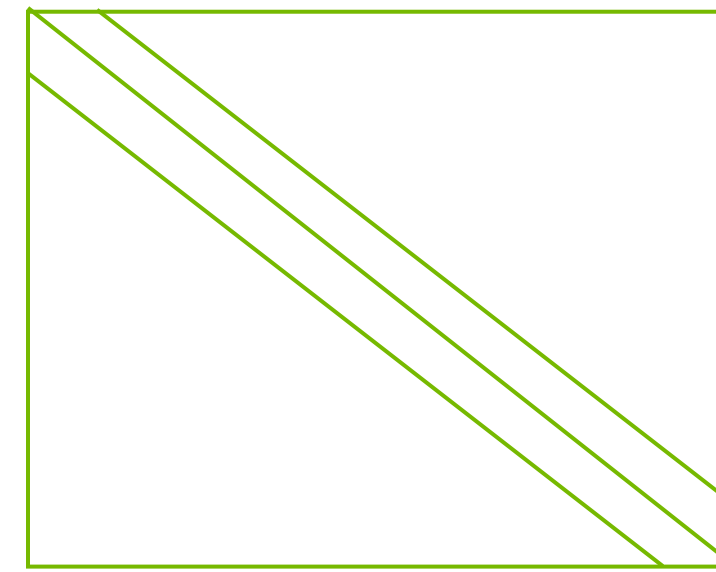
- Tuned for the NVIDIA Platform: GPUs, CPUs, and Interconnect
- Packages for x86_64 and Arm

GPU accelerated Math libraries



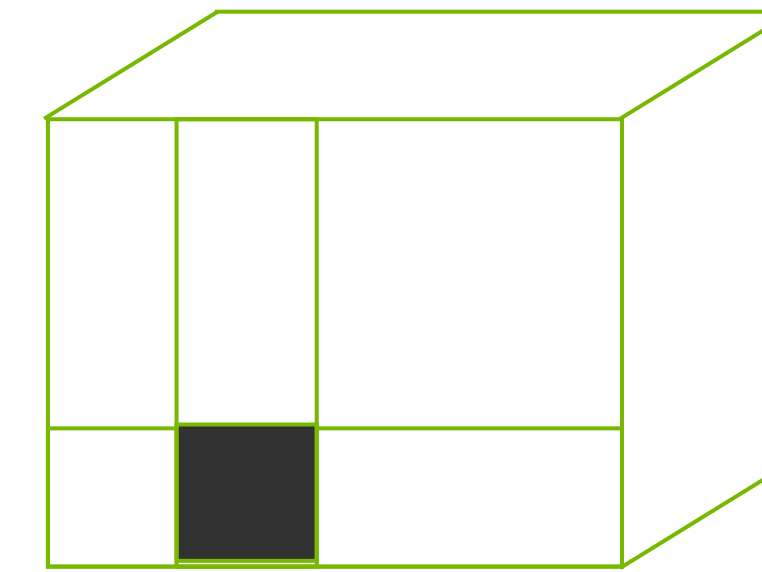
cuBLAS

BF16, TF32 and FP64
Tensor Cores



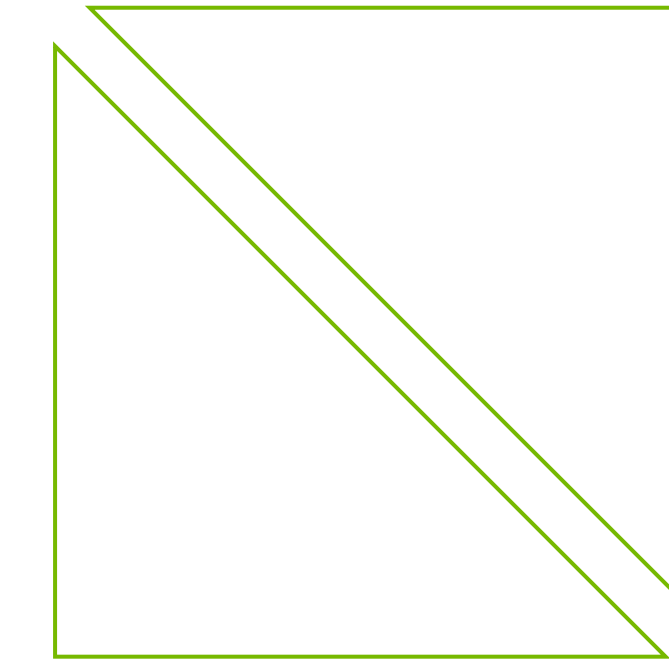
cuSPARSE

Increased memory BW,
Shared Memory & L2



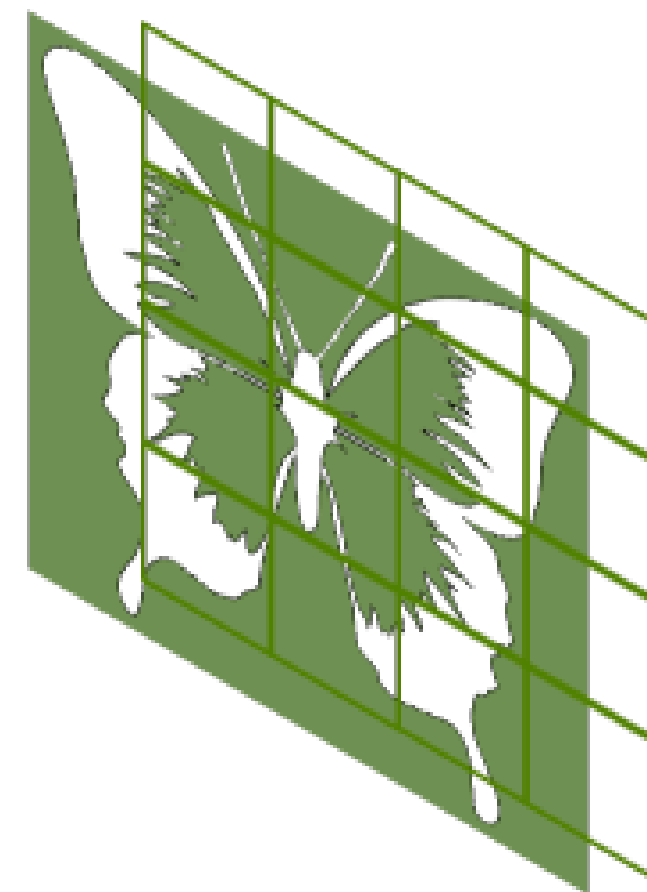
cuTENSOR

BF16, TF32 and FP64
Tensor Cores



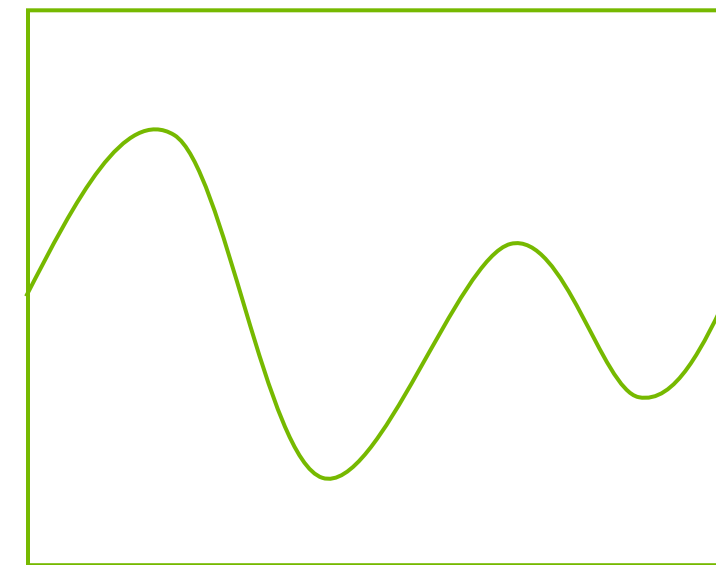
cuSOLVER

BF16, TF32 and FP64
Tensor Cores



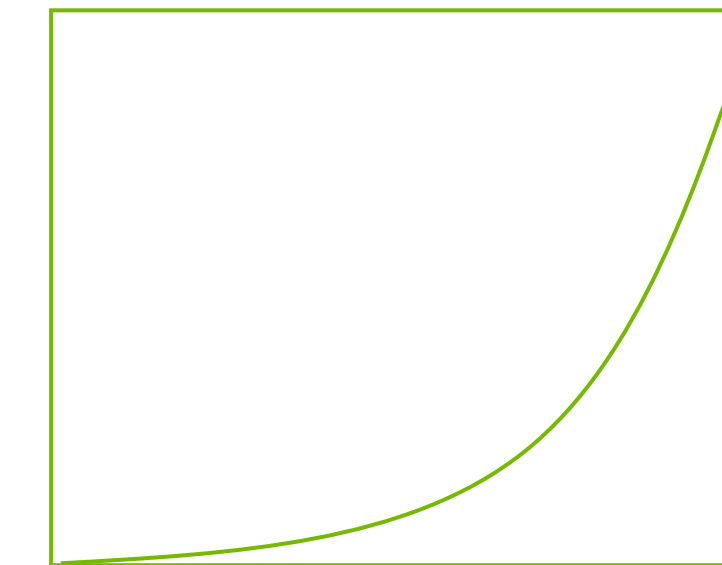
nvJPEG

Hardware Decoder



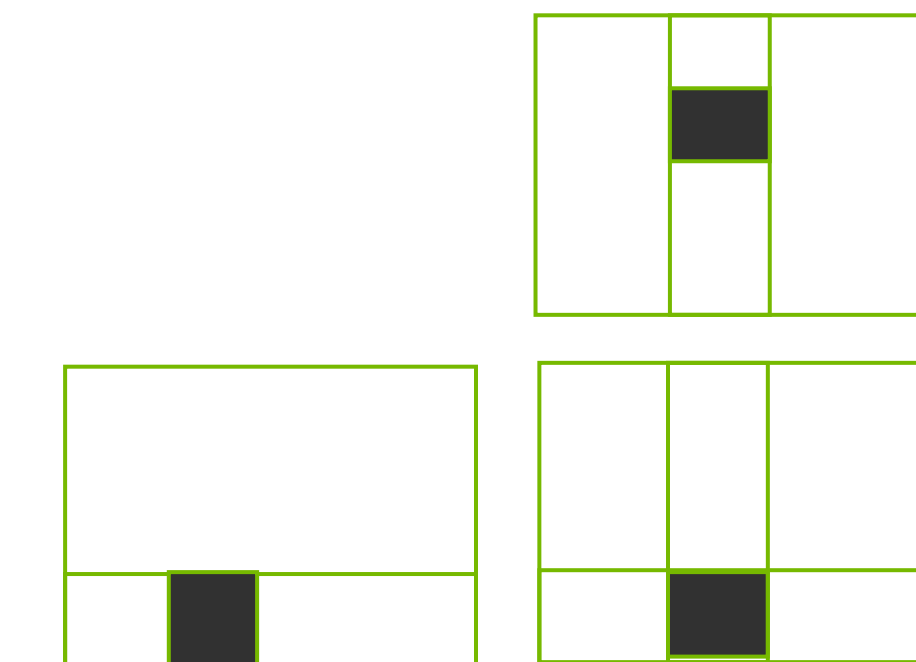
cuFFT

BF16, TF32 and FP64
Tensor Cores



CUDA Math API

Increased memory BW,
Shared Memory & L2



CUTLASS

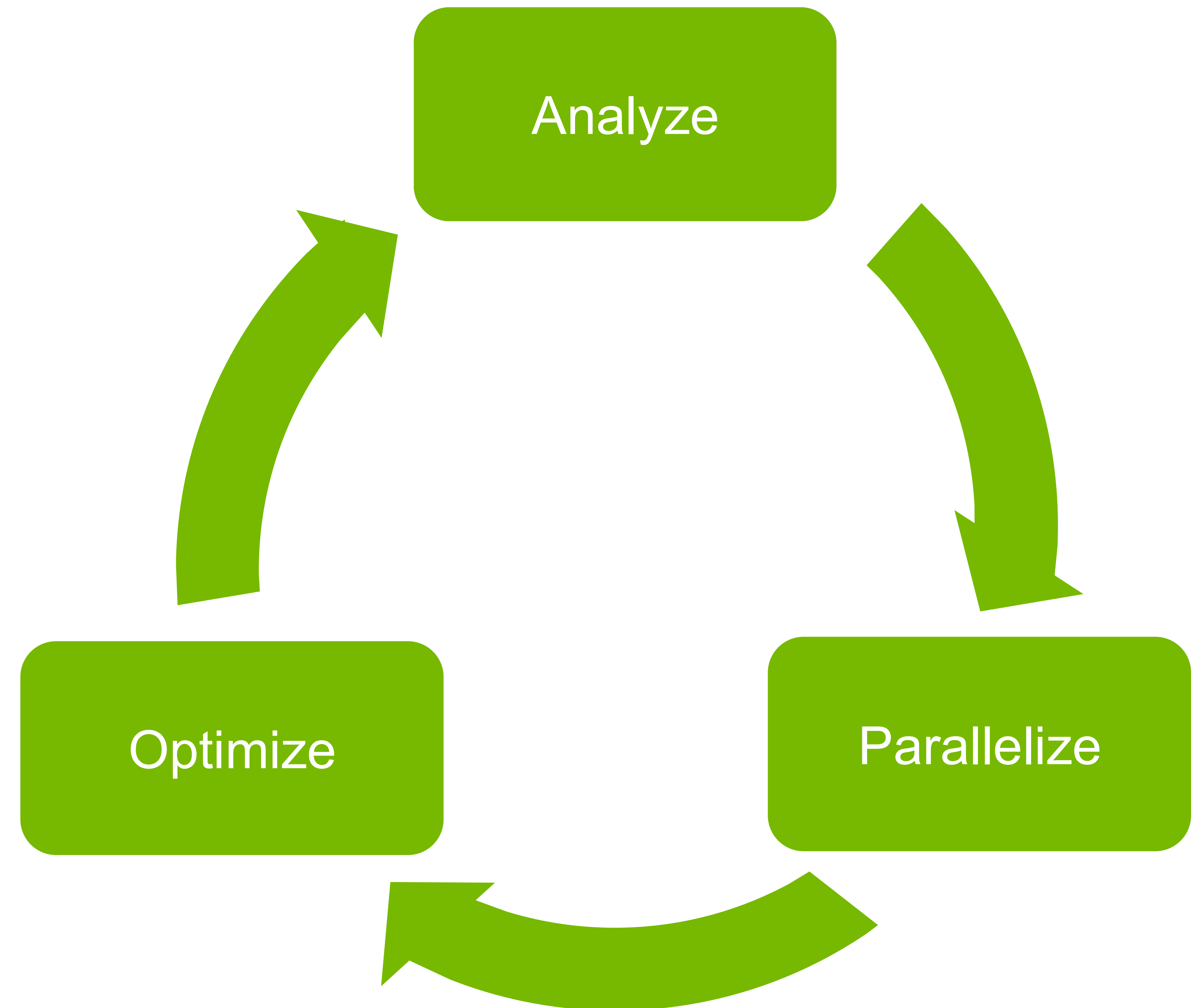
BF16 & TF32 Support

Development Cycle

Analyze your code to determine most likely places needing parallelization or optimization.

Parallelize your code by starting with the most time consuming parts and check for correctness.

Optimize your code to improve observed speed-up from parallelization.



Nsight Product Family

Workflow

Nsight Systems

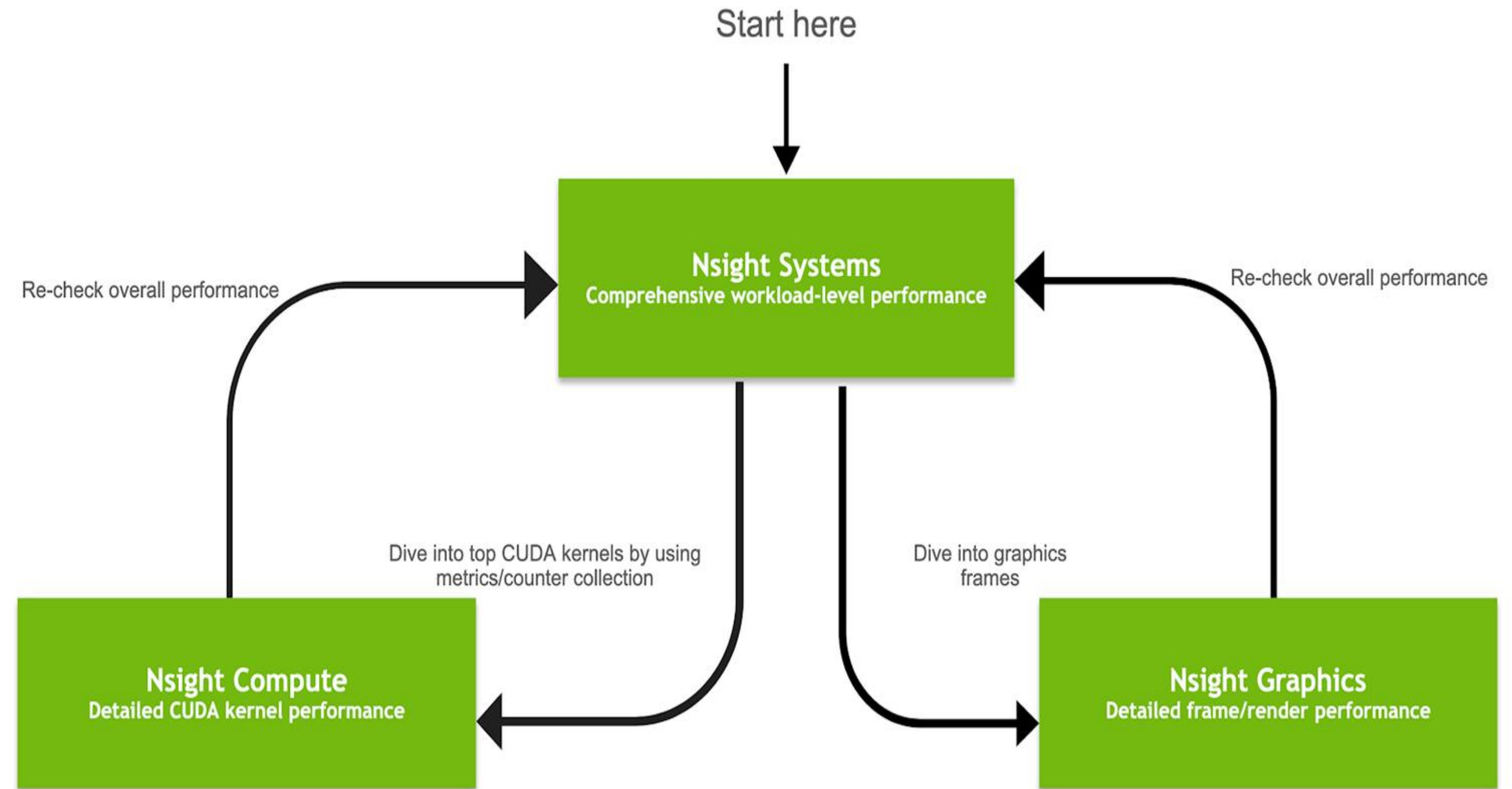
- Analyze application algorithm system-wide

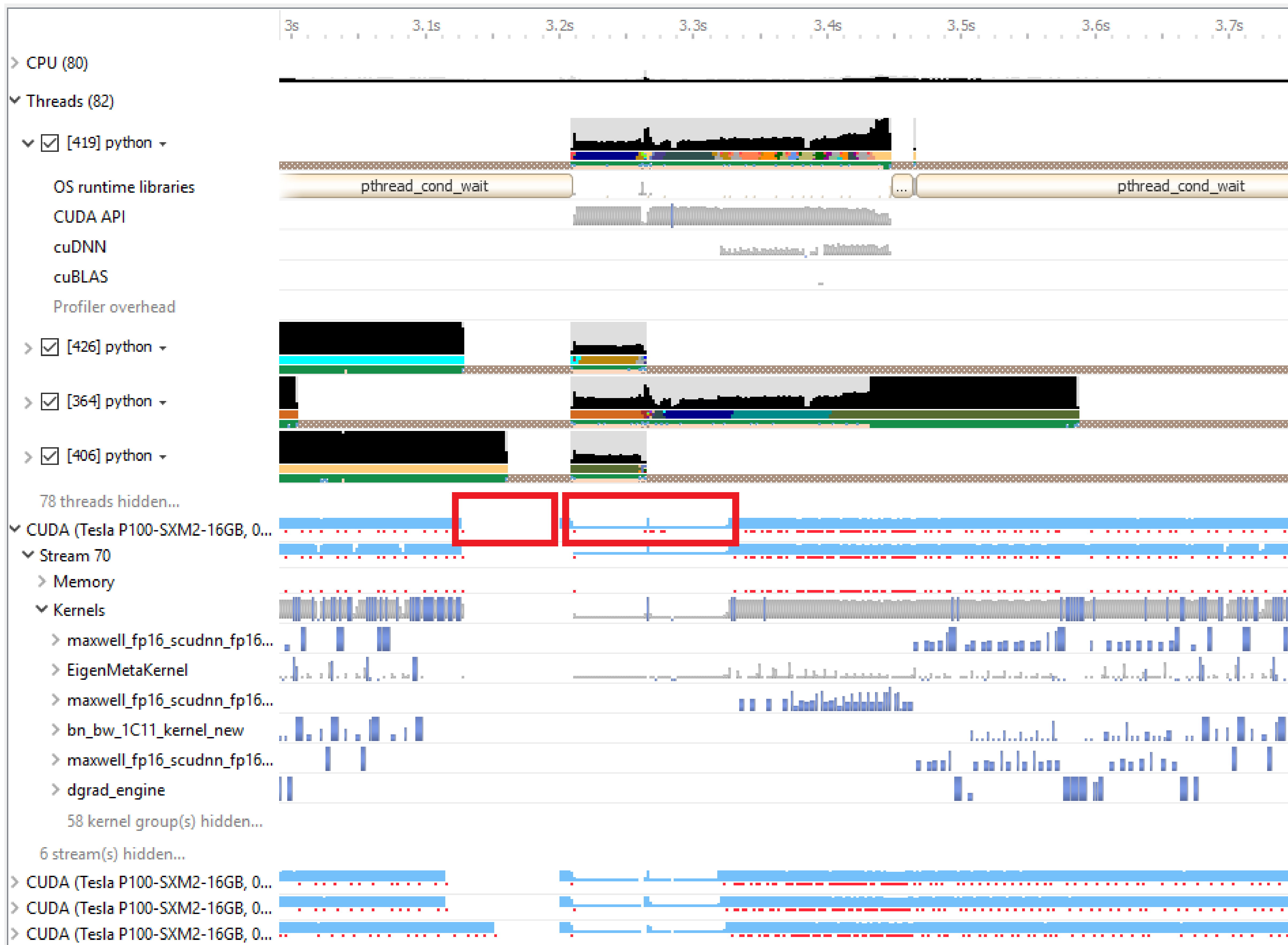
Nsight Compute

- Debug/optimize CUDA kernel

Nsight Graphics

- Debug/optimize graphics workloads





GPU idle and low utilization level of detail

NCHC N-WAY GPU Bootcamp

<https://www.youtube.com/watch?v=qqU0jKyMwzg>



Self-paced NVIDIA DLI

<https://developer.nvidia.com/join-nvidia-developer-program>

 **NVIDIA DEVELOPER**

[Home](#) [Blog](#) [Forums](#) [Docs](#) [Downloads](#) [Training](#)

 [Join](#) 

[Topics](#) [Platforms and Tools](#) [Industries](#) [Resources](#)

Choose a Technical Training Course. It's On Us.

Join the NVIDIA Developer Program and take one of the complimentary technical self-paced courses below (worth up to \$90).

[Claim Now](#)

Generative AI and LLMs

Graphics and Simulation

Accelerated Computing

Data Science

Deep Learning

8 hours

Fundamentals of Accelerated Computing With Python

Explore how to use Numba—the just-in-time, type-specializing Python function compiler—to create and launch CUDA® kernels to accelerate Python programs on massively parallel NVIDIA GPUs.

8 hours

Getting Started with Accelerated Computing in Modern CUDA C++

Learn how to accelerate and optimize existing C/C++ CPU-only applications to apply the power of GPUs using the most essential CUDA techniques and the NVIDIA Nsight™ Systems profiler.

2 hours

Optimizing CUDA Machine Learning Codes With Nsight Profiling Tools

Check out NVIDIA Developer Tools, a collection of applications that span desktop and mobile targets and enable developers to build, debug, profile, and develop cutting-edge software using the latest visual computing hardware from NVIDIA. In this course, you'll learn the effective use of two powerful NVIDIA developer tools: Nsight Systems and Nsight Compute.

67



Nsight Tutorial Sessions

<https://developer.nvidia.com/nsight-systems/get-started#tutorials>

Download

Latest version

Supported Platforms

System Requirements

Release Notes

Feature Table

Archives

Resources

Documentation

Self-Paced Training

Tutorial Sessions

Video Series

Support

Tutorial Sessions



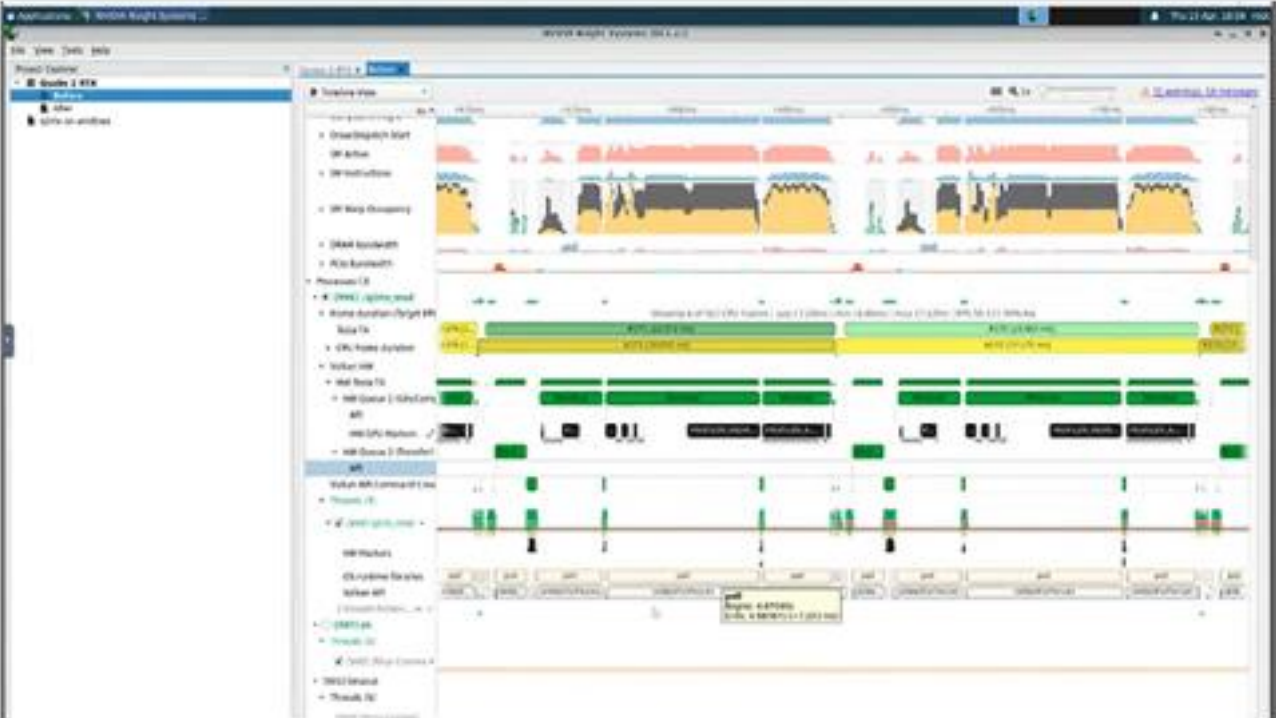
WHAT THE PROFILER IS TELLING YOU

Profiling Keras training, Sept 2020

Profiling GPU Applications with Nsight Systems

This webinar gives an overview of NVIDIA's Nsight profiling tools. It explores how to analyze and optimize the performance of GPU-accelerated applications.

Watch (54:52) >



Fundamentals of Ray Tracing Development using Nsight Graphics and Nsight Systems

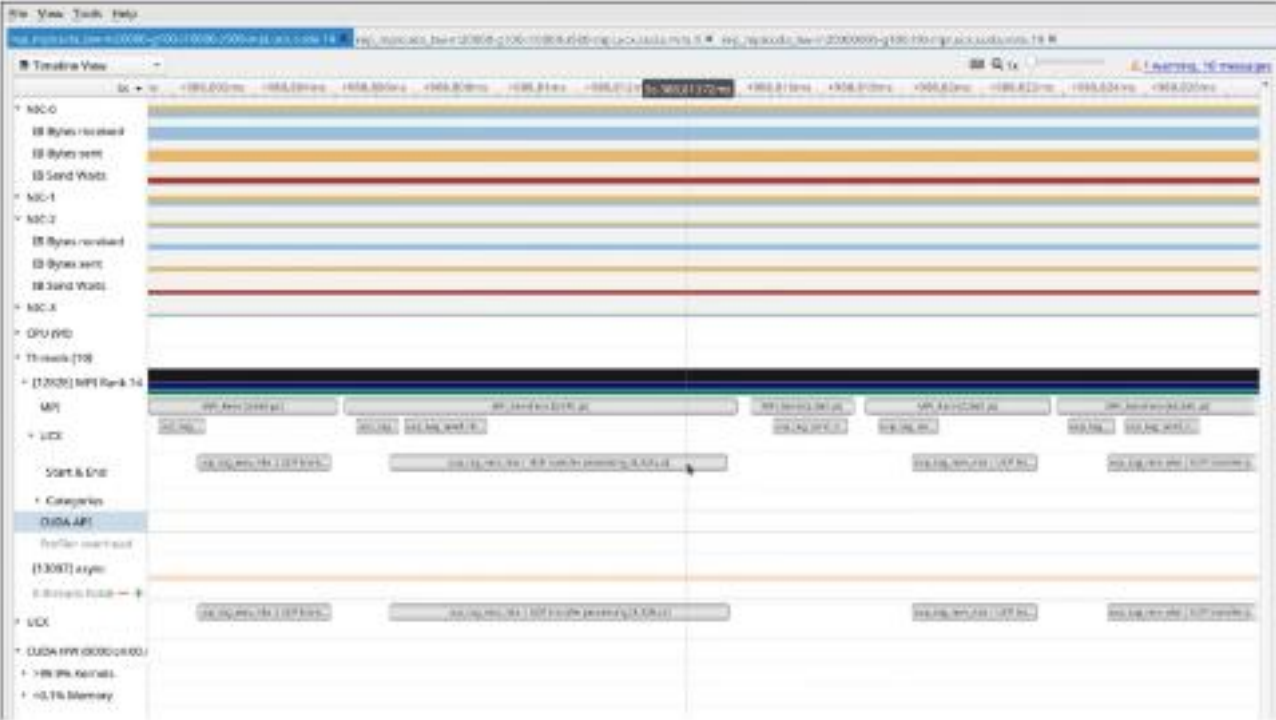
Learn how to utilize Nsight Graphics and Nsight Systems to profile and optimize 3D Applications that are using Ray Tracing.

Watch (2:04:45) >



Investigating Hidden Bottlenecks for Multi-Node Workloads

Learn how Nsight Systems can help users identify bottlenecks, investigate their causes,



Optimizing Communication with Nsight Systems Network Profiling

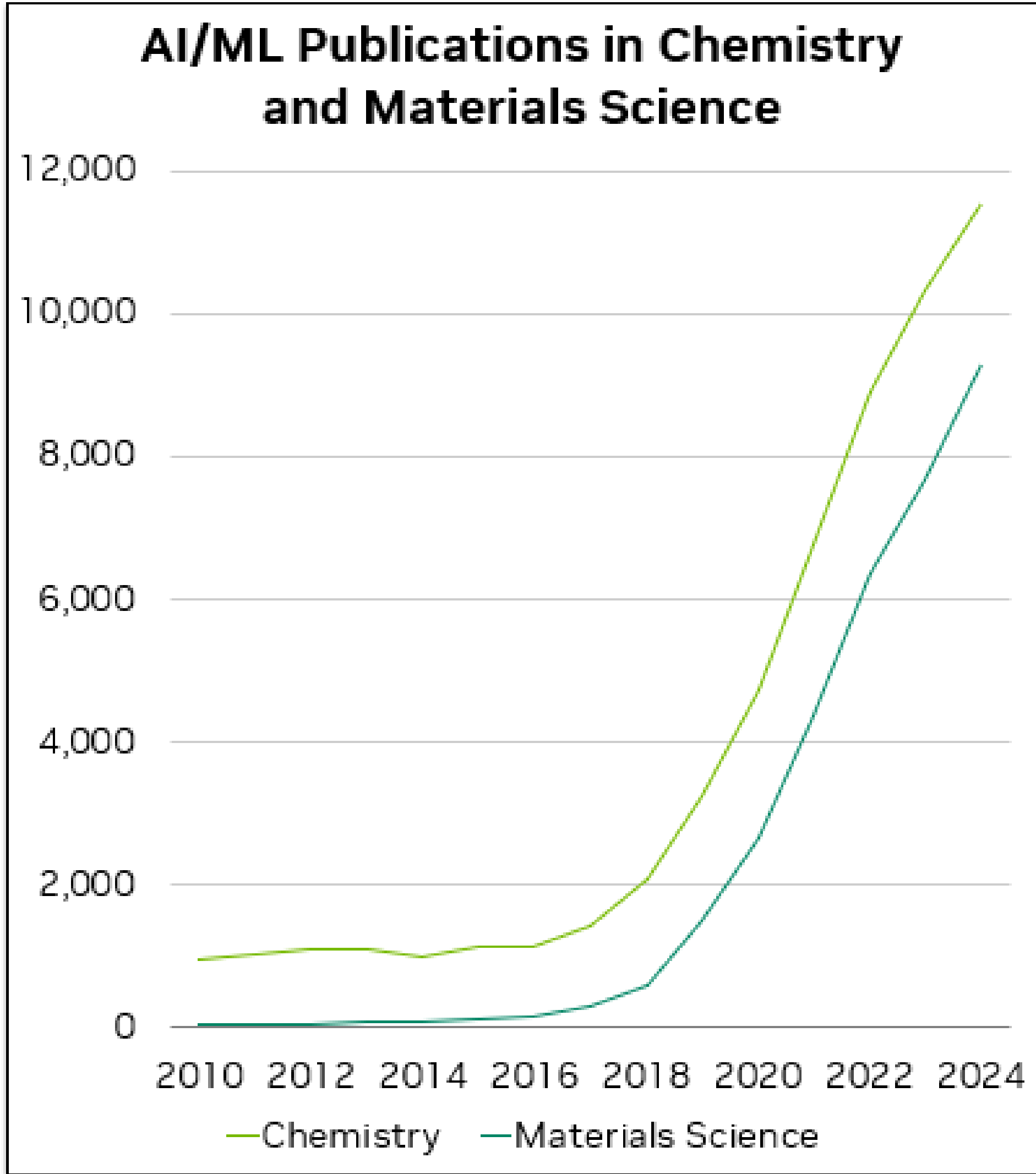
Learn how to use Nsight Systems' network profiling capabilities and see how real-world

68

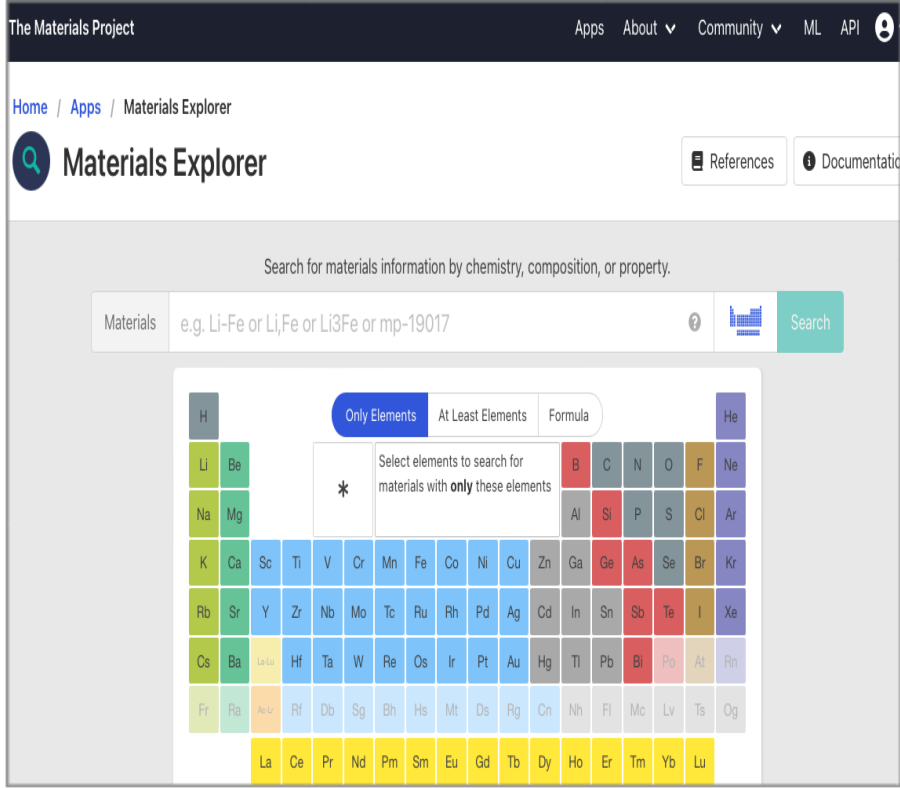


AI is transforming Chemistry and Materials Science Research

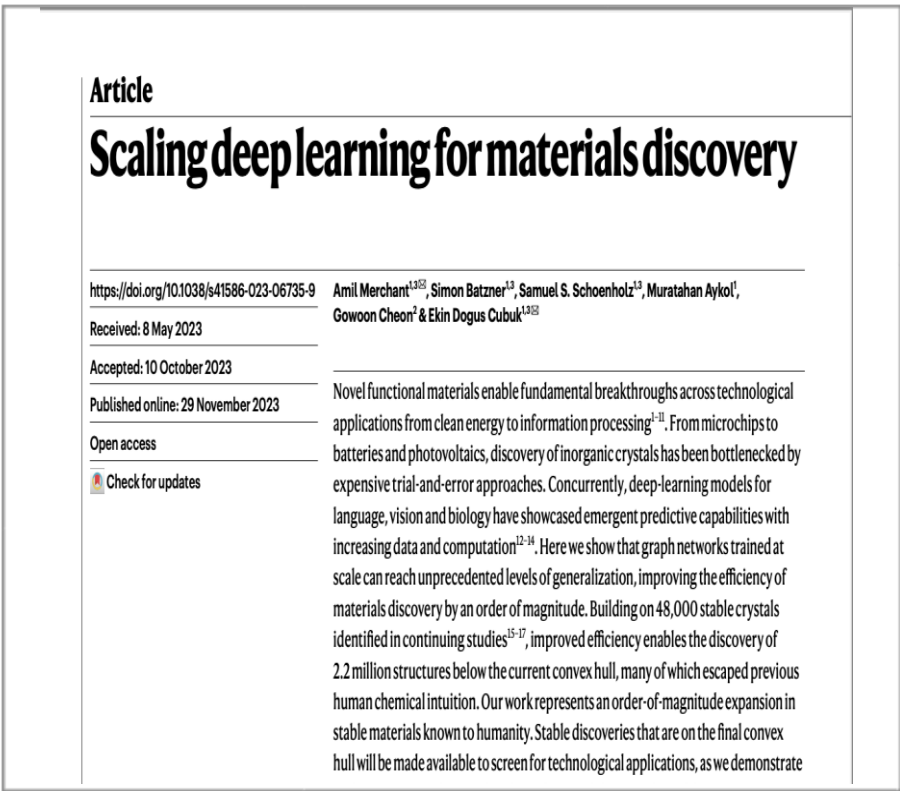
Exponential rise in AI-driven Chemistry and Materials Science research in Academia and Big Tech



Materials Project Multi-National Effort



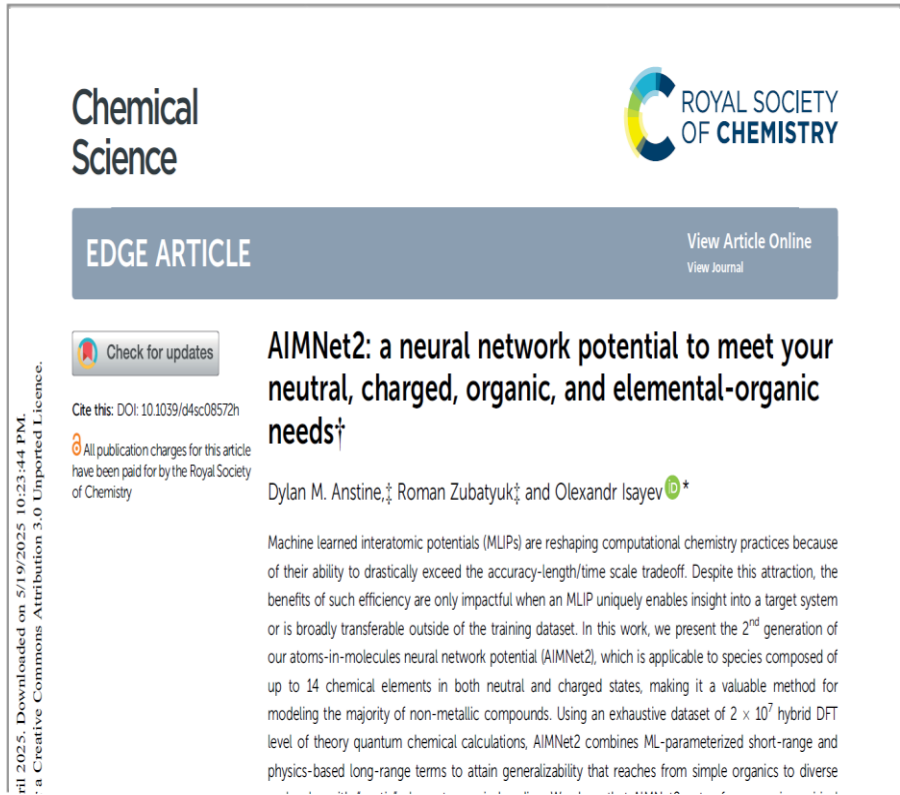
GNoME Google DeepMind



Academia

AIMNet2

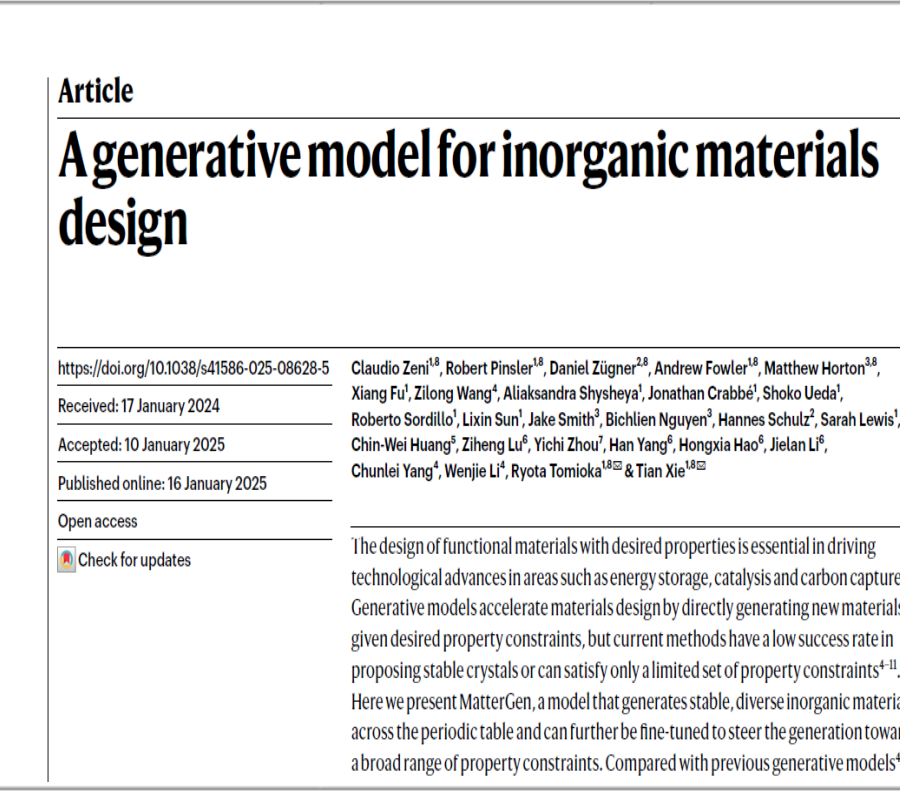
Carnegie Mellon University



Big Tech

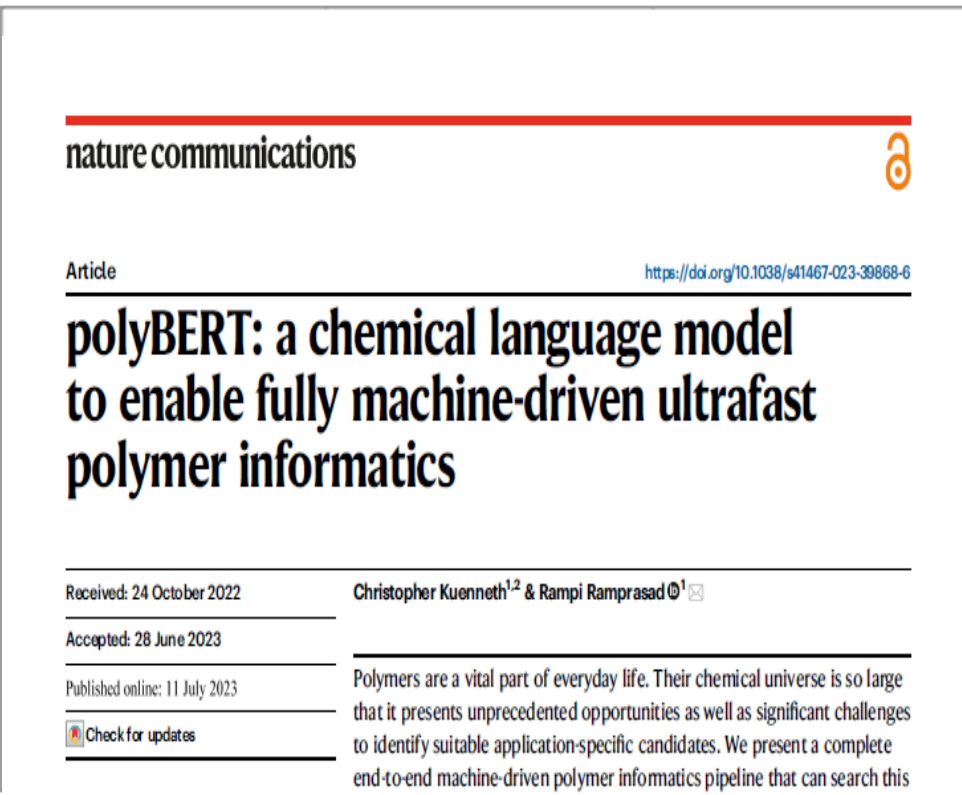
MatterGen

Microsoft Research

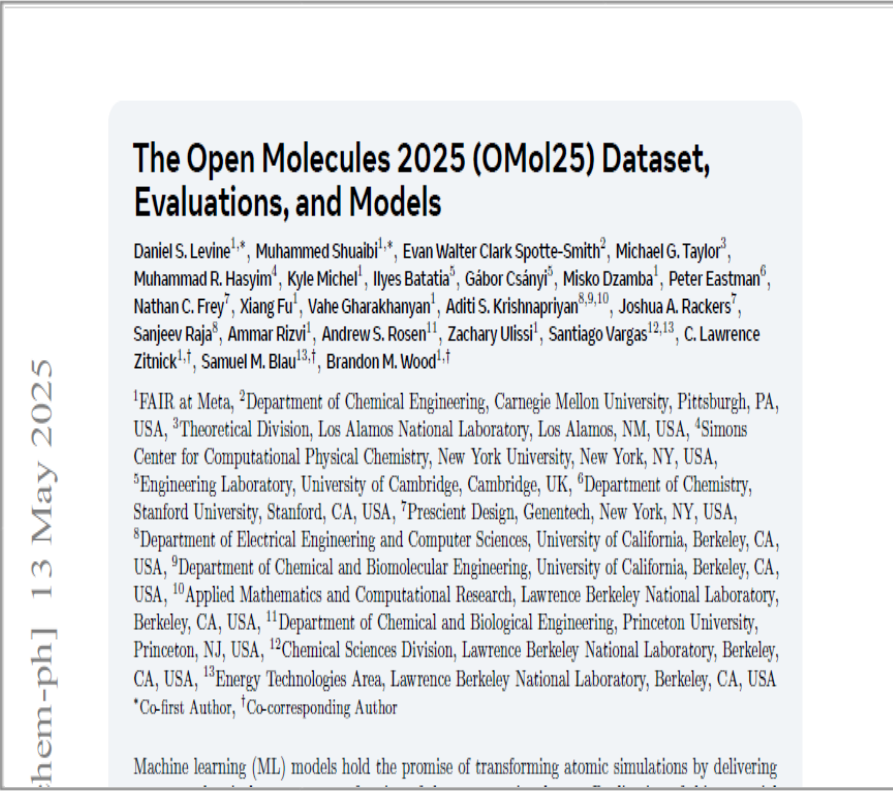


PolyBERT

Georgia Institute of Technology



Open Molecules 2025 Meta FAIR



MLIP Models

(Machine Learning
Interatomic Potential)

Generative Models

(Cheminformatics)

Datasets

(from simulations and
experiments)

Agentic AI Workflows

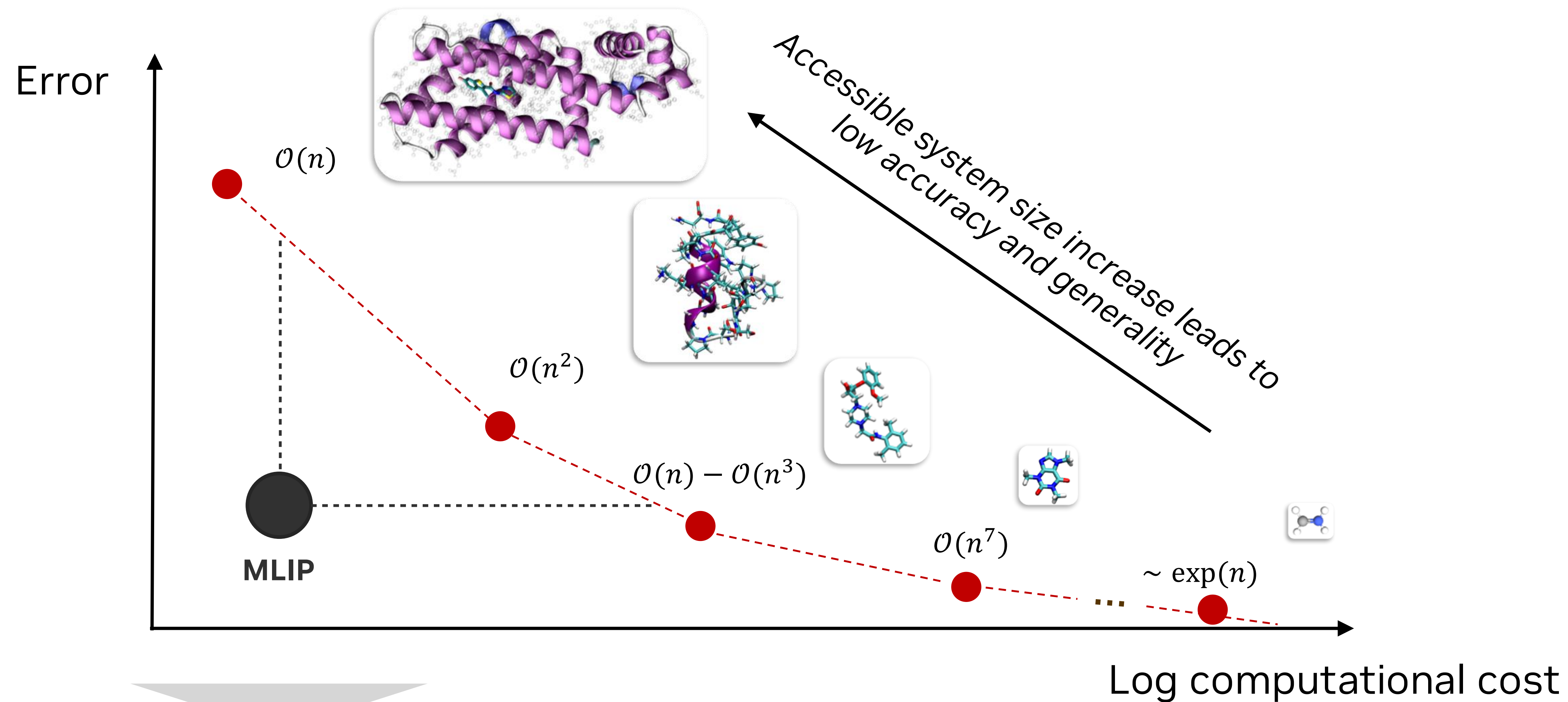
Lab-in-the-Loop

Major advancement of AI models and datasets in last 2 years

Source : “Blaiszik/ml_publication_charts: AI/ML Publication Statistics for 2024”. Zenodo, March 22, 2025

Machine Learning Interatomic Potentials (MLIP)

AI surrogate models approaching quantum chemistry accuracy but orders of magnitude lower cost



- Each computational method comes with accuracy vs. cost trade-off – improving accuracy (lower error) increases cost exponentially
- Simulation of large complex systems typically requires compromising on accuracy and generality (e.g., using classical force fields)
- Machine learning interatomic potentials (MLIPs) enable simulations to approach quantum chemistry accuracy at near force field cost
- Geometry relaxation and molecular dynamics (MD) are common simulation use cases

Approach quantum chemistry accuracy...

Time-independent Schrödinger equation

$$\hat{H}|\psi\rangle = E|\psi\rangle$$

$$\hat{H} = KE + ER + EN + NN$$

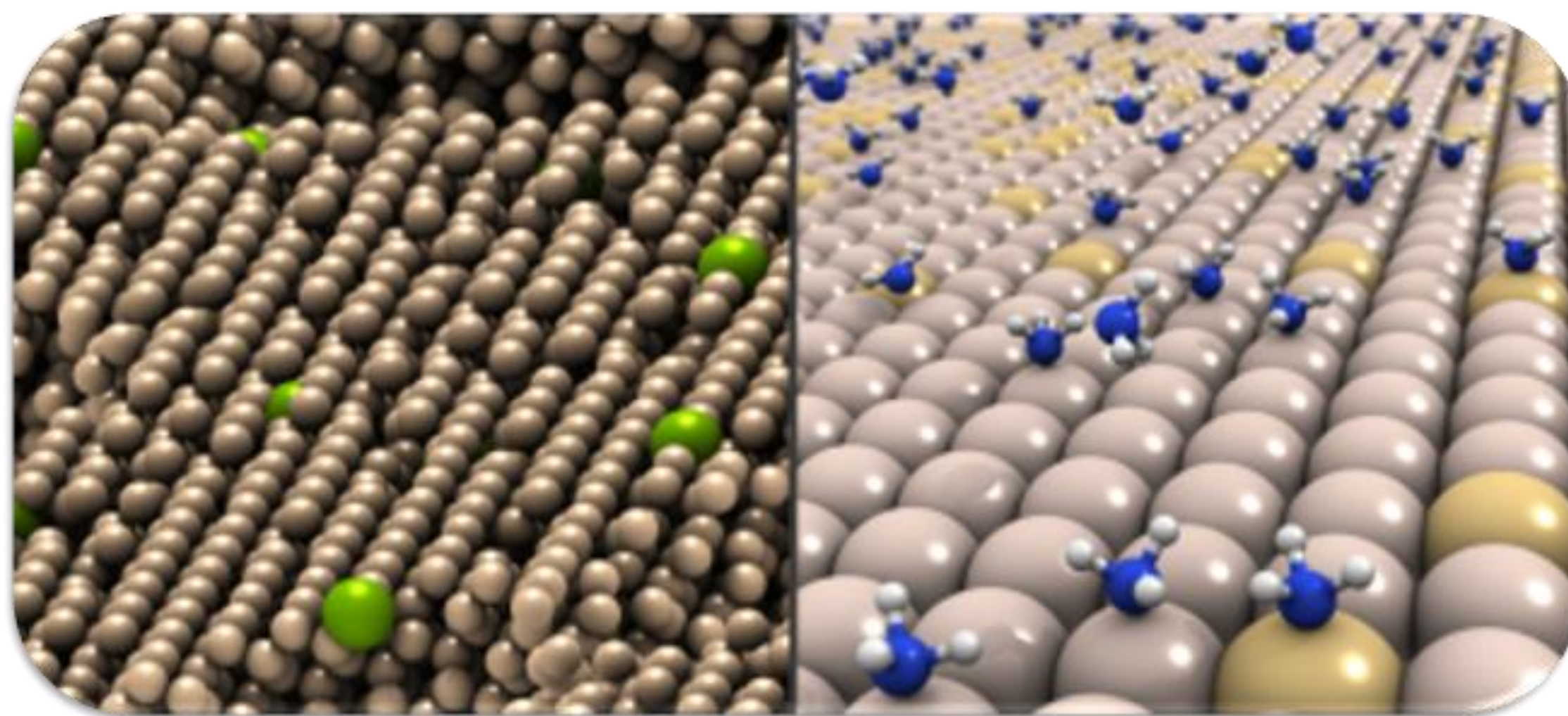
KE: Electronic Kinetic Energy

... at near classical force fields cost

$$E_{FF} = \sum E_{ij}^{nonbonded} + \sum E_{ijkl}^{Torsions} + \sum E_{ijk}^{Angles} + \sum E_{ij}^{Bonds}$$

NVIDIA ALCHEMI

By accelerating AI-enabled atomistic simulation at the machine learning framework level,
See more in <https://developer.nvidia.com/cuda/cuda-x-libraries/alchemi>.



Catalysts for Hydrogen Production

10x acceleration of discovery timelines



Molecules for OLEDs

10,000x acceleration vs. simulation methods



Electrolyte Materials for Li Batteries

800x acceleration in MLIP calculations



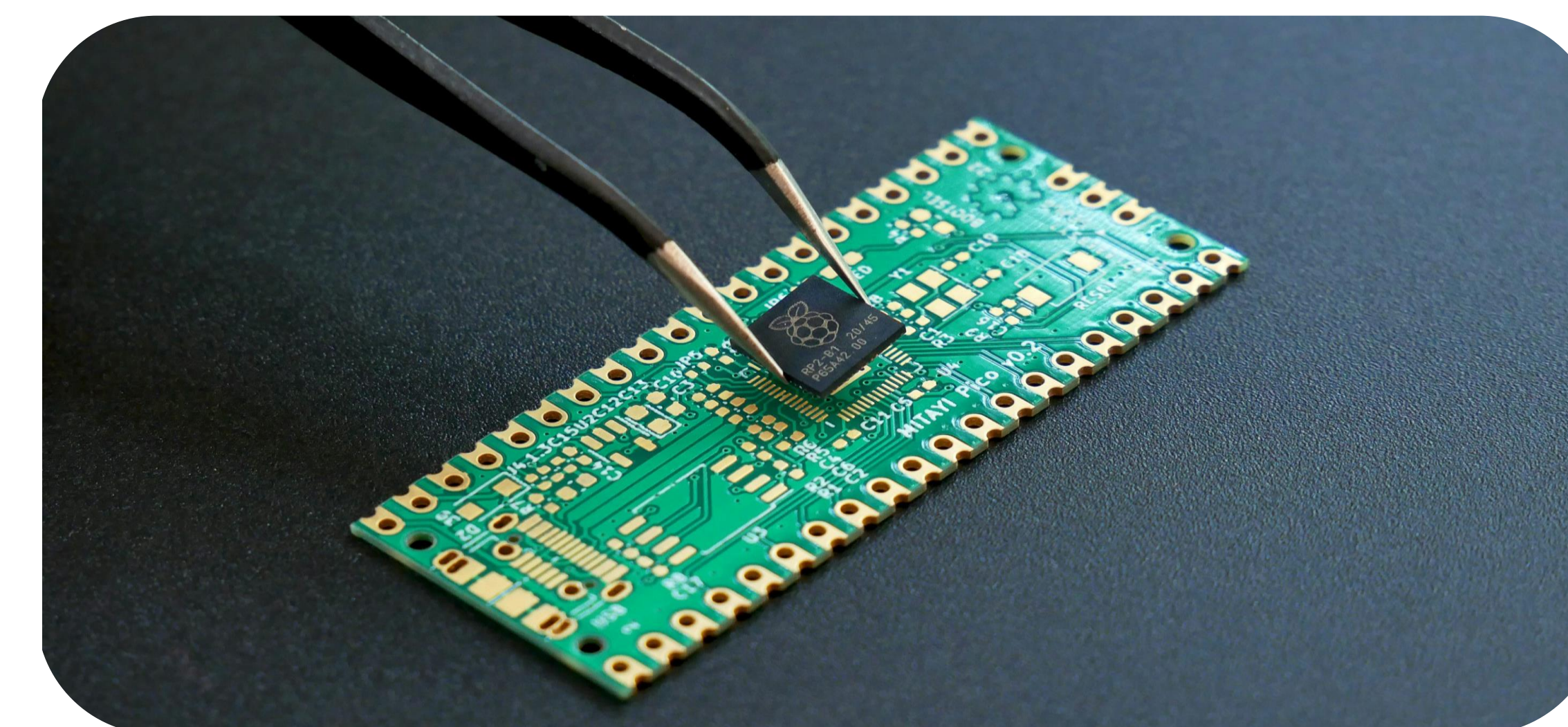
Co-Solvents for Skincare Formulations

100x faster screening for sun protection



Non-Rare-Earth Permanent Magnets

Up to 50x acceleration in magnetic materials screening



Deposition in semiconductor manufacturing

10-20X acceleration observed over popular OSS MD package

AI-Physics



Saturating Performance in Traditional HPC

Simulations are getting larger and more complex



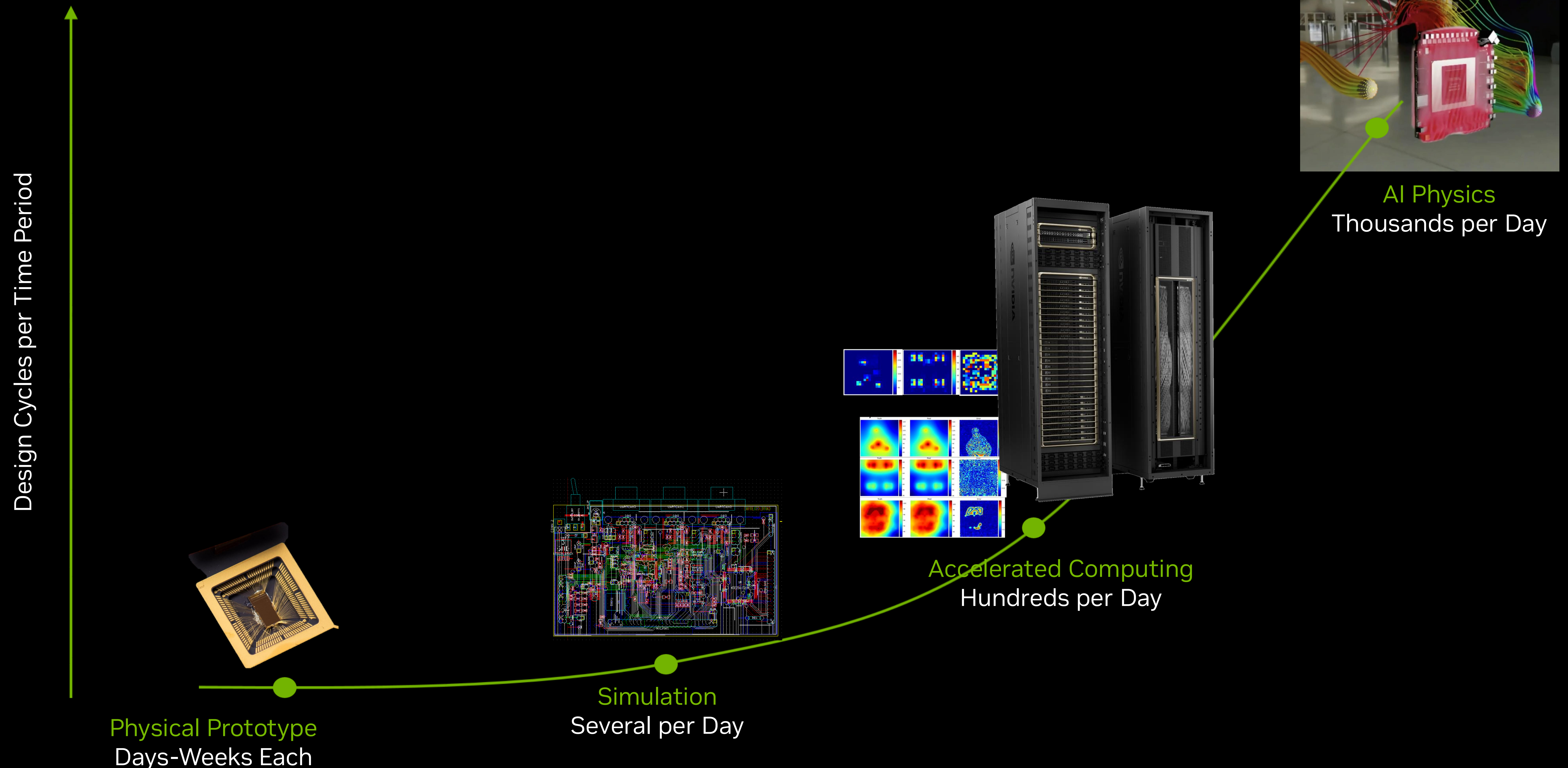
Traditional simulation methods are:

- Computationally Expensive
- Limited Parallelization gains
- Not suitable for Inverse problems
- Extremely Sensitive to Mesh Quality
- Inefficient for Real-Time Applications

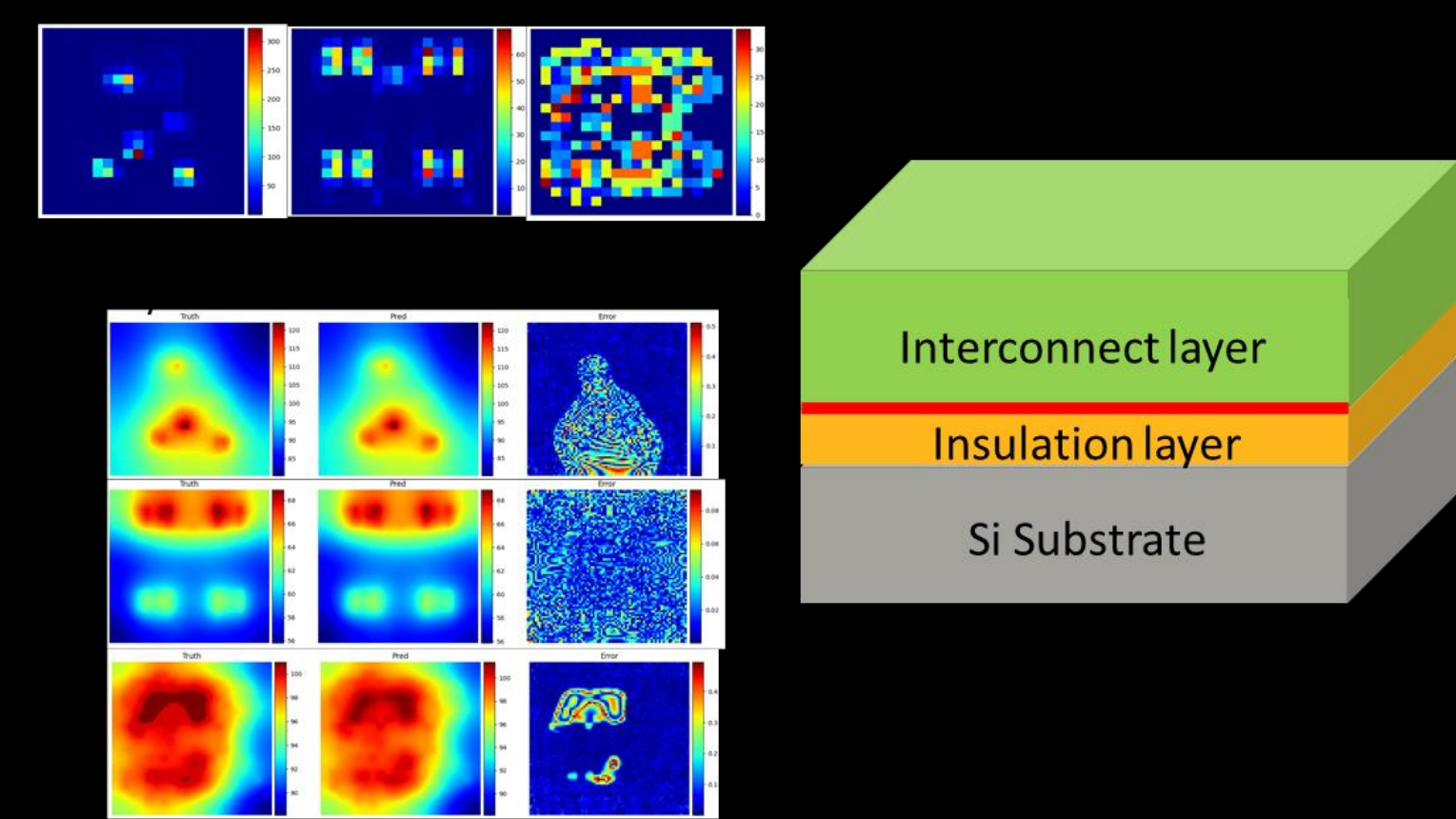
AI-driven Acceleration can:

- Computationally efficient
- Scalable performance
- Naturally suited for inverse problems
- Eliminate the need for mesh
- Capable of real-time processing

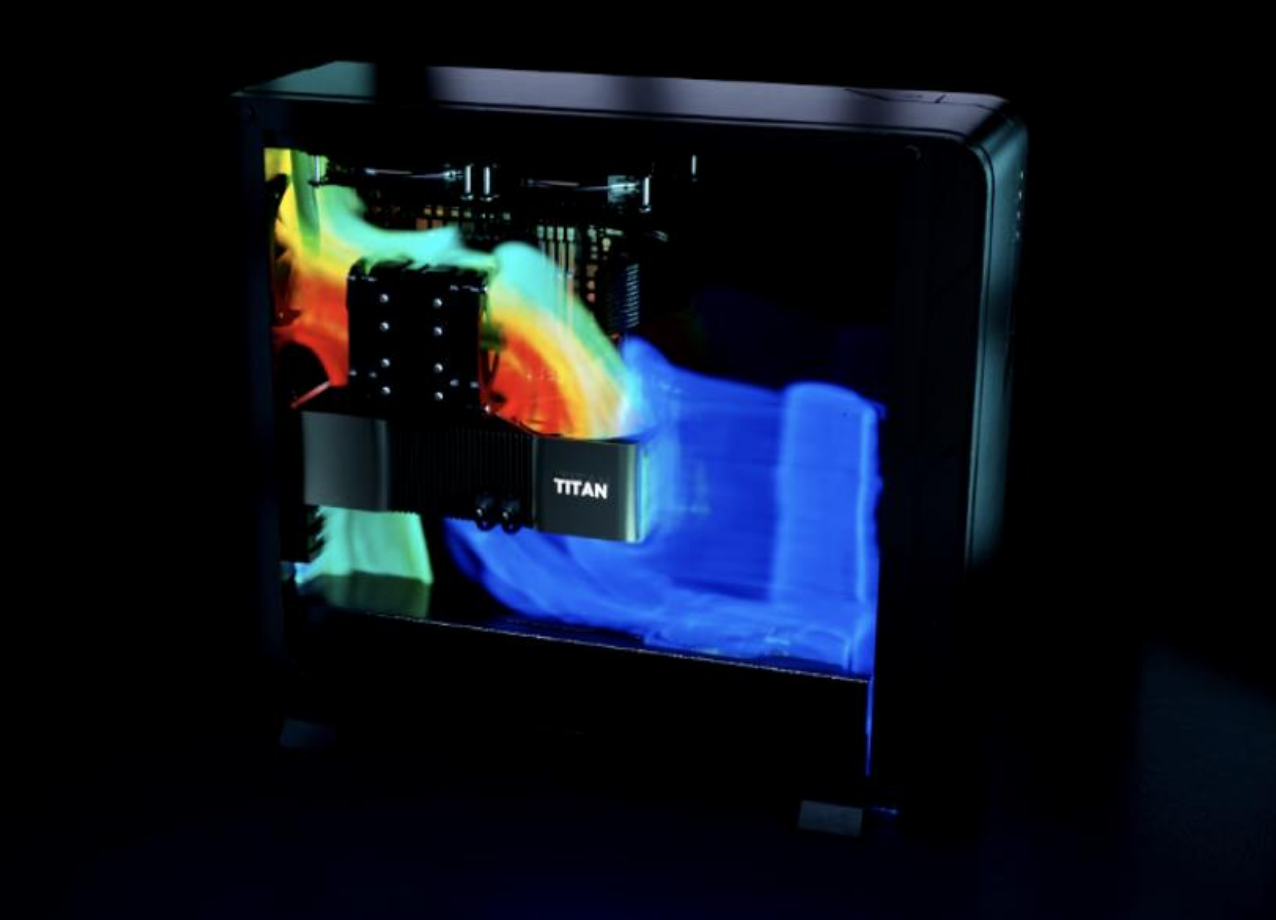
Accelerated Computing and AI Physics Revolutionizing Engineering Design



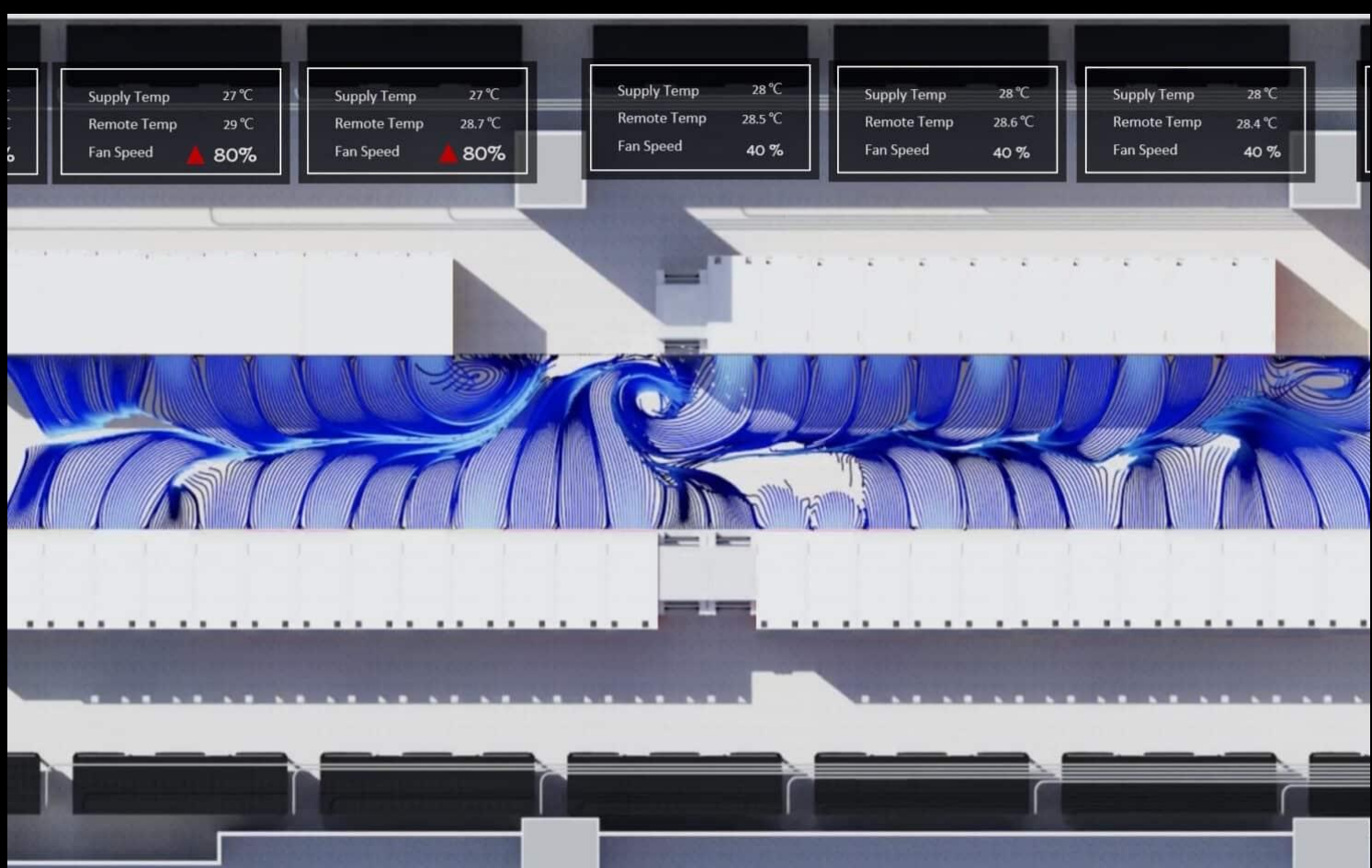
Transforming Industrial Simulations with Physics-AI



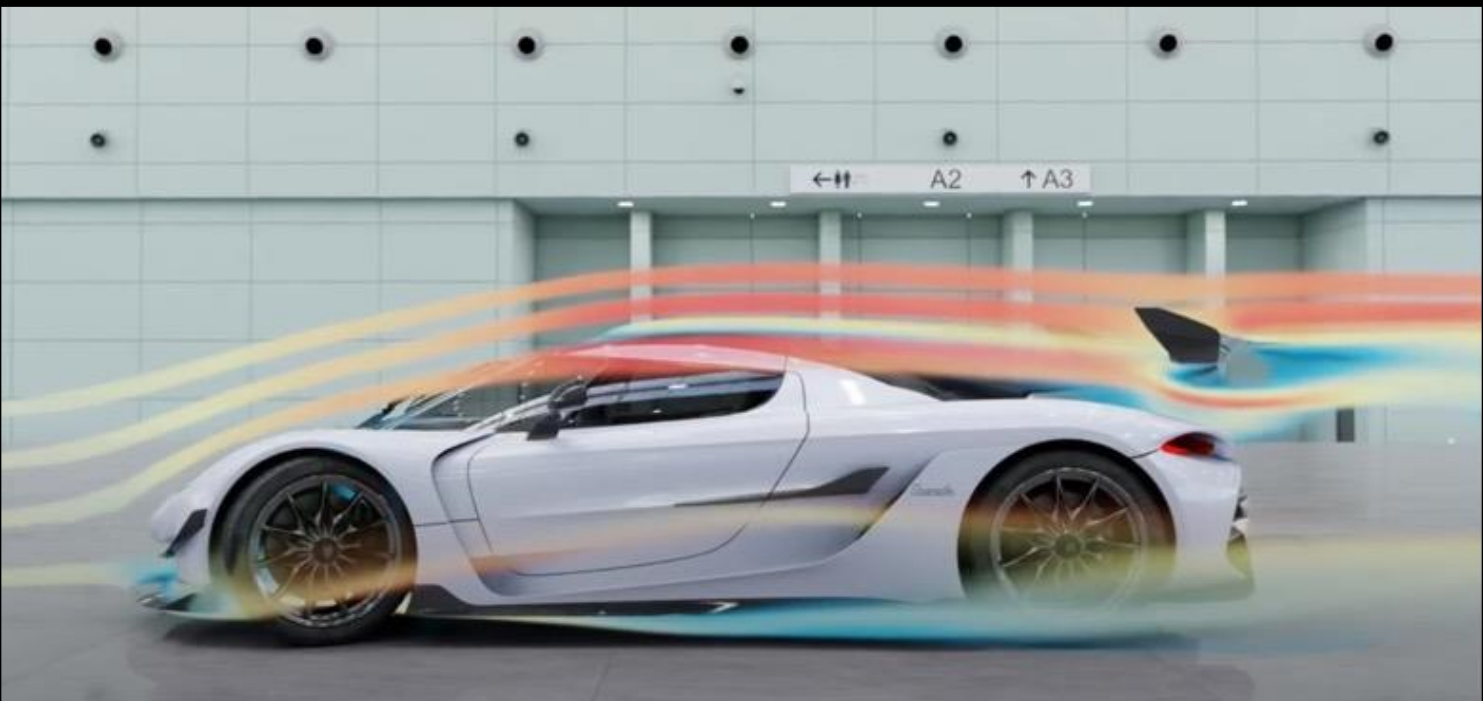
Electro-thermal cooling
Blog: [Link](#)



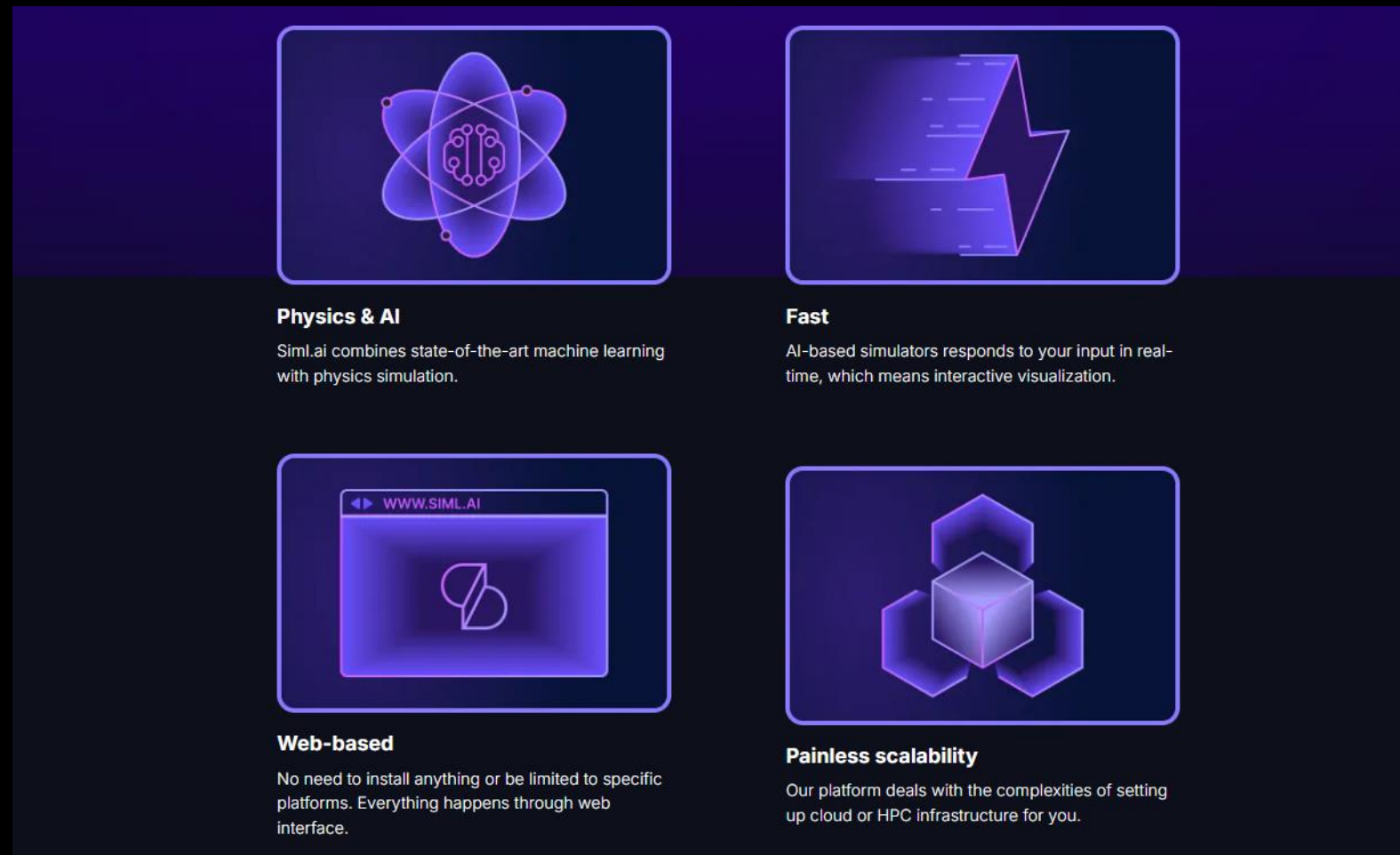
RTX 4090 heat sink design
Demo: [Link](#)



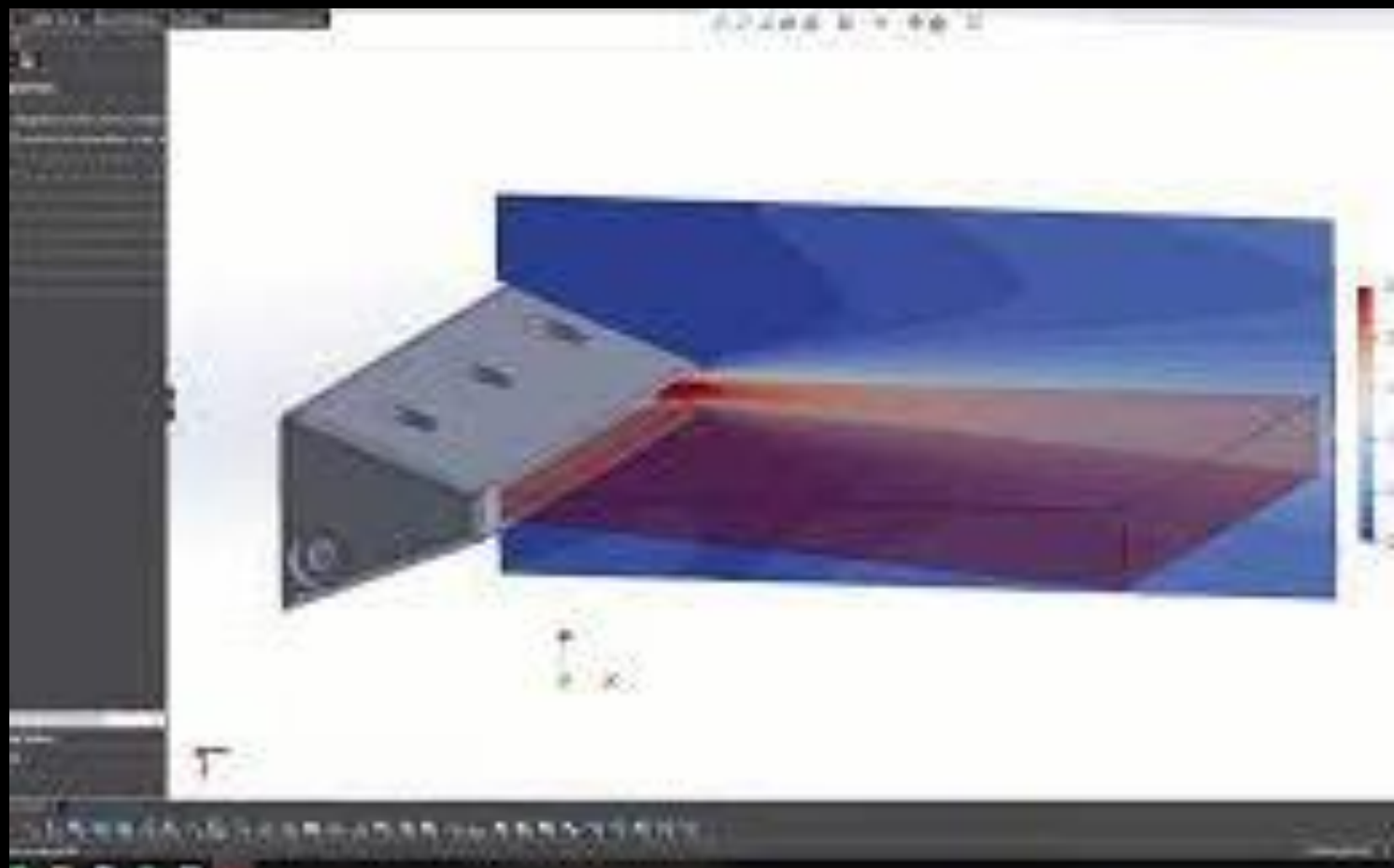
Data Center Digital Twin
Blog: [Link](#), GTC Session: [Link](#)



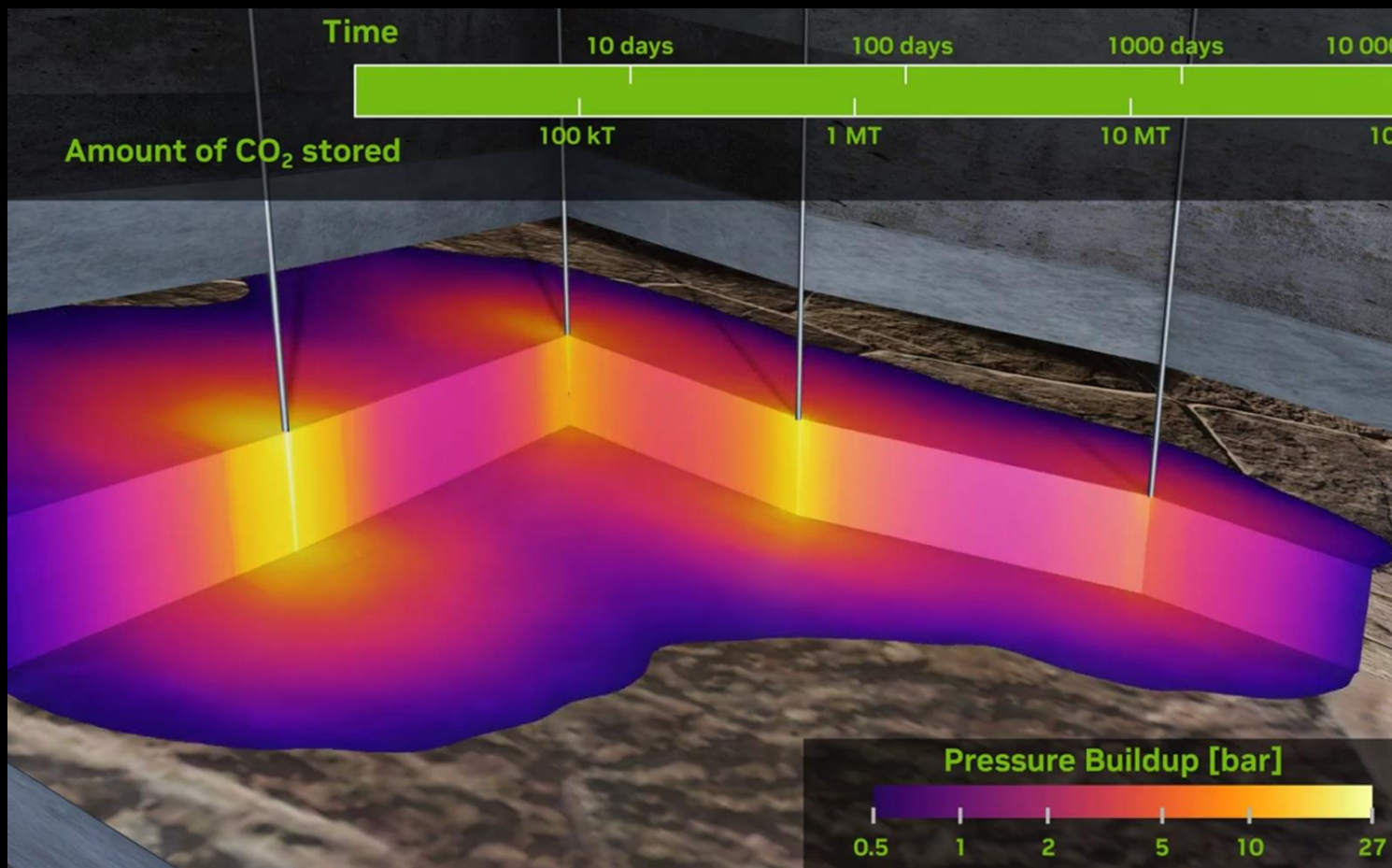
Automotive CFD
GTC 2024 Keynote



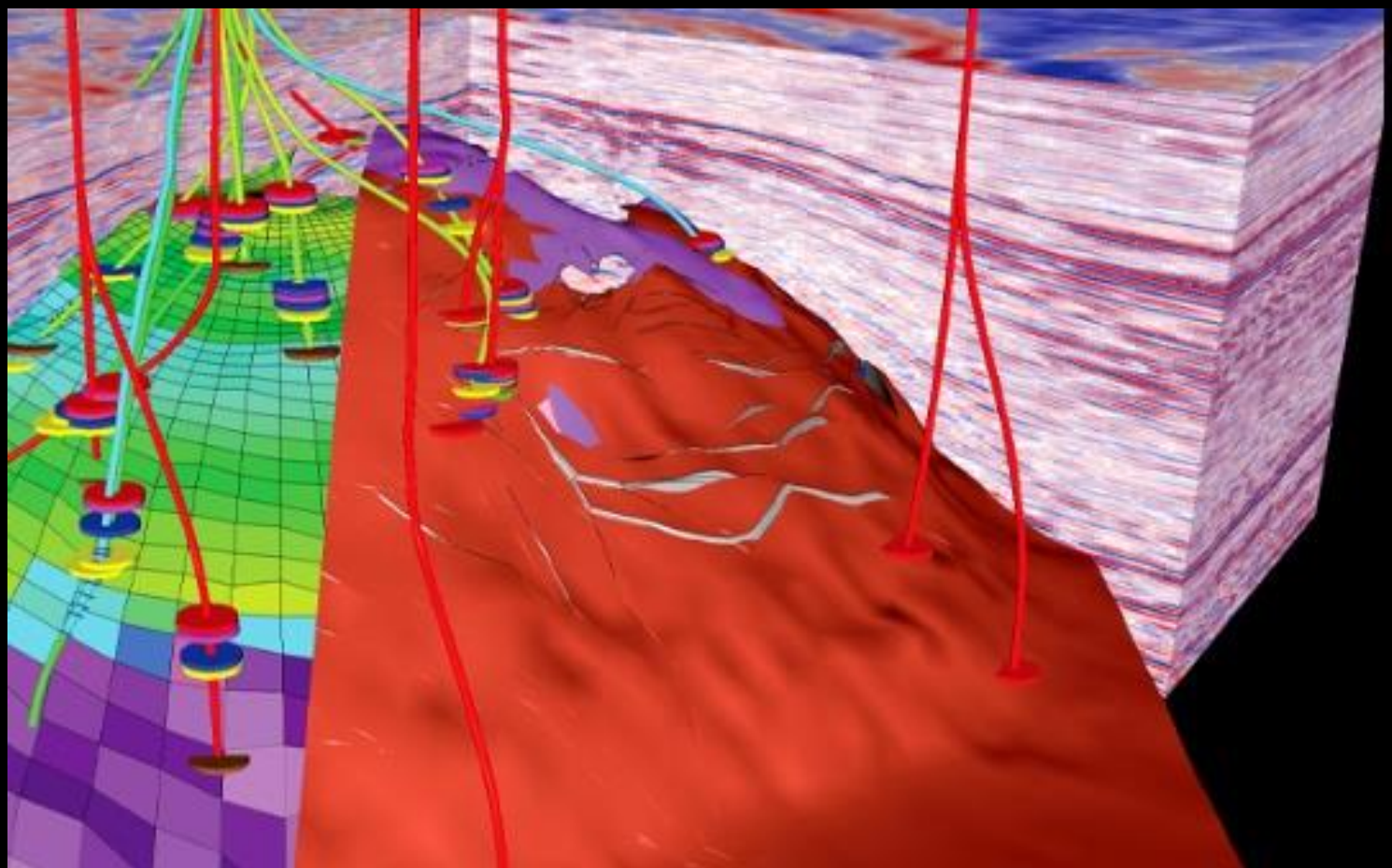
Siml.ai
Case Study: [Link](#), Blog: [Link](#)



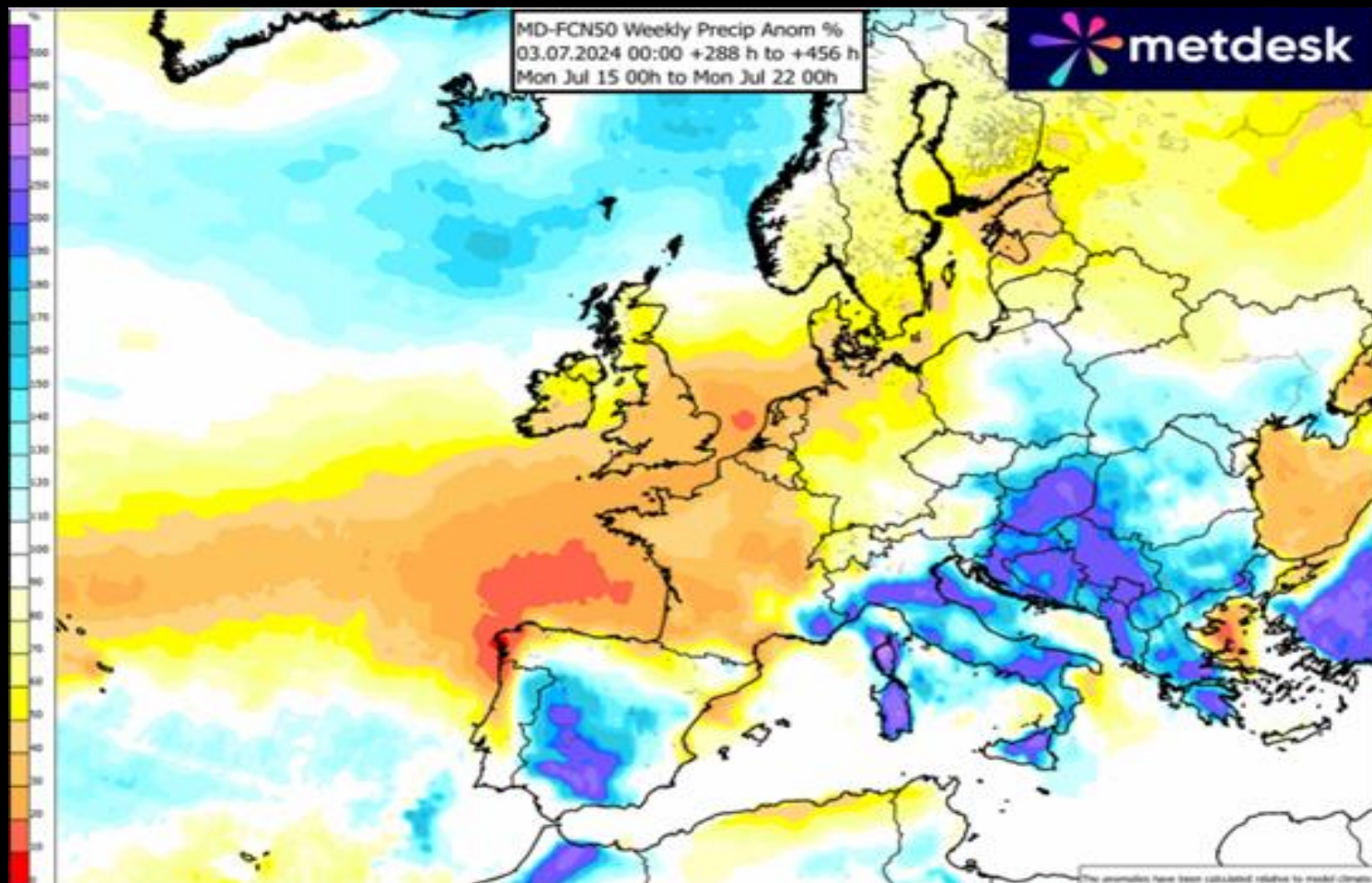
Design optimization



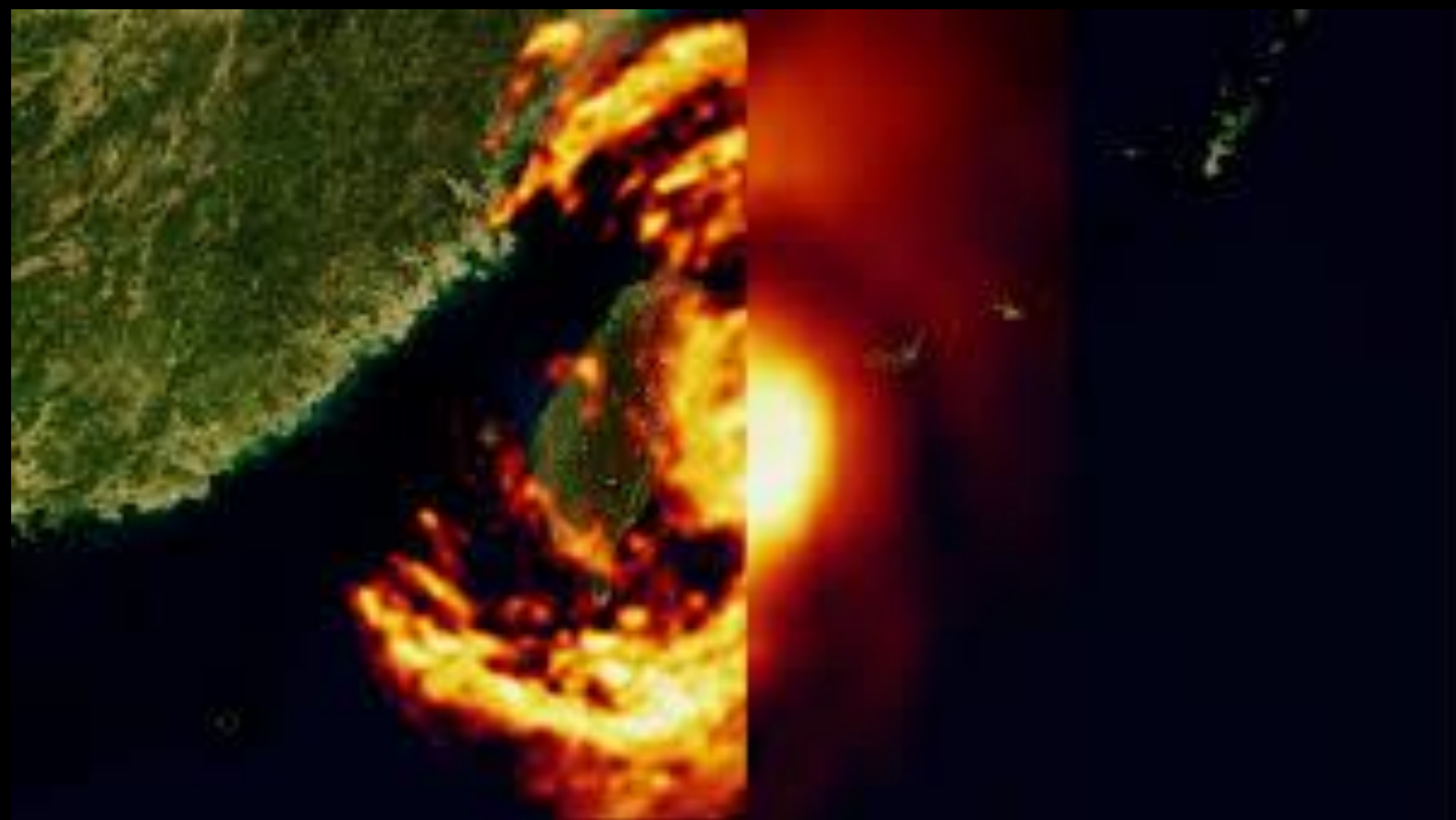
Carbon capture and storage
Demo: [Link](#), Blog: [Link](#)



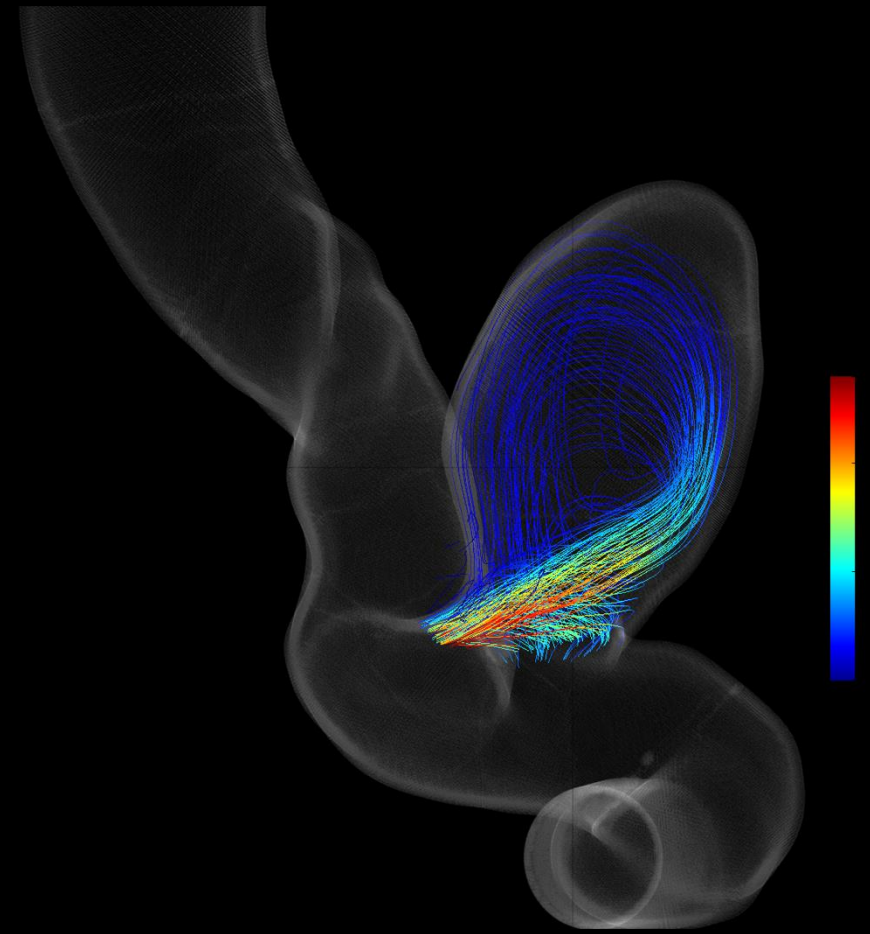
Sub surface simulations
Resource: [Link](#)



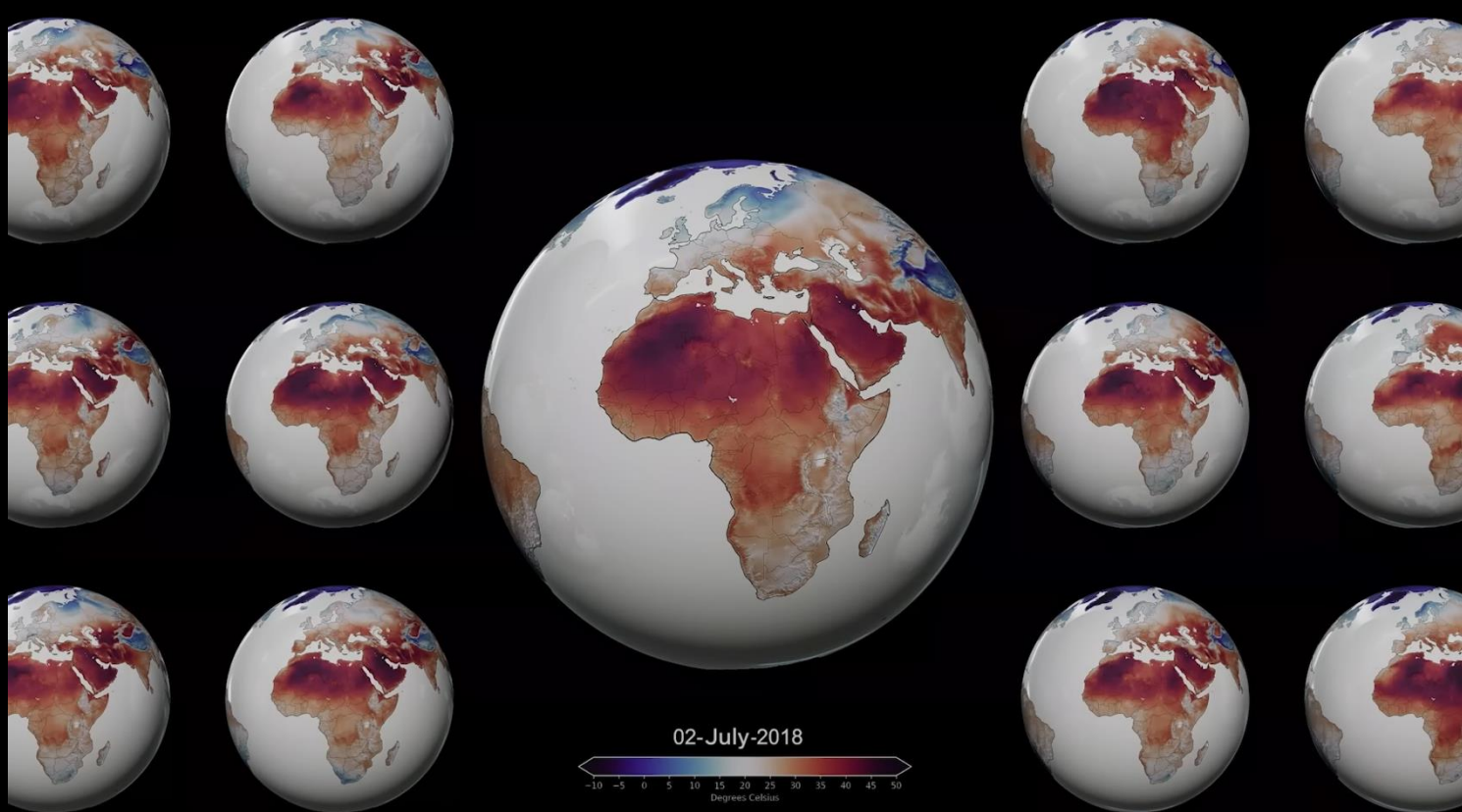
Energy Trading
Blog: [Link](#)



TWC: Weather modeling
Blog: [Link](#)

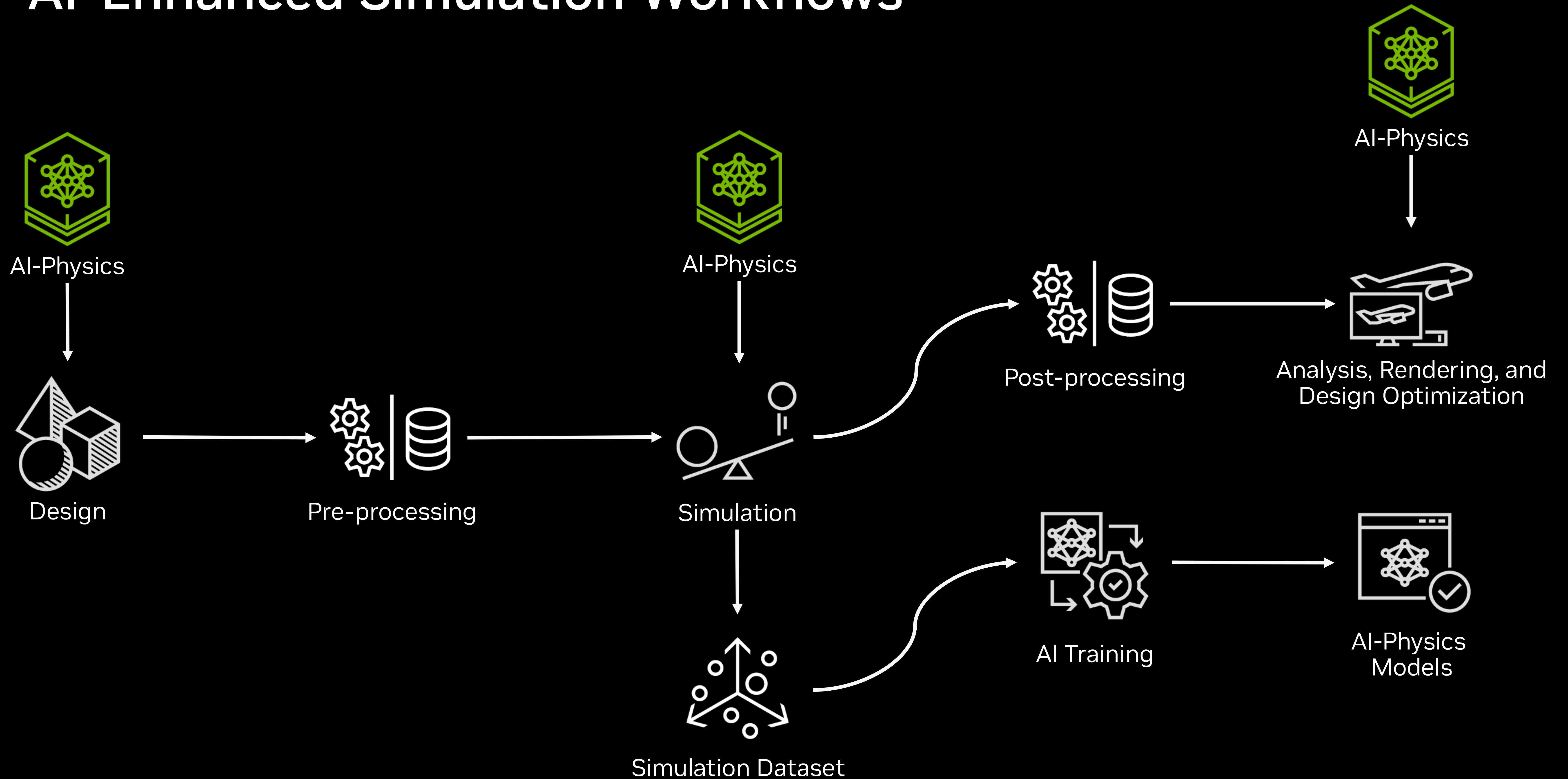


Brain Aneurysm Simulation
Demo: [Link](#)



Meteomatics: Weather modeling
Blog: [Link](#)

AI-Enhanced Simulation Workflows

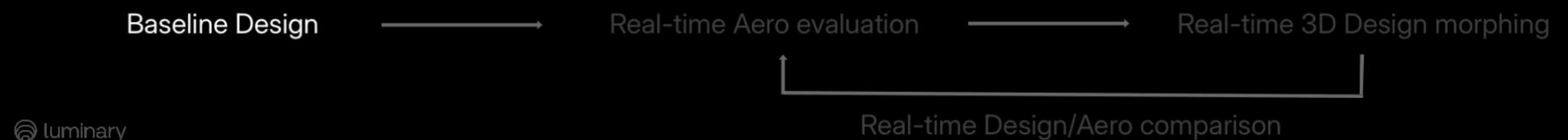
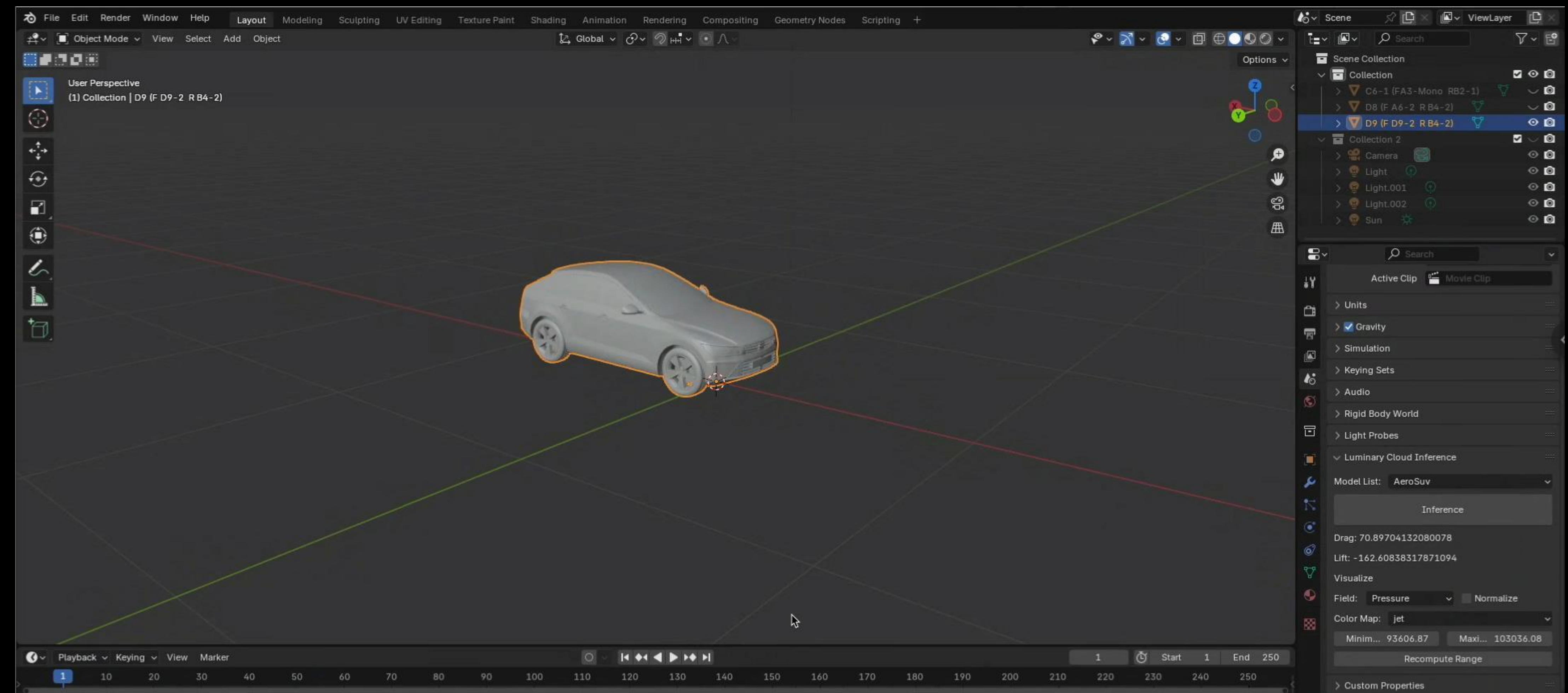


Luminary Cloud

Physics AI CFD solution using PhysicsNeMo in 2 months

Luminary uses PhysicsNeMo and the DoMINO architecture for their Physics AI solution

Luminary in collaboration with nTop were able to develop a rapidly iterative design optimization workflow for their customers.



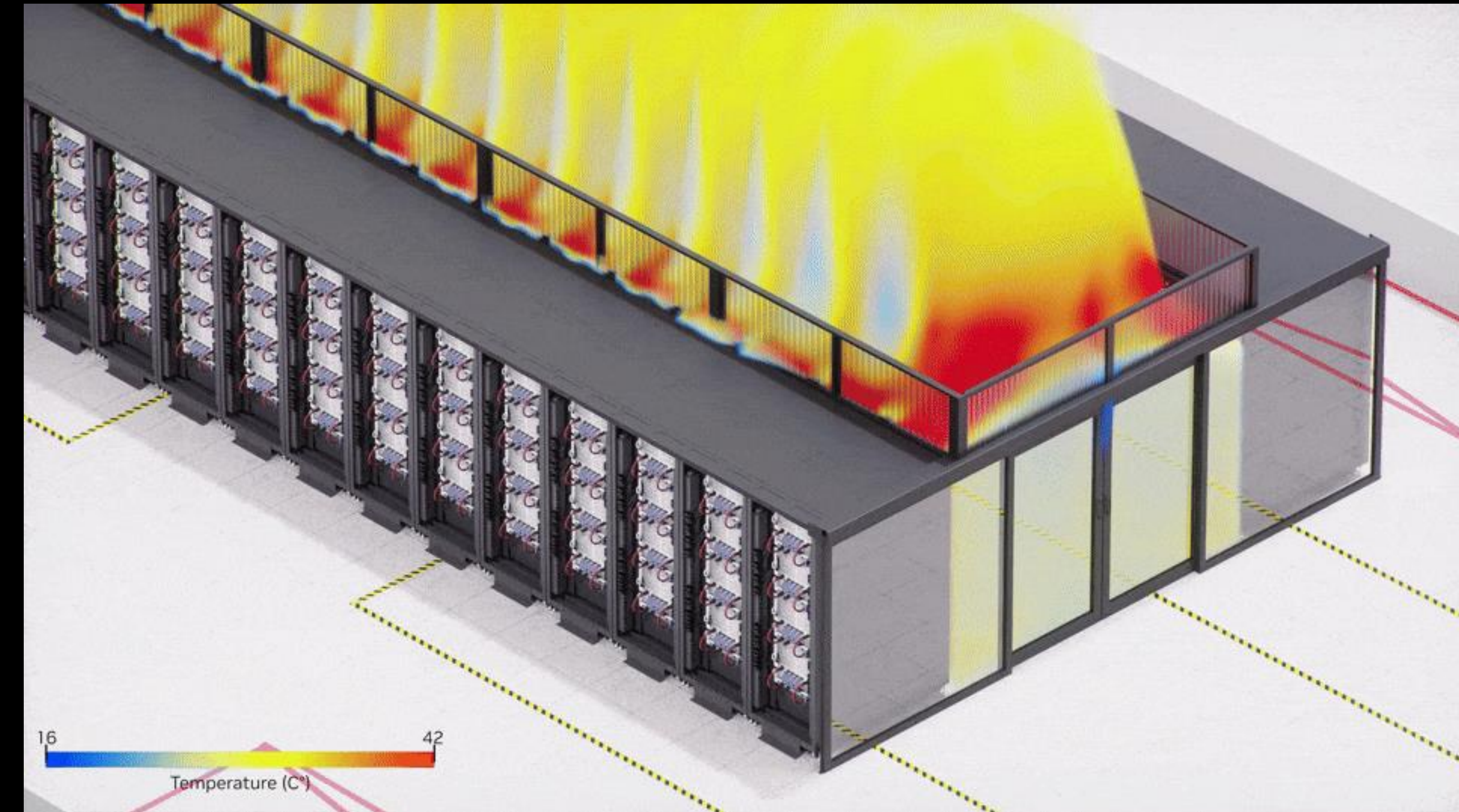
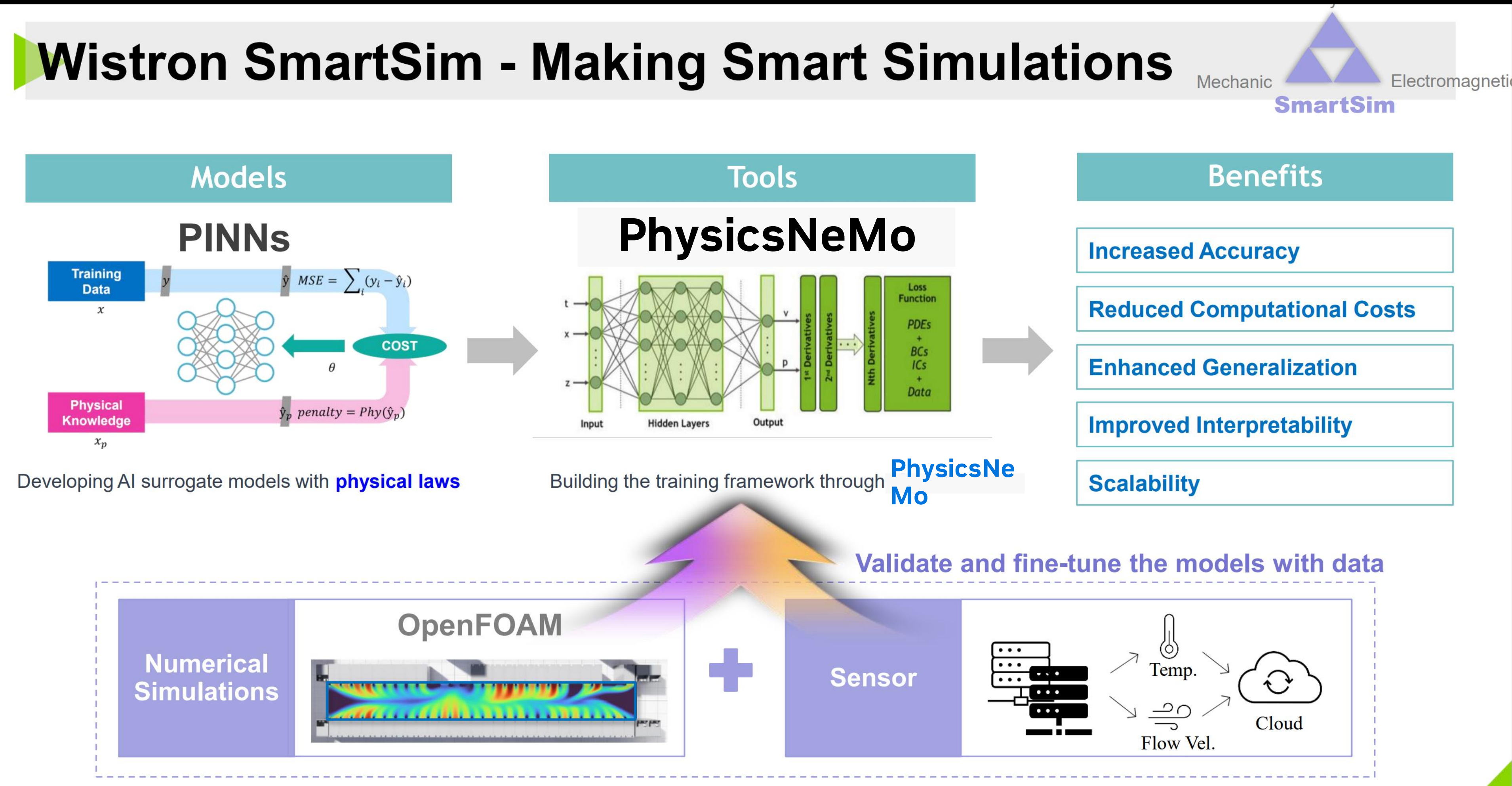
<https://www.luminarycloud.com/blog/introducing-luminary-shift-models-a-suite-of-physics-ai-foundation-models-to-transform-engineering-design/>

Wistron

Physics AI CFD solution

NVIDIA PhysicsNeMo used to develop Physics-informed AI for near-real-time thermal simulations in a run-in test room.

Wistron SmartSim - Making Smart Simulations



References: [Wistron PR](#), [Blog](#), [Example code](#)

HiLiftAeroML: From High-Fidelity CFD to Real-Time AI Surrogate

1,800 WMLES cases · 180 geometries × 10 angles of attack · NASA high-lift CRM · 300–500M cells · CC-BY-4.0

01 DATA GENERATION

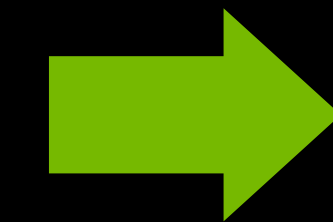
460,000 GB200 hrs

8hrs per case · 32× GB200

1,800 cases generated

GPU wall-modeled LES · 300–500M cells using Cadence CharLES

FP32 high-quality data



02 TRAINING

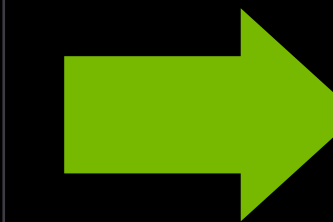
3280 GB200 hours

on 32× GB200

1,440 train + validation cases

GeoTransolver geometry → flow surrogate

FP16 Trained once, reused for every query



03 INFERENCE

~1 sec on GB200

per case · 4× GB200

any new geometry / angle of attack

Real-time aerodynamic prediction

FP16 – 10,000's of inference

ACCURACY · Full split — random 70/10/20, held-out test · GeoTransolver baseline

FIELD ERROR · RELATIVE L2 (LOWER IS BETTER)

8.8%

Surface pressure (Cp)

11.1%

Wall shear stress (Cf)

8.8%

Volume pressure

10.8%

Volume velocity

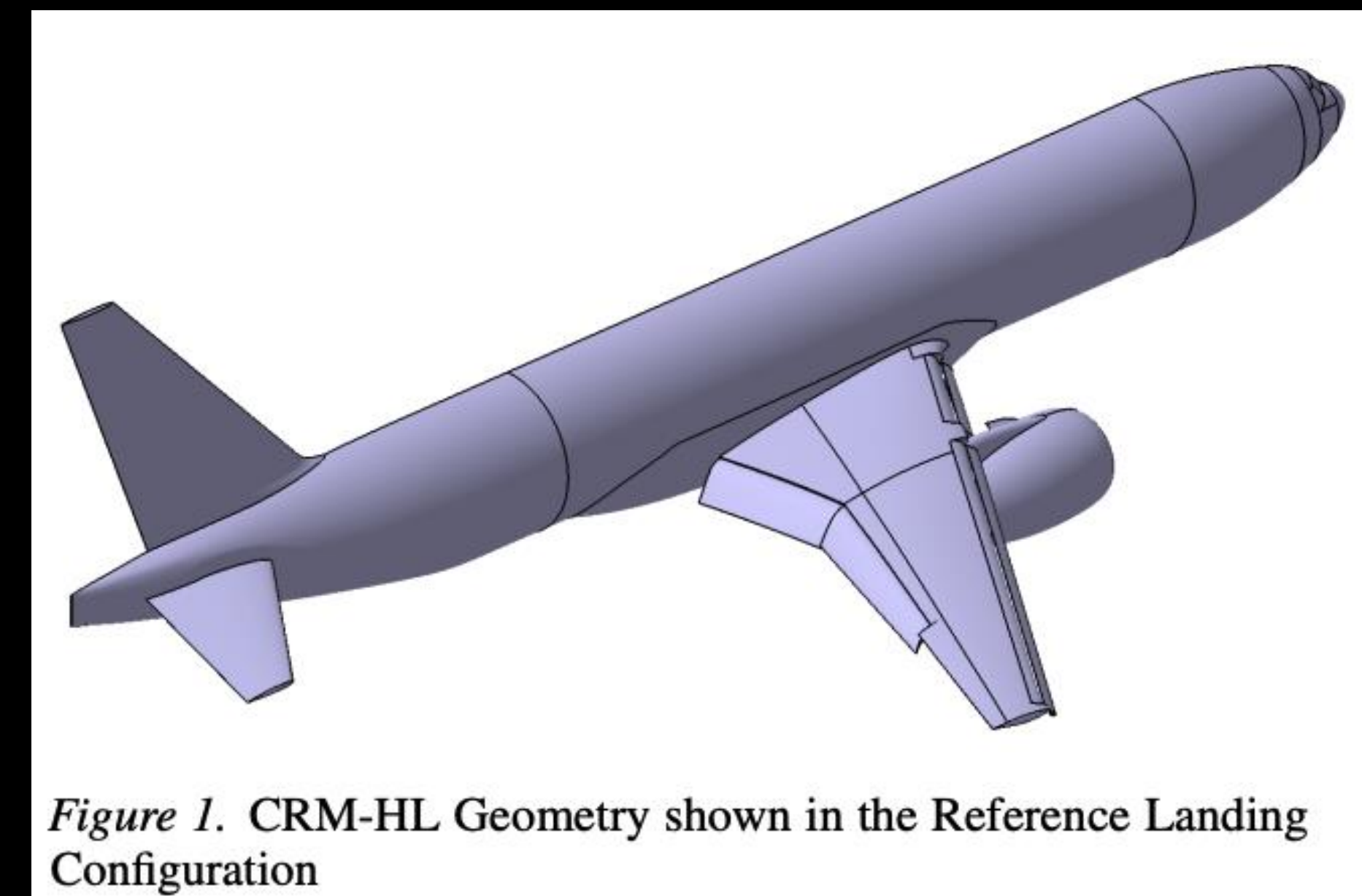


Figure 1. CRM-HL Geometry shown in the Reference Landing Configuration

AI in Computational Sciences

A High-level Overview with Primary Drivers: Data vs. Physics



Traditional Numerical Solvers

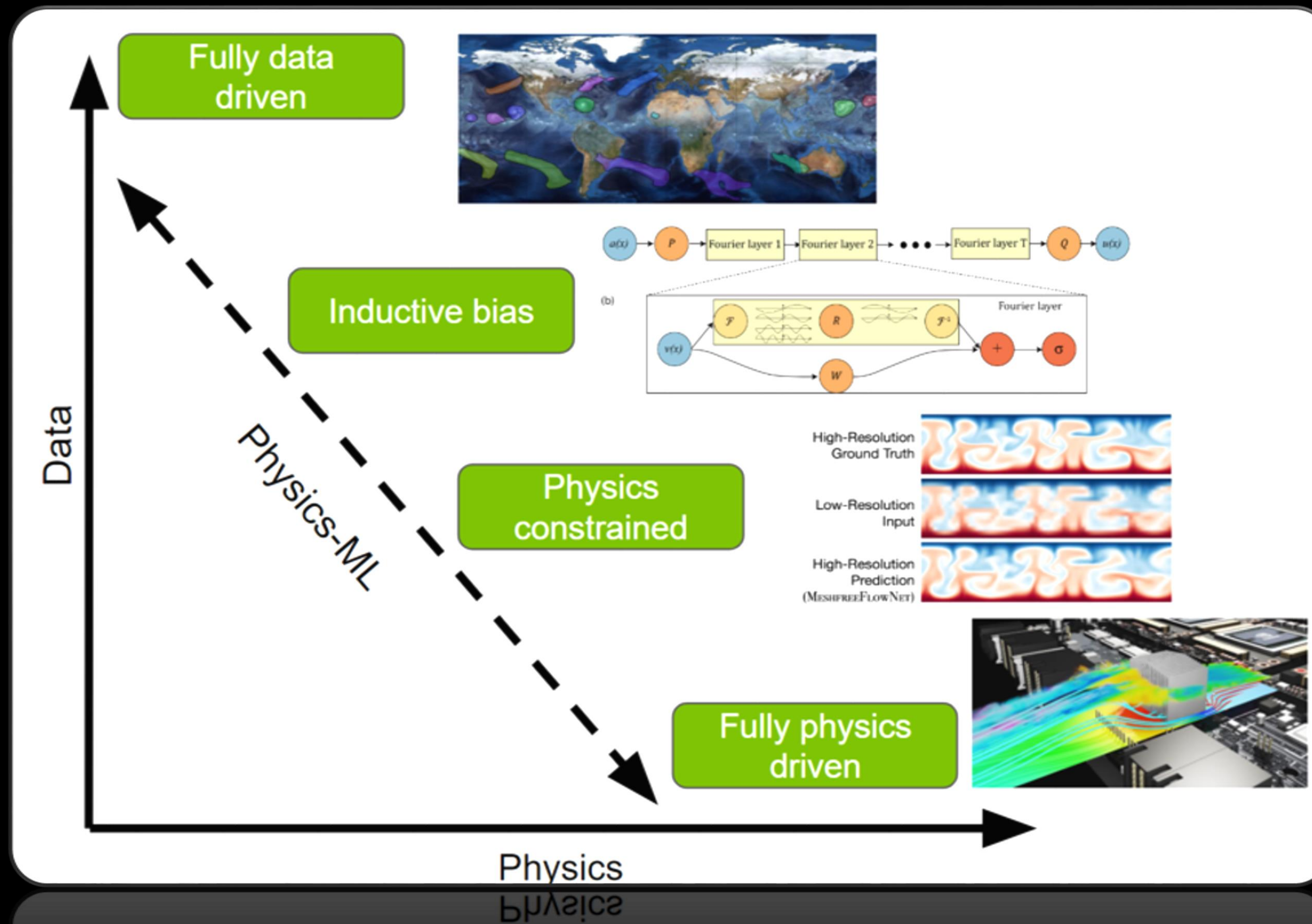
Traditional AI/ML/DL Models

PhysicsNeMo, an open-source AI Physics framework.

For examples: PINN, PINO, FNO, MeshGraphNet, DoMINO, Transolver, DiT, Geotransolver ... etc.

AI in Computational Sciences

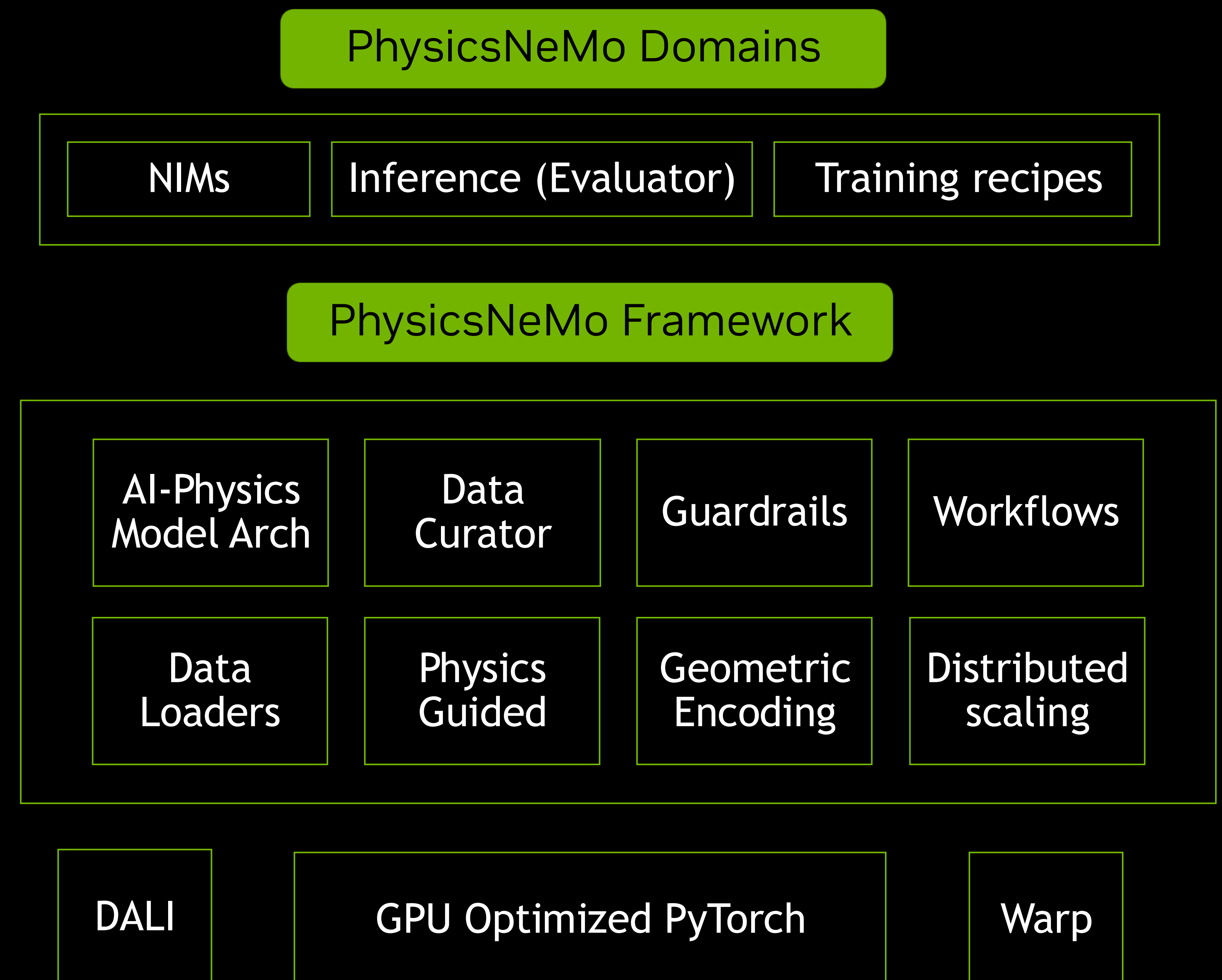
A High-level Overview with Primary Drivers: Data vs. Physics



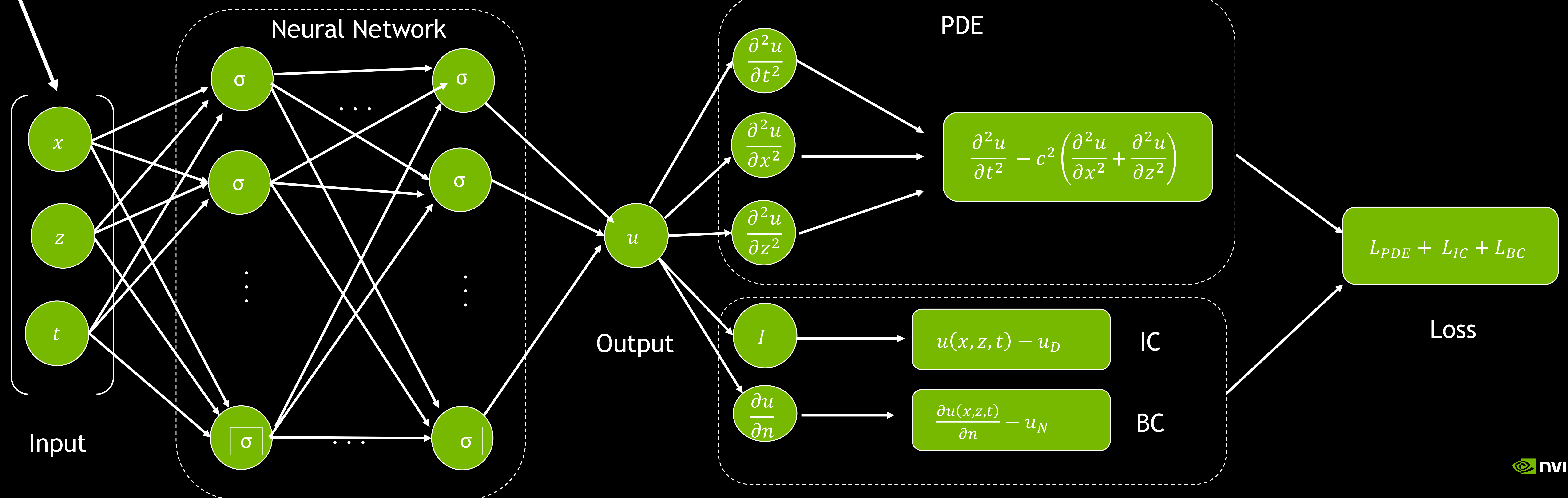
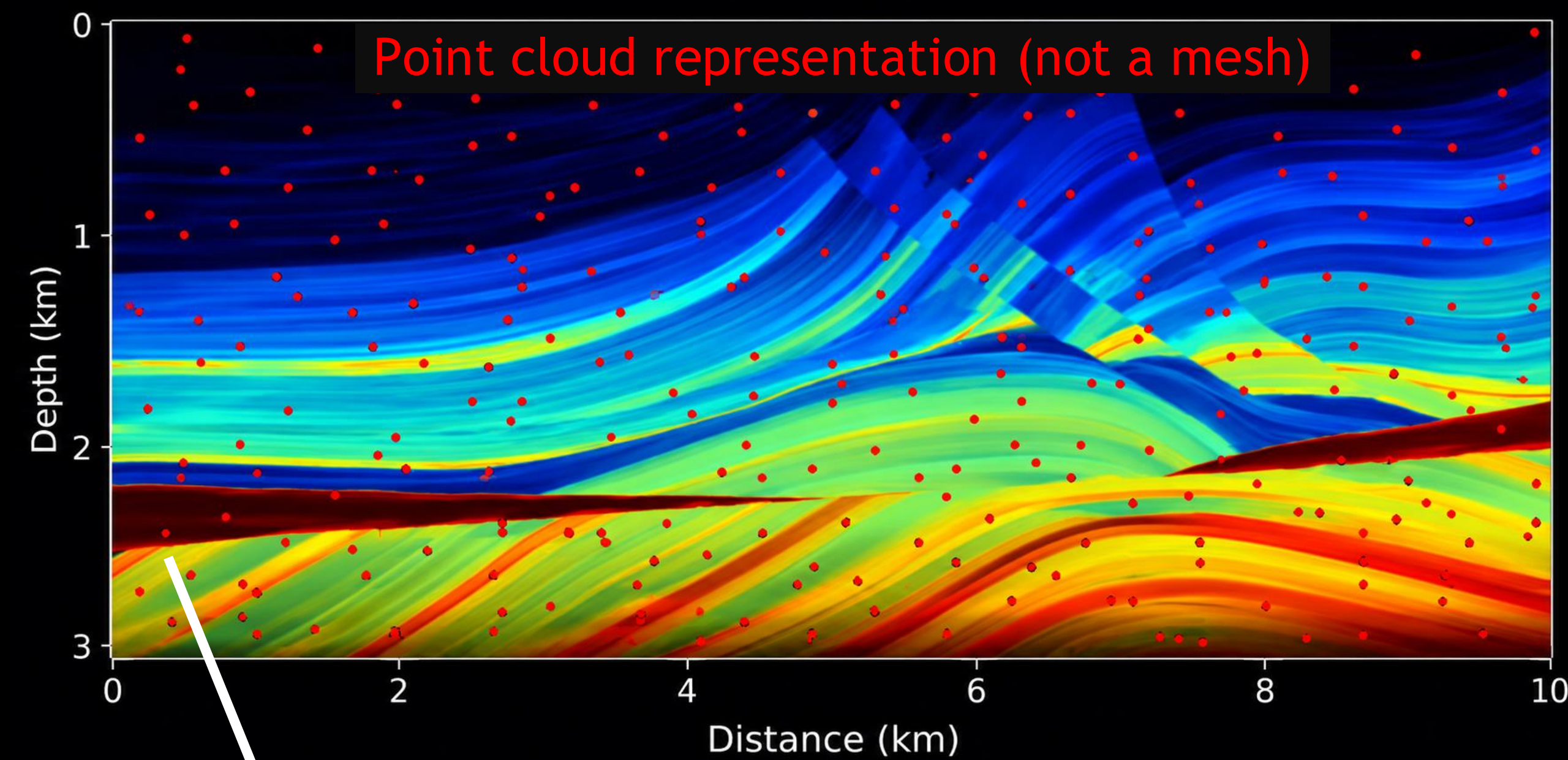
What is PhysicsNeMo?

NVIDIA PhysicsNeMo is an open-source AI Physics framework.

- Open-source, production grade PyTorch based toolkit
- For exploring, developing, training, fine-tuning and validating enterprise scale Physics AI models.
- Key Features:
 - State of the art architectures
 - Designed and validated on engineering and science use cases
 - Optimized AI Physics model blocks
 - Warp accelerated kernels
 - Complex but seamless interop between frameworks
 - SciML specific training modules
 - Optimized and specialized Datapipes for eng and science data
 - Unique Parallelism features (domain_parallelism)
 - Physics guided algorithms and utilities
 - SciML specific data curation and guardrailing

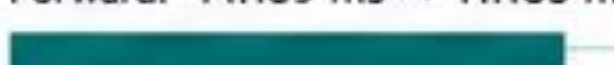


A PINN Example: Seismic Problem



Performance Reference

<https://docs.nvidia.com/physicsnemo/latest/user-guide/performance.html>
https://archive.docs.nvidia.com/physicsnemo/26.03/physicsnemo-sym/user_guide/features/performance.html (archived but worth to read)

CATEGORY	TECHNIQUE	TYPICAL SPEEDUP (vs. baseline)	BEST USED FOR / NOTES
 LARGE GAINS (~9 – 12x)	 NVIDIA Warp / Custom Kernels Example: radius-bounded aggregation	Forward: 161.396 ms → 13.511 ms  ≈ 11.9x Backward: 260.719 ms → 29.953 ms  ≈ 8.7x	Geometry, radius search, point cloud, mesh, differentiable physics kernels Highest potential gains
	 RAPIDS cuML External Kernels Example: kNN (forward/backward)	Forward: 144.045 ms → 14.139 ms  ≈ 10.2x Backward: 144.758 ms → 15.842 ms  ≈ 9.1x	kNN, neighbor search, point cloud, sparse / geometry pipelines May cause graph breaks if not registered as custom op
 MEDIUM GAINS (~2 – 4x)	 torch.compile Example: kNN (forward/backward)	Forward: 144.045 ms → 74.237 ms  ≈ 1.9x Backward: 144.758 ms → 74.657 ms  ≈ 1.9x	PyTorch-heavy models with many pointwise ops; enables fusion Effect depends on graph breaks and model
	 Meshless Finite Derivatives (MFD) Example: LDC case	 ≈ 2 – 4x (LDC example ≈ 3x)	PINNs with high-order derivatives, autodiff-heavy PDE loss dx is important; may add noise and affect convergence
 TARGETED GAINS (~20 – 50%+)	 RMM Shared Memory Pool (with cuML)	Forward: 14.139 ms → 11.185 ms  ≈ +21% Backward: 15.842 ms → 12.818 ms  ≈ +19%	PyTorch + RAPIDS mixed workloads; reduces memory allocation overhead
	 Transformer Engine LayerNorm (MeshGraphNet case)	 ≈ 1/3 runtime reduction (~33% faster)	When LayerNorm is a hotspot (> 25% of time in some cases)
	 Backend Choice: PyG vs DGL (MeshGraphNet case)	 Up to ~2x latency improvement (training iteration latency almost halves)	Graph workloads / MeshGraphNet-style training
WORKLOAD-DEPENDENT / ENABLING	 Optimized Functionals (radius_search, knn, interpolation, SDF)	Leverage accelerated backends (e.g., NVIDIA Warp)	Neighbor search, interpolation, SDF, geometry ops with optimized implementations
	 Mixed / Lower Precision (BF16 / FP16)	Potential gain (workload-dependent) Reduces memory footprint and can use Tensor Cores; validate accuracy	Tensor Core-friendly models (A100/H100 and later)

**Key Takeaway:** The biggest wins come from using the right algorithms or optimized operators (Warp, cuML). Compiler, memory, and precision optimizations bring solid additional gains. Always profile end-to-end to focus on the real bottleneck!

**Use Profilers**
PhysicsNeMo Profiler / profile / annotate

**PyTorch Profiler**
(kineto)

**Nsight Systems / Nsight Compute**

**NVTX Annotations**



**Use External Kernels**
RAPIDS cuML, NVIDIA Warp, or other optimized libraries

**Use torch.compile**
Enable fusion and reduce Python overhead

**Register as Custom Ops**
Wrap external kernels as PyTorch custom operators to avoid graph breaks & keep compile benefits

**Optimize Memory**
Enable RMM shared memory pool for PyTorch + cuML

**Use Optimized Layers**
Transformer Engine LayerNorm (or other optimized layers)

**Choose the Right Backend**
Use PyG backend (instead of DGL) for MeshGraphNet

**Leverage Lower Precision**
BF16 / FP16 where appropriate; use Tensor Cores

**Reduce Launch Overhead**
CUDA Graphs when many small kernels / launch latency bound

**Re-profile & Verify Gains**
Confirm the bottleneck has shifted

**Validate Accuracy**
Ensure numerical accuracy is maintained

**Scale Out if Needed**
Multi-GPU / Multi-Node (near ~85% scaling efficiency from 16 → 1024 GPUs in FPGA example)

Important Notes

- Reported speedups are from official docs examples on NVIDIA A100 unless noted.
- Actual speedup depends on workload size, model, hardware, data pipeline, and implementation.
- Always measure end-to-end (data loading → training step).

Reference Docs

- Profiling Guide
- torch.compile Support
- Optimized Layers

NVIDIA Special Prize



20 Years of CUDA: The Foundation of Modern Science

Every Industry | One Architecture | CUDA Everywhere

7M

Developers

600M

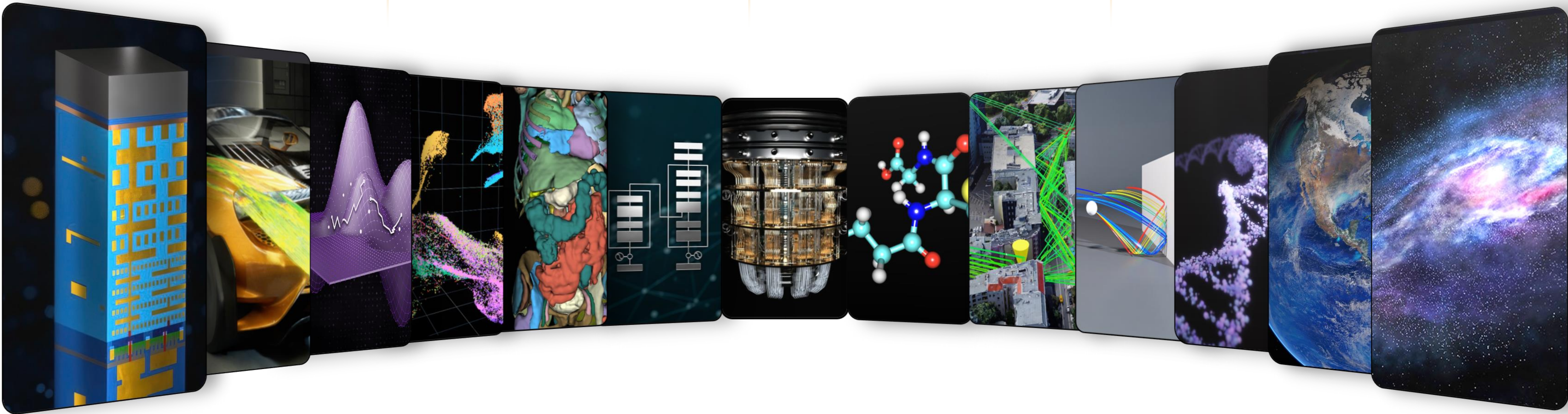
Downloads

6K

Accelerated
Applications

1K

SDKs and Models



cuLitho
Computational
Lithography

cuDSS
cuSPARSE
cuFFT
AmgX
Computer-Aided
Engineering

cuOpt
Decision
Optimization

cuDF
cuVS
cuML
cuGraph
Data Science
and Processing

MONAI
Medical
Imaging

Megatron
Dynamo
NIXL
cuDNN
CUTLASS
Deep
Learning

cuQuantum
CUDA-Q
Quantum
Computing

cuEST
cuEquivariance
cuTensor
ALCHEMI
Quantum
Chemistry

Aerial
Sionna
Holoscan
5G/6G
DAQIRI
Signal
Processing

Warp
Physics

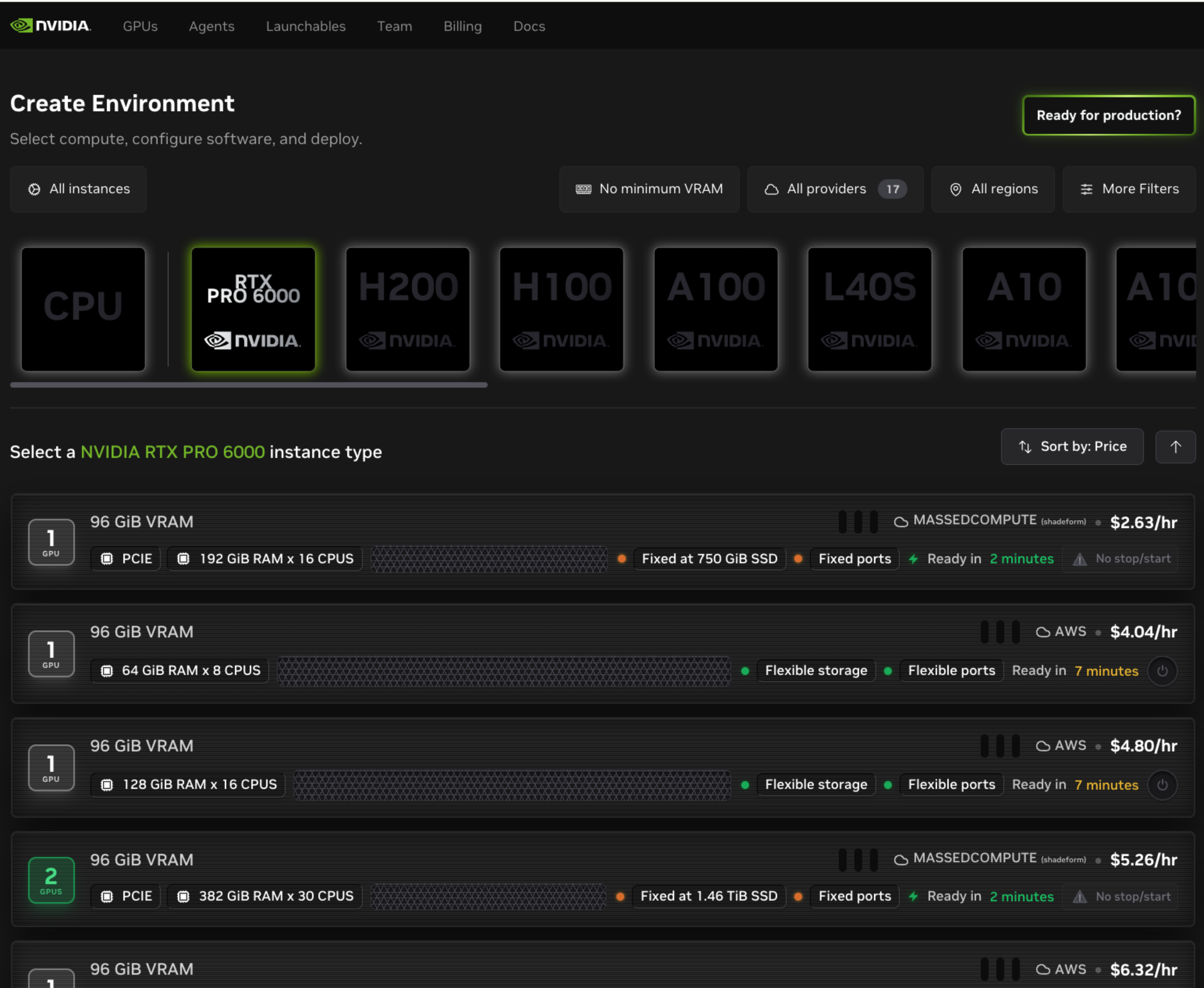
Parabricks
Genomics

Earth-2
Weather
Analytics

cuPyNumeric
cuPhoton
Numerical
Computing

NVIDIA Special Prize: US\$1,000 in NVIDIA Brev Credits for Each of the Top Three Teams Most Aligned with NVIDIA CUDA-X

<https://brev.nvidia.com/>



Thank You!

